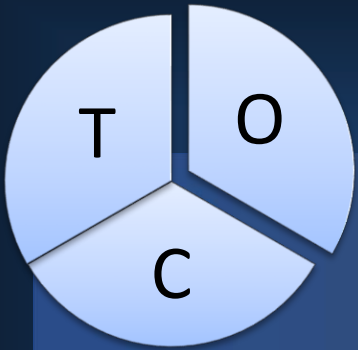
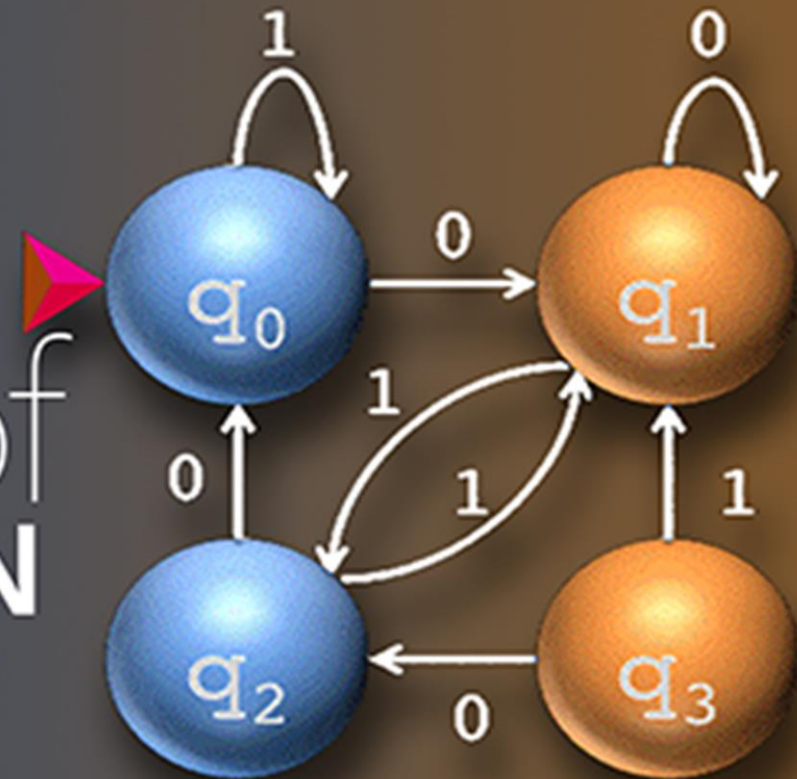




Theory of COMPUTATION



COMPILED BY:
ER. SHIVA RAM DAM

S.NO.	TITLE	PAGE NO
1.	INTRODUCTION	1
2.	FINITE AUTOMATA	11
3.	CONTEXT-FREE GRAMMAR	41
4.	PUSH DOWN AUTOMATA	61
5.	TURING MACHINE	85
6.	UNDECIDABILITY	103
7.	COMPUTATIONAL COMPLEXITY THEORY	113

1. Brief Review of Set

1.1 Definition of Set

Sets are represented as a collection of well-defined objects or elements. A set is represented by a capital letter. The number of elements in the finite set is known as the cardinal number of a set.

1.2 Elements of a Set

Let us take an example:

$$A = \{1, 2, 3, 4, 5\}$$

Since a set is usually represented by the capital letter. Thus, A is the set and 1, 2, 3, 4, 5 are the elements of the set or members of the set. The elements that are written in the set can be in any order but cannot be repeated. All the set elements are represented in small letter in case of alphabets. The cardinal number of the set is 5. Some commonly used sets are as follows:

N: Set of all natural numbers

Z: Set of all integers

Q: Set of all rational numbers

R: Set of all real numbers

Z+: Set of all positive integers

1.3 Order of Sets

The order of a set defines the number of elements a set is having. It describes the size of a set. The order of set is also known as the cardinality.

1.4 Representation of Sets

The sets are represented in curly braces, $\{\}$. For example, $\{2,3,4\}$ or $\{a,b,c\}$ or $\{\text{Bat}, \text{Ball}, \text{Wickets}\}$. The elements in the sets are depicted in either the Statement form, Roster Form or Set Builder Form.

1. Statement Form

In statement form, the well-defined descriptions of a member of a set are written and enclosed in the curly brackets.

For example, the set of even numbers less than 15.

In statement form, it can be written as {even numbers less than 15}.

2. Roster Form

In Roster form, all the elements of a set are listed.

For example, the set of natural numbers less than 5.

Natural Number = 1, 2, 3, 4, 5, 6, 7, 8,.....

Natural Number less than 5 = 1, 2, 3, 4

Therefore, the set is $N = \{1, 2, 3, 4\}$

3. Set Builder Form

The general form is, $A = \{x : \text{property}\}$

Example: Write the following sets in set builder form: $A = \{2, 4, 6, 8\}$

The set builder form is $A = \{x: x=2n, n \in N \text{ and } 1 \leq n \leq 4\}$

1.5 Types of Sets

We have several types of sets in math. They are empty set, finite and infinite sets, proper set, equal sets, etc. Let us go through the classification of sets here.

Empty Set	A set which does not contain any element is called an empty set or void set or null set. It is denoted by $\{\}$ or $\emptyset$.
Singleton Set	A set which contains a single element is called a singleton set. Example: There is only one apple in a basket of grapes.
Finite set	A set which consists of a definite number of elements is called a finite set. Example: A set of natural numbers up to 10. $A = \{1,2,3,4,5,6,7,8,9,10\}$
Infinite set	A set which is not finite is called an infinite set. Example: A set of all natural numbers. $A = \{1,2,3,4,5,6,7,8,9,.....\}$

1. INTRODUCTION

Equivalent set	If the number of elements is the same for two different sets, then they are called equivalent sets. The order of sets does not matter here. It is represented as: $n(A) = n(B) \quad \text{where } A \text{ and } B \text{ are two different sets with the same number of elements.}$ Example: If $A = \{1,2,3,4\}$ and $B = \{\text{Red, Blue, Green, Black}\}$ In set A, there are four elements and in set B also there are four elements. Therefore, set A and set B are equivalent
Equal sets	The two sets A and B are said to be equal if they have exactly the same elements, the order of elements do not matter. Example: $A = \{1,2,3,4\}$ and $B = \{4,3,2,1\}$ Here, $A = B$
Disjoint Sets	The two sets A and B are said to be disjoint if the set does not contain any common element. Example: Set $A = \{1,2,3,4\}$ and set $B = \{5,6,7,8\}$ are disjoint sets, because there is no common element between them.
Subsets	A set 'A' is said to be a subset of B if every element of A is also an element of B, denoted as $A \subseteq B$. Even the null set is considered to be the subset of another set. In general, a subset is a part of another set. Example: $A = \{1,2,3\}$ Then $\{1,2\} \subseteq A$. Similarly, other subsets of set A are: $\{1\}, \{2\}, \{3\}, \{1,2\}, \{2,3\}, \{1,3\}, \{1,2,3\}, \{\}$. If A is not a subset of B, then it is denoted as $A \not\subseteq B$.
Proper Subset	If $A \subseteq B$ and $A \neq B$, then A is called the proper subset of B and it can be written as $A \subset B$. Example: If $A = \{2,5,7\}$ is a subset of $B = \{2,5,7\}$ then it is not a proper subset of $B = \{2,5,7\}$ But, $A = \{2,5\}$ is a subset of $B = \{2,5,7\}$ and is a proper subset also.
Superset	Set A is said to be the superset of B if all the elements of set B are the elements of set A. It is represented as $A \supset B$.
Universal	For example, if set $A = \{1, 2, 3, 4\}$ and set $B = \{1, 3, 4\}$, then

Set	set A is the superset of B. A set which contains all the sets relevant to a certain condition is called the universal set. It is the set of all possible values. Example: If $A = \{1,2,3\}$ and $B = \{2,3,4,5\}$, then universal set here will be: $U = \{1,2,3,4,5\}$
Power set	The power set is a set which includes all the subsets including the empty set and the original set itself. If set $A = \{x, y, z\}$ is a set, then all its subsets $\{x\}, \{y\}, \{z\}, \{x, y\}, \{y, z\}, \{x, z\}, \{x, y, z\}$ and $\{\}$ are the elements of power set, such as: Power set of A, $P(A) = \{ \{x\}, \{y\}, \{z\}, \{x, y\}, \{y, z\}, \{x, z\}, \{x, y, z\}, \{\} \}$

1.6 Operations on Sets

In set theory, the operations of the sets are carried when two or more sets combine to form a single set under some of the given conditions. The basic operations on sets are:

- Union of sets
- Intersection of sets
- A complement of a set
- Cartesian product of sets.
- Set difference

Union of Sets	If set A and set B are two sets, then A union B is the set that contains all the elements of set A and set B. It is denoted as $A \cup B$. Example: Set $A = \{1,2,3\}$ and $B = \{4,5,6\}$, then A union B is: $A \cup B = \{1,2,3,4,5,6\}$
---------------	---

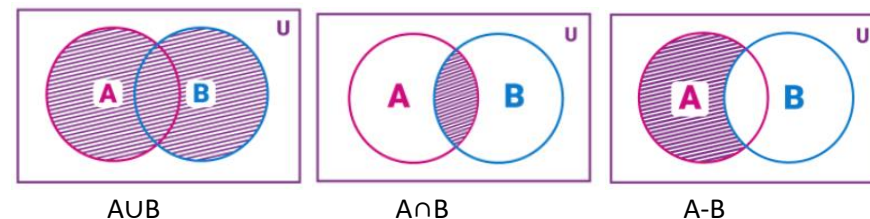
Intersection of Sets	If set A and set B are two sets, then A intersection B is the set that contains only the common elements between set A and set B. It is denoted as $A \cap B$. Example: Set $A = \{1,2,3\}$ and $B = \{4,5,6\}$, then A intersection B is: $A \cap B = \{ \}$ or $\emptyset$
Complement of Sets	The complement of any set, say P, is the set of all elements in the universal set that are not in set P. It is denoted by P'. Properties of Complement sets 1. $P \cup P' = U$ 2. $P \cap P' = \Phi$ 3. Law of double complement : $(P')' = P$ 4. Laws of empty/null set(Φ) and universal set(U), $\Phi' = U$ and $U' = \Phi$.
Cartesian Product of sets	If set A and set B are two sets then the cartesian product of set A and set B is a set of all ordered pairs (a,b), such that a is an element of A and b is an element of B. It is denoted by $A \times B$. We can represent it in set-builder form, such as: $A \times B = \{(a, b) : a \in A \text{ and } b \in B\}$ Example: set $A = \{1,2,3\}$ and set $B = \{4,5\}$, then; $A \times B = \{(1,4),(1,5),(2,4),(2,5),(3,4),(3,5)\}$ $B \times A = \{(4,1),(4,2),(4,3),(5,1),(5,2),(5,3)\}$
Difference of Sets	If set A and set B are two sets, then set A difference set B is a set which has elements of A but no elements of B. It is denoted as $A - B$. Example: $A = \{1,2,3\}$ and $B = \{2,3,4\}$ $A - B = \{1\}$

1.7 Properties of Sets

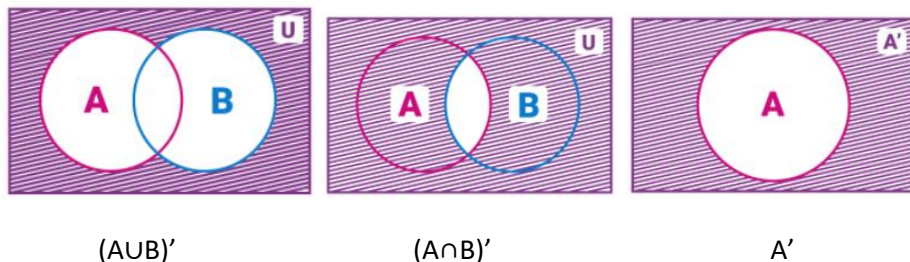
Commutative Property :	• $A \cup B = B \cup A$ • $A \cap B = B \cap A$
Associative Property :	• $A \cup (B \cap C) = (A \cup B) \cap C$ • $A \cap (B \cup C) = (A \cap B) \cup C$
Distributive Property :	• $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ • $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
De morgan's Law :	• Law of union : $(A \cup B)' = A' \cap B'$ • Law of intersection : $(A \cap B)' = A' \cup B'$
Complement Law :	• $A \cup A' = A' \cup A = U$ • $A \cap A' = \emptyset$
Idempotent Law And Law of a null and universal set :	For any finite set A • $A \cup A = A$ • $A \cap A = A$ • $\emptyset' = U$ • $\emptyset = U'$

1.8 Venn diagram

Venn diagrams are the diagrams that are used to represent the sets, relation between the sets and operation performed on them, in a pictorial way.



1. INTRODUCTION

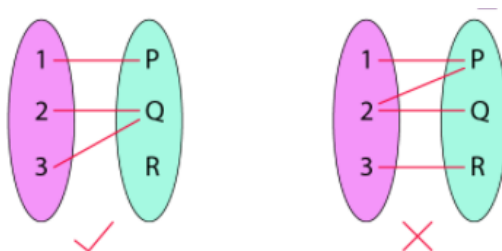


2. Function and Relation

2.1 Function

A function is a relation between a set of inputs and a set of permissible outputs with the property that each input is related to exactly one output. Let A & B be any two non-empty sets, mapping from A to B will be a function only when every element in set A has one and only one image in set B.

Example:



In case of function: for (x, y) , each x has only one y .

In case of relation: for (x, y) , one, some or all x can have more than one y .

2.2 Domain and Range

Domain:

For a function, $y = f(x)$, the set of all the values of x is called the domain of the function. It refers to the set of possible input values.

Range:

Range of $y = f(x)$ is a collection of all outputs $f(x)$ corresponding to each real number in the domain. Range is the set of all the values of y . It refers to the set of possible output values.

For example, consider the following relation.

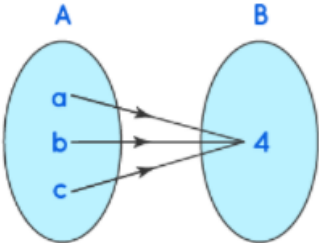
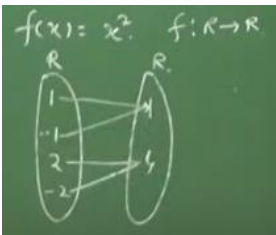
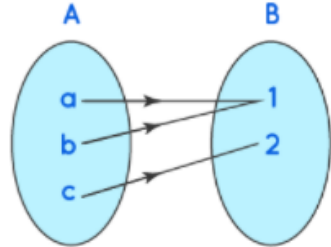
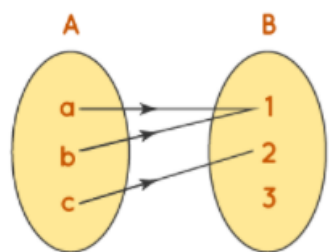
$\{(2, 3), (4, 5), (6, 7)\}$

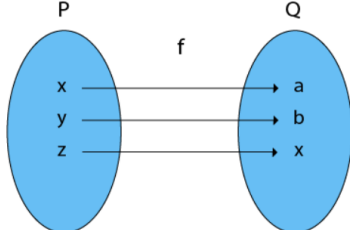
Here Domain = $\{2, 4, 6\}$

Range = $\{3, 5, 7\}$

2.3 Types of functions

Types	Details	Mapping
One-to-One Functions (Injective)	A one-to-one function is defined by $f: A \rightarrow B$ such that every element of set A is connected to a distinct element in set B.	

Many-to-One function	A many to one function is defined by the function $f: A \rightarrow B$, such that more than one element of the set A are connected to the same element in the set B. In a many to one function, more than one element has the same co-domain or image.	 
Onto Function (Surjective)	In an onto function, every codomain element is related to the domain element. For a function defined by $f: A \rightarrow B$, such that every element in set B has a pre-image in set A. The onto function is also called a surjective function.	
Into function	The into function is exactly opposite in properties to an onto function. Here there are certain elements in the co-domain that do not have any pre-image. The elements in the set B are excess and are not connected to any elements in the set A.	

One-to-One Onto Function (Bijective)	A function which is both injective (one to - one) and surjective (onto) is called bijective (One-to-One Onto) Function.	
--------------------------------------	---	---

2.4 Relation

A relation R, from a non-empty set P to another non-empty set Q, is a subset of $P \times Q$.

For example, Let $P = \{a, b, c\}$ and $Q = \{3, 4\}$ and

Let $R = \{(a, 3), (a, 4), (b, 3), (b, 4), (c, 3), (c, 4)\}$

Here R is a subset of $A \times B$. Therefore, R is a relation from P to Q.

2.5 Properties of relation

Let R be a relation on A, and let $x, y, z \in A$.

A relation R is ...	if ...
reflexive	xRx
symmetric	xRy implies yRx
transitive	xRy and yRz implies xRz
Irreflexive	xRy implies $x \neq y$
antisymmetric	xRy and yRx implies $x=y$

2.6 Relation's types

Reflexive relation	Reflexive relation is a relation of elements of a set A such that each element of the set is related to itself. Let $A = \{1, 2, 3\}$
--------------------	--

	Then $R=\{(1,1), (2,2), (3,3)\}$ is reflexive relation defined on set A.
Symmetric relation	Relation R is symmetric for all a and b in A if $(a,b) \in R$, then also $(b,a) \in R$ Let $A=\{1,2,3\}$ Then $R=\{(1,2)(2,1), (1,3) (3,1)\}$ is symmetric relation defined on set A.
Transitive relation	Relation R is transitive if for all a,b and c in A $(a,b) \in R$ and $(b,c) \in R$, then $(a,c) \in R$ Let $A=\{1,2,3\}$ Then $R=\{(1,2), (2,3), (1,3)\}$ is transitive relation defined on set A.
Equivalence relation	A relation is an Equivalence Relation if it is reflexive, symmetric, and transitive. i.e. relation $R=\{(1,1),(2,2),(3,3),(1,2),(2,1),(2,3),(3,2),(1,3),(3,1)\}$ on set $A=\{1,2,3\}$ is equivalence relation as it is reflexive, symmetric, and transitive.
Partial order relation	Let R be a relation defined on set A, then R is partial order relation if it is reflexive, anti-symmetric and transitive. Let $A=\{1,2,3\}$ Then $R=\{(1,1)(2,2), (3,3) (1,2), (2,3), (1,3)\}$ is partial order relation defined on set A.
Total order relation	A partial order relation is Total order relation if for all a and b in A, either $(a,b) \in R$ or $(b,a) \in R$. Let $A=\{1,2,3\}$ Then $R=\{(1,1)(2,2), (3,3) (2,1), (2,3), (1,3)\}$ is total order relation defined on set A.

3. Alphabets and Languages

3.1 Alphabets

Theory of computation is entirely based on symbols. These symbols are generally letters and digits.

Alphabets are defined as *a finite set of symbols*.

Examples:

$\Sigma = \{0, 1\}$ is an alphabet of binary digits

$\Sigma = \{A, B, C, \dots, Z\}$ is an alphabet.

3.2 String

A string is a finite sequence of symbols selected from some alphabet. It is generally denoted as w. For example, for alphabet $\Sigma = \{0, 1\}$ w = 010101 is a string.

Length of a string is denoted as |w| and is defined as the number of positions for the symbol in the string. For the above example length is 6.

The empty string is the string with zero occurrence of symbols. This string is represented as ϵ or λ .

The set of strings, including the empty string, over an alphabet Σ is denoted by Σ^* .

For $\Sigma = \{0, 1\}$ we have set of strings as $\Sigma^* = \{\epsilon, 0, 1, 01, 10, 00, 11, 10101, \dots\}$. and $\Sigma^1 = \{0, 1\}$, $\Sigma^2 = \{00, 01, 10, 11\}$ and so on.

3.3 Power of alphabet

If Σ is an alphabet, the set of all strings can be expressed as a certain length from that alphabet by using exponential notation. The power of an alphabet is denoted by Σ^k and is the set of strings of length k.

For example,

- $\Sigma = \{0,1\}$
- $\Sigma^1 = \{0,1\}$
- $\Sigma^2 = \{00,01,10,11\}$
- $\Sigma^3 = \{000,001,010,011,100,101,110,111\}$

3.4 Concatenation of strings

Let w_1 and w_2 be two strings then w_1w_2 denotes their concatenation w . The concatenation is formed by making a copy of w_1 and followed by a copy of w_2 .

For example, $w_1 = 001$, $w_2 = 101$
then $w = w_1w_2 = 001101$

3.5 Kleen closure

If S is a set of words then by S^* we mean the set of all finite strings formed by concatenating words from S , where any word may be used as often we like, and where the null string is also included.

S^* is the Kleen closure for S . We can think of kleen star (S^*) as an operation that makes an infinite language of strings of letters out of an alphabet

For example, for $\Sigma = \{a\}$
 $\Sigma^* = \{\epsilon, a, aa, aaa, \dots\}$

3.6 Positive closure

The set of all strings over an alphabet Σ except the empty string is called positive closure. It is denoted by Σ^+ .

$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

For example, for $\Sigma = \{a\}$
 $\Sigma^+ = \{a, aa, aaa, \dots\}$

3.7 Reversal of string

For any string W , its reversal denoted by W^R us a sting spelled backward.

Eg. $W = 1010$
 $W^R = 0101$

3.8 Language

A language is a set of string all of which are chosen from some Σ^* , where Σ is a particular alphabet. This means that language L is subset of Σ^* .

Given : $L = \{w \in \{a, b\}^* : w \text{ has odd number of } a\}$

This means : $L = \{a, ab, aaba, \dots\}$

The relation between Regular expression and the language they represent is established by a function L , such that if α is any regular expression, then $L(\alpha)$ is language represented by α .

The function L is defined as follows:

1. $L(\Phi) = \Phi$ and $L(a) = \{a\}$ for each $a \in \Sigma$
2. If α and β are regular expressions, then $L(\alpha \cup \beta) = L(\alpha) \cup L(\beta)$
3. If α and β are regular expressions, then $L(\alpha\beta) = L(\alpha) \cdot L(\beta)$
4. If α is regular expression, then $L(\alpha^*) = L(\alpha)^*$

What language is represented by following RE: $L(((a \cup b)^*a))$

Here , $L(((a \cup b)^*a))$	= $L((a \cup b)^*) L(a)$	From Rule 3
	= $L((a \cup b)^*) \{a\}$	From Rule 1
	= $L((a \cup b))^* \{a\}$	From Rule 4
	= $(L(a) \cup L(b))^* \{a\}$	From Rule 2
	= $(\{a\} \cup \{b\})^* \{a\}$	From Rule 1
	= $(a, b)^* \{a\}$	
	= $\{w \in \{a, b\}^* : w \text{ ends with an } a\}$	

3.9 Regular expressions

Regular expression is a formula representing a language in terms of a form that uses three operations : concatenation, union and kleen closure. Regular Expressions are used to denote regular languages. An expression is regular if:

- ϕ is a regular expression for regular language ϕ .
- ϵ is a regular expression for regular language $\{\epsilon\}$.

1. INTRODUCTION

- If $a \in \Sigma$ (Σ represents the input alphabet), a is regular expression with language $\{a\}$.
- If a and b are regular expression, $a + b$ is also a regular expression with language $\{a, b\}$.
- If a and b are regular expression, ab (concatenation of a and b) is also regular.
- If a is regular expression, a^* (0 or more times a) is also regular.

Regular Expressions	Regular Set
$(0 + 10^*)$	$L = \{0, 1, 10, 100, 1000, 10000, \dots\}$
(0^*10^*)	$L = \{1, 01, 10, 010, 0010, \dots\}$
$(0 + \epsilon)(1 + \epsilon)$	$L = \{\epsilon, 0, 1, 01\}$
$(a+b)^*$	Set of strings of a 's and b 's of any length including the null string. So $L = \{\epsilon, a, b, aa, ab, bb, ba, aaa, \dots\}$
$(a+b)^*abb$	Set of strings of a 's and b 's ending with the string abb . So $L = \{abb, aabb, babb, aaabb, ababb, \dots\}$
$(11)^*$	Set consisting of even number of 1's including empty string, So $L = \{\epsilon, 11, 1111, 111111, \dots\}$
$(aa)^*(bb)^*b$	Set of strings consisting of even number of a 's followed by odd number of b 's, so $L = \{b, aab, aabbb, aabbbbb, aaaab, aaaabbb, \dots\}$
$(aa + ab + ba + bb)^*$	String of a 's and b 's of even length can be obtained by concatenating any combination of the strings aa, ab, ba and bb including null, so $L = \{aa, ab, ba, bb, aaab, aaba, \dots\}$

$(aa)^*(bb)^*b$	$L = \{w \in \{a,b\}^* : w \text{ has even no. of } a \text{ followed by odd no. of } b\}$ i.e. $L = \{b, aab, aabbb, aaaab, \dots\}$
$(0U1)^*00$ Or $(0+1)^*00$	$L = \{w \in \{0, 1\}^* : w \text{ has strings of } 0 \text{ and } 1 \text{ ending in } 00\}$ $L = \{0100, 110100, 100, \dots\}$
$0(0U1)^*1$ or $0(0+1)^*1$	$L = \{w \in \{0, 1\}^* : w \text{ has strings of } 0 \text{ and } 1 \text{ beginning with } 0 \text{ and ending with } 1\}$ $L = \{01, 001, 0011, \dots\}$
$((00)^*1^*) + (01 - 0)^*$	Set of strings of 0 and 1 with even numbers of 0 i.e. $00, 001, 00, 0001, 1010, \dots$
$0^*(10^*10^*)^*10^*$	Language containing odd no. of 1 i.e. $1, 01, 01101, 0111, 111, \dots$

Given alphabet $\Sigma = \{a, b\}$, Write Regular expressions for below:

S.No.	language	Regular Expression
1.	Start with ab	$ab(a+b)^*$
2.	Start with bba	$bba(a+b)^*$
3.	Ends with abb	$(a+b)^*abb$
4.	Contains a substring aab	$(a+b)^*abb(a+b)^*$
5.	Start and ends with a	$a+a(a+b)^*a$
6.	Starts and ends with same symbol	$a+a(a+b)^*a + b + b(a+b)^*b$
7.	Starts and ends with different symbol	$a(a+b)^*b + b(a+b)^*a$
8.	$ w = 3$	$(a+b)(a+b)(a+b)$ OR $(a+b)^3$
9.	$ w \geq 3$	$(a+b)^3(a+b)^*$

10.	$ w \leq 3$	$\epsilon + (a+b) + (a+b)^2 + (a+b)^3$ OR $(a+b+\epsilon)^3$
11.	$ w _a = 2$	$b^*ab^*ab^*$
12.	$ w _a \geq 2$	$(a+b)^*a(a+b)^*(a+b)^*$
13.	$ w _a \leq 2$	$b^*(a+\epsilon)b^*(a+\epsilon)b^*$ OR $b^*+b^*ab^*+b^*ab^*ab^*$
14.	3 rd symbol from left end is b	$(a+b)^2b(a+b)^*$
15.	28 th symbol from right end is a	$(a+b)^*a(a+b)^{27}$
16.	$ w = 0 \pmod{3}$ i.e. length of string divided by 3 is 0. 0,3,6,9,.....	$((a+b)^3)^*$
17.	$ w = 2 \pmod{3}$	$a+b)^2((a+b)^3)^*$
18.	$ w _b = 0 \pmod{2}$ i.e. no of b divisible by 2	$a^* + (a^*ba^*ba^*)^*$
19.	$ w _a = 1 \pmod{3}$	$b^*ab^*(b^*ab^*ab^*ab^*)^*$
20.	$ w _b = 2 \pmod{3}$	$a^*ba^*ba^*(a^*ba^*ba^*ba^*)^*$

Describe the following sets by Regular expressions.

1.	{101}	101
2.	{abba}	Abba
3.	{01, 10}	01+10
4.	{^,ab}	^+ab
5.	{abb, a,b, bba}	abb+ a+ b+ bba
6.	{^, 0,00,000,....}	0*
7.	{1,11,111,....}	11* OR 1 ⁺

End of Chapter 1

2.1 Theory of Computation

Automata theory (aka Theory of Computation) is a theoretical branch of Computer Science and Mathematics, which mainly deals with the logic of computation with respect to simple machines, referred to as automata.

Automata* enables the scientists to understand how machines compute the functions and solve problems. The main motivation behind developing Automata Theory was to develop methods to describe and analyze the dynamic behavior of discrete systems. Automata is originated from the word “Automaton” which is closely related to “Automation”.

In theoretical computer science, the theory of computation is the branch that deals with whether and how efficiently problems can be solved on a model of computation, using an algorithm. The field is divided into three major branches: automata theory, computability theory and computational complexity theory. In order to perform a rigorous study of computation, computer scientists work with a mathematical abstraction of computers called a model of computation. There are several models in use, but the most commonly examined is the Turing machine.

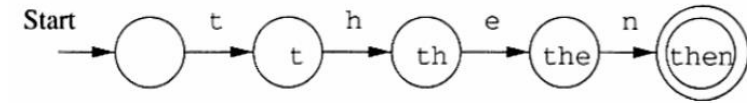
Automata theory

In theoretical computer science, automata theory is the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems) and the computational problems that can be solved using these machines. These abstract machines are called automata. This automaton consists of:

- states (represented in the figure by circles),
- and transitions (represented by arrows).

As the automaton sees a symbol of input, it makes a transition (or jump) to another state, according to its transition function (which takes the current state and the recent symbol as its inputs).

Uses of Automata: compiler design and parsing.



2.2 Finite Automata

- Finite automata are used to recognize patterns.
- It takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then the transition occurs.
- At the time of transition, the automata can either move to the next state or stay in the same state.
- Finite automata have two states, Accept state or Reject state. When the input string is processed successfully, and the automata reached its final state, then it will accept.

Formal Definition of FA

A finite automaton is a collection of 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

- Q : finite set of states
- Σ : finite set of the input symbol
- q_0 : initial state
- F : final state
- δ : Transition function

Finite Automata Model:

Finite automata can be represented by input tape and finite control.

- **Input tape:** It is a linear tape having some number of cells. Each input symbol is placed in each cell.
- **Finite control:** The finite control decides the next state on receiving particular input from input tape.
- The tape reader reads the cells one by one from left to right, and at a time only one input symbol is read.

2. FINITE AUTOMATA

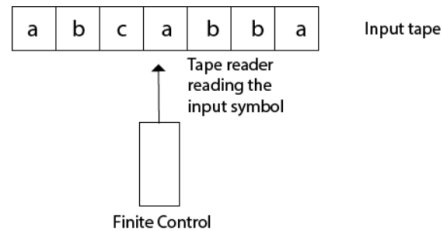


Fig - Finite automata model

Transition Diagram

A transition diagram or state transition diagram is a directed graph which can be constructed as follows:

- There is a node for each state in Q , which is represented by the circle.
- There is a directed edge from node q to node p labeled a if $\delta(q, a) = p$.
- In the start state, there is an arrow with no source.
- Accepting states or final states are indicating by a double circle.

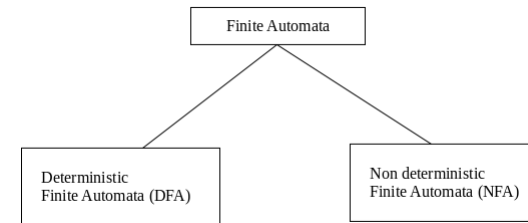
Some Notations that are used in the transition diagram:



Types of Automata:

There are two types of finite automata:

1. DFA (deterministic finite automata)
2. NFA (non-deterministic finite automata)



DFA:

DFA refers to deterministic finite automata. Deterministic refers to the uniqueness of the computation. In the DFA, the machine goes to one state only for a particular input character. DFA does not accept the null move.

NFA:

NFA stands for non-deterministic finite automata. It is used to transmit any number of states for a particular input. It can accept the null move.

Some important points about DFA and NFA:

- Every DFA is NFA, but NFA is not DFA.
- There can be multiple final states in both NFA and DFA.
- DFA is used in Lexical Analysis in Compiler.
- NFA is more of a theoretical concept.

DFA vs NDFA

The following table lists the differences between DFA and NDFA.

DFA	NFA
All transitions are deterministic.	Transitions can be non-deterministic.
Each transition leads to exactly one state.	A transition could lead to a subset of states.
For each state, transitions on all possible symbols(alphabets) should be defined	For each state, not all symbols necessarily have to be defined in the transition function.

Accepts input if the last states in in F	Accepts input if one of the last sates is in F
Sometimes harder to construct because of the numbers of states.	Generally easier thana DFA to construct.
Practical implementation is feasible.	Practical implementation has to be deterministic (so needs conversion to DFA)
Back tracking is allowed in DFA	Back tracking is not allowed in NFA
Requires more memory	Less memory

2.3 DFA

In DFA, for each input symbol, one can determine the state to which the machine will move. Hence, it is called Deterministic Automaton. As it has a finite number of states, the machine is called Deterministic Finite Machine or Deterministic Finite Automaton.

Formal Definition of a DFA

A DFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where –

- Q is a finite set of states.
- Σ is a finite set of symbols called the alphabet.
- δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

Graphical Representation of a DFA

- A DFA is represented by digraphs called state diagram.
- The vertices represent the states.
- The arcs labeled with an input alphabet show the transitions.
- The initial state is denoted by an empty single incoming arc.
- The final state is indicated by double circles.

Example

Let a deterministic finite automaton be $\rightarrow$

$$Q = \{a, b, c\},$$

$$\Sigma = \{0, 1\},$$

$$q_0 = \{a\},$$

$$F = \{c\}, \text{ and}$$

Transition function δ as shown by the following table –

Q	0	1
a	a	b
b	c	a
c	b	c

Or the transition function can be written as:

$$\delta(a, 0) \rightarrow a$$

$$\delta(a, 1) \rightarrow b$$

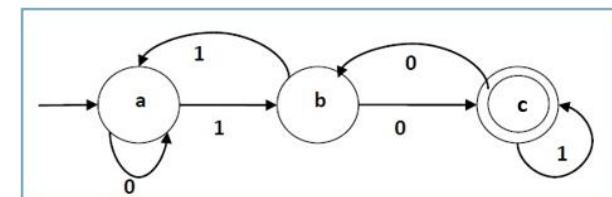
$$\delta(b, 0) \rightarrow c$$

$$\delta(b, 1) \rightarrow a$$

$$\delta(c, 0) \rightarrow b$$

$$\delta(c, 1) \rightarrow c$$

Its graphical representation would be as follows –

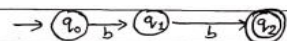


2.4 DFA Numerical Problems:

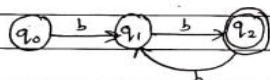
1. Design a DFA that accepts the language given by: $L = \{w \in \{a, b\}^* : w \text{ has even numbers of } b\}$

Sol: Here, $L = \{abb, bb, bbbb, bab, bba, bbabb, baababb, \dots\}$

First let us design for bb



Again design for $bbbb$



Hence the DFA is

$$M = \{Q, \Sigma, \delta, q_0, F\}$$

where,

$$Q = \{q_0, q_1, q_2\}$$

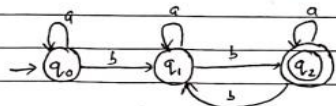
$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_2\}$$

$\delta :$	Q	a	b
	q_0	q_0	q_1
	q_1	q_1	q_2
	q_2	q_2	q_1

State diagram is:



2. Design a DFA that accepts the language given by: $L = \{w \in \{1, 0\}^* : w \text{ ends with } 100\}$

Here,

$$L = \{100, 1100, 0100, 10100, 010100, \dots\}$$

Let $M = \{Q, \Sigma, \delta, q_0, F\}$ where

$$Q = \{q_0, q_1, q_2, q_3\}$$

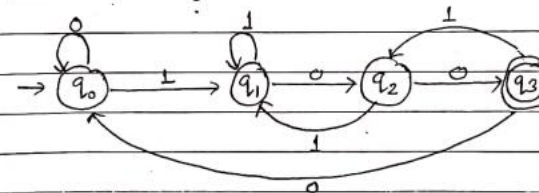
$$\Sigma = \{0, 1\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_3\}$$

δ :	Q	0	1
	q_0	q_0	q_1
	q_1	q_2	q_1
	q_2	q_3	q_1
	q_3	q_0	q_1

Hence, the state diagram is:



Verification:

10100 → Verified ends at q_3 , accepted
10101 → ends at q_1 , rejected

3. Construct a DFA starting with aba , $\Sigma = a, b$.

Here,

$$L = \{aba, abaa, abab, \dots\}$$

Let $M = \{Q, \Sigma, \delta, q_0, F\}$

where,

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

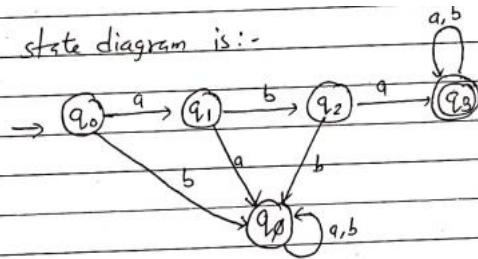
$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_3\}$$

δ	Q	a	b
	q_0	q_1	q_0
	q_1	q_0	q_2
	q_2	q_3	q_0
	q_3	q_3	q_3
	q_0	q_0	q_0

The state diagram is:-



Verification:

Accepted strings

aba

abaa

abab

Rejected strings

b

aa

abb

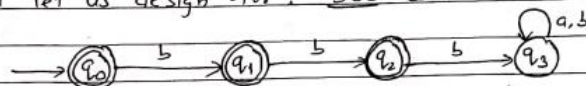
aabb

4. Design a DFA that accepts the language given by: $L = \{w \in \{a, b\}^* : w \text{ does not contain three consecutive } b\}$

Here,

$L = \{ab, aba, abb, aabb, abba, abbab, \dots\}$

First let us design for: bbb which is invalid



Let, $M = \{Q, \Sigma, \delta, q_0, F\}$

where,

$Q = \{q_0, q_1, q_2, q_3\}$

$q_0 = \{q_0\}$

$F = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

and δ :

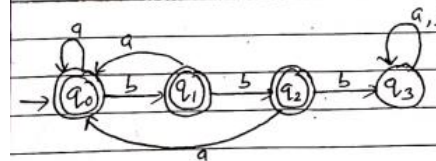
q_0 a b

q_1 a b

q_2 a b

q_3 a b

State diagram is:-



Verification:

Accepted strings

b

ab

abb

bbabb

Rejected strings

bbb

abbb

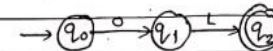
bbbab

5. Design a DFA that accepts the language given by: $L = \{w \in \{0, 1\}^* : w \text{ starts with } 01 \text{ having even length}\}$

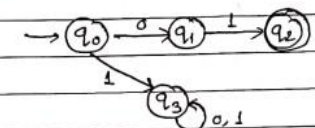
Here,

$L = \{01, 0100, 0111, 010101, 010010, \dots\}$

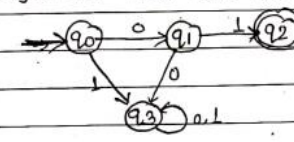
Design for 01



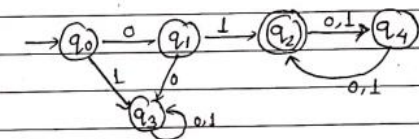
Reject for start with 1



Reject for start with 00



Finally, try for rejecting 010 or 011



2. FINITE AUTOMATA

Hence, DFA is:

$M = \{Q, \Sigma, \delta, q_0, F\}$

where,

$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{0, 1\}$

$q_0 = \{q_0\}$

$F = \{q_2\}$

and

$\delta:$

	0	1
q_0	q_1	q_3
q_1	q_3	q_2
q_2	q_4	q_4
q_3	q_3	q_3
q_4	q_2	q_2

Verification:

$0111 \Rightarrow \delta(q_0, 0111)$

$\vdash (q_1, 111)$

$\vdash (q_2, 11)$

$\vdash (q_4, 1)$

$\vdash (q_2, \epsilon)$

Since q_2 lies in final state,

DFA accepts

$010 \Rightarrow \delta(q_0, 010)$

$\vdash (q_1, 10)$

$\vdash (q_2, 0)$

$\vdash (q_4, \epsilon)$

Since q_4 is not a final state, DFA rejects

6. Design a DFA that accepts the language given by: $L = \{w \in \{0, 1\}^* : w \text{ starts with } 01\}$

Here,

$L = \{01, 0100, 010, 0111, 010111, \dots\}$

The DFA is:

$M = \{Q, \Sigma, \delta, q_0, F\}$

where,

$Q = \{$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

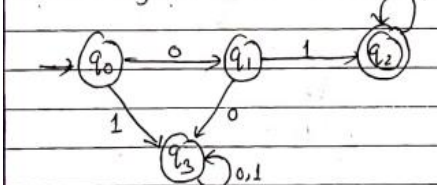
$F = \{q_2\}$

$\delta:$

	0	1
q_0	q_1	q_3
q_1	q_3	q_2
q_2	q_2	q_2
q_3	q_3	q_3

and

State diagram is:-



Verification: Accepted string

0101

$\delta(q_0, 0101)$

$\vdash (q_1, 101)$

$\vdash (q_1, 01)$

$\vdash (q_2, 1)$

$\vdash (q_2, \epsilon)$

$\vdash (q_2, \epsilon)$

Since, q_2 is final state,

DFA accepts

Rejected string

001

$\delta(q_0, 001)$

$\vdash (q_1, 01)$

$\vdash (q_3, 1)$

$\vdash (q_3, \epsilon)$

Since, q_3 is not final

state, DFA rejects

Tutorial:

Page no: 29 : Example no: 2.1 – 2.11

2.5 N DFA

n NDFA, for a particular input symbol, the machine can move to any combination of the states in the machine. In other words, the exact state to which the machine moves cannot be determined. Hence, it is called Non-deterministic Automaton. As it has finite number of states, the machine is called Non-deterministic Finite Machine or Non-deterministic Finite Automaton.

Formal Definition of an NFA

An NFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where –

- Q is a finite set of states.
- Σ is a finite set of symbols called the alphabets.
- δ is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$
(Here the power set of Q (2^Q) has been taken because in case of NFA, from a state, transition can occur to any combination of Q states)
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

Graphical Representation of an NFA: (same as DFA)

- An NFA is represented by digraphs called state diagram.
- The vertices represent the states.
- The arcs labeled with an input alphabet show the transitions.
- The initial state is denoted by an empty single incoming arc.
- The final state is indicated by double circles.

Example

Let a non-deterministic finite automaton be $\rightarrow$

$$Q = \{a, b, c\}$$

$$\Sigma = \{0, 1\}$$

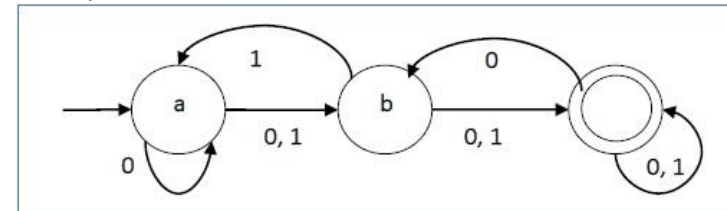
$$q_0 = \{a\}$$

$$F = \{c\}$$

The transition function δ as shown below –

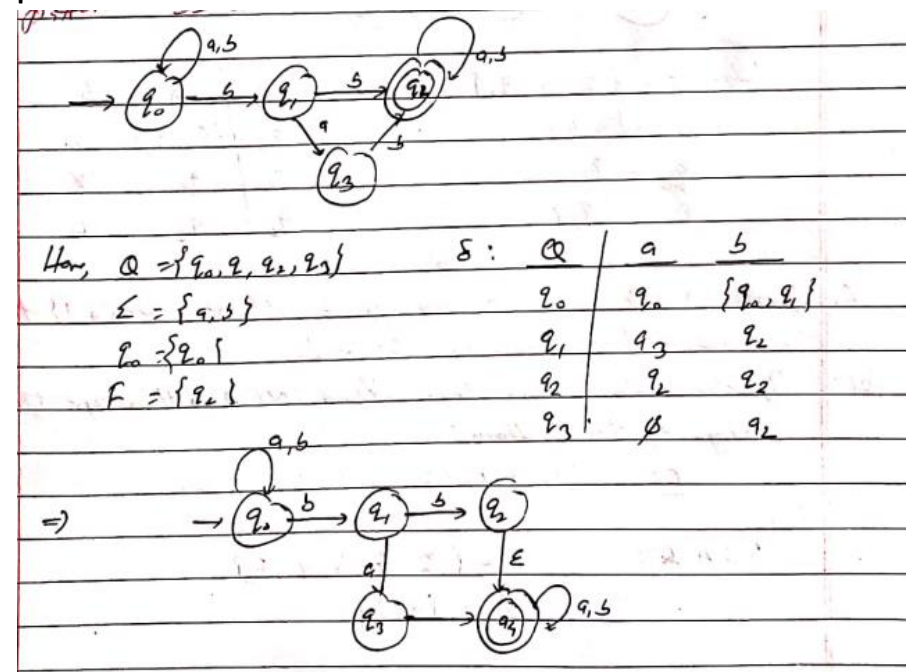
Present State	Next State for Input 0	Next State for Input 1
a	a, b	b
b	c	a, c
c	b, c	c

Its graphical representation would be as follows –



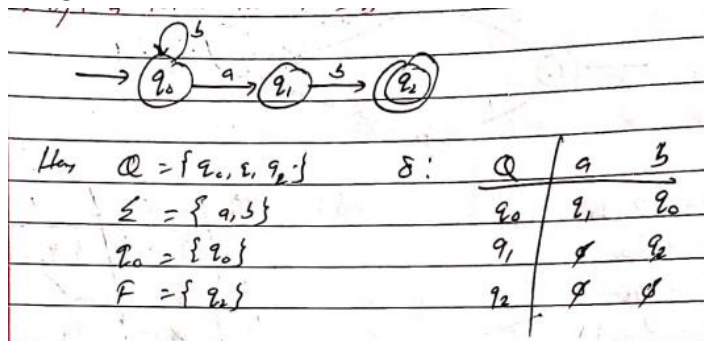
2.6 NFA Numerical Problems

1. Design a NFA that accepts the set of strings containing occurrence of pattern bb or bab.

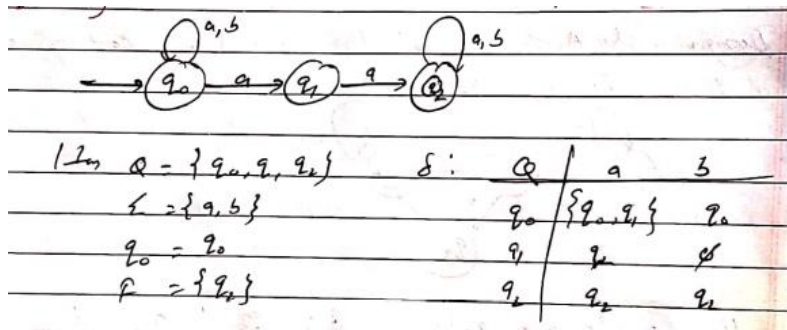


2. FINITE AUTOMATA

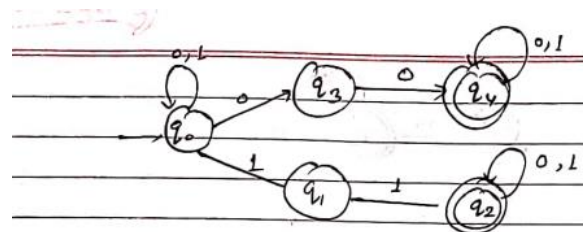
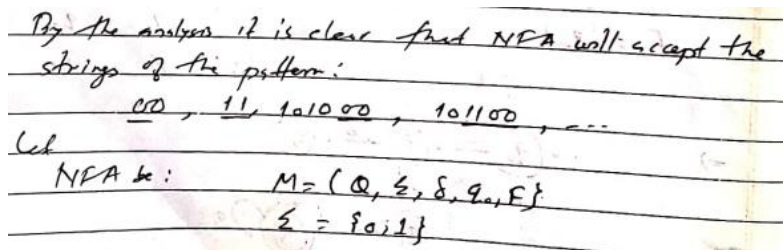
2. Design a NFA for $R = b^*ab$



3. Design a NFA over $\Sigma = \{a, b\}$ that accepts strings having aa as substring.

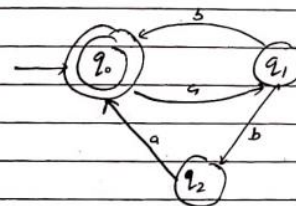


4. Design a NFA for the language $L = \text{all strings over } \{0, 1\} \text{ that have at least two consecutive 0s and 1s.}$



5. Design a NFA for the language $L = (ab \cup aba)^*$

Here, $L = (ab \cup aba)^*$ means either ab , aba or any combination of ab and aba should be accepted by the NFA, null string (ϵ) is also accepted.



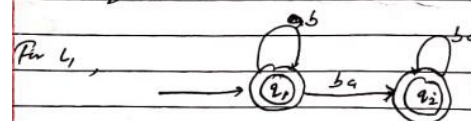
6. Draw the state diagram for NFA accepting the language $L = (ab)^*(ba)^* \cup aa^*$

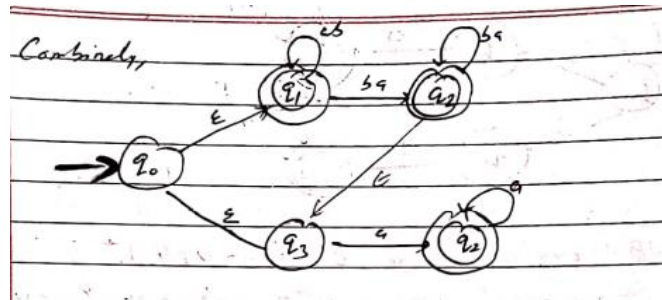
We can construct NFA for the language L in two parts i.e.

$$L = L_1 \cup L_2$$

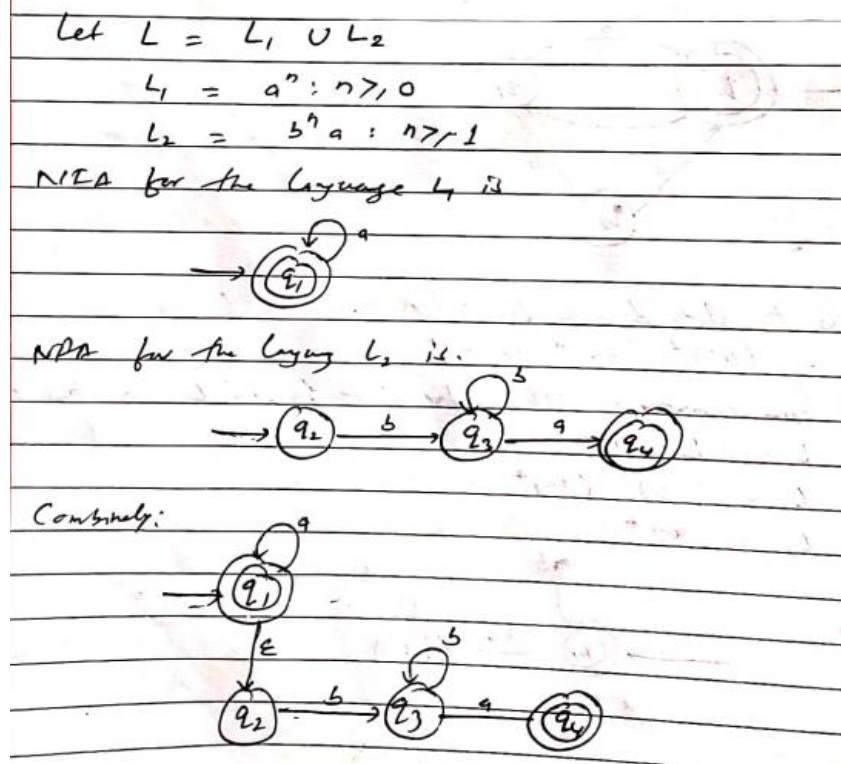
where $L_1 = (ab)^* (ba)^*$

$$L_2 = aa^*$$





7. Find NFA with four state for the language $L = \{a^n : n \geq 0\} \cup \{b^n a : n \geq 1\}$



2.7 Equivalence of DFA and NFA

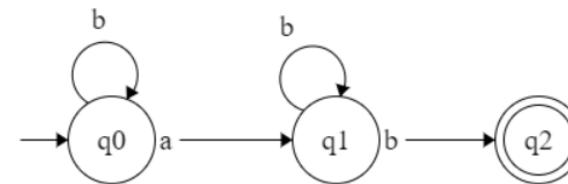
In NFA, when a specific input is given to the current state, the machine goes to multiple states. It can have zero, one or more than one move on a given input symbol. On the other hand, in DFA, when a specific input is given to the current state, the machine goes to only one state. DFA has only one move on a given input symbol. However, for each NFA, there is an equivalent DFA.

Steps for converting NFA to DFA:

1. Observe all the transitions from starting state q_0 for every symbol in alphabet (Σ).
2. For new states from step 1, check all transitions for every Σ .
3. Repeat step 2 till new states are appeared.

2.8 NFA to DFA Numerical Problems

1. Convert below NFA to DFA.



Step 1: Initial state q_0 , for new DFA $\{q_0\}$

Now,

$$\delta'(\{q_0\}, a) \Rightarrow \delta(q_0, a) = \{q_1\} \text{ new state}$$

$$\delta'(\{q_0\}, b) \Rightarrow \delta(q_0, b) = \{q_0\}$$

Step 2: For new state $\{q_1\}$

$$\delta'(\{q_1\}, a) \Rightarrow \delta(q_1, a) = \emptyset$$

$$\delta'(\{q_1\}, b) \Rightarrow \delta(q_1, b) = \{q_1, q_2\} \text{ new state.}$$

2. FINITE AUTOMATA

Step 3: For new state $\{q_1, q_2\}$

$$\delta'(\{q_1, q_2\}, a) \Rightarrow \delta(q_1, a) \cup \delta(q_2, a)$$

$$\Rightarrow \emptyset \cup \emptyset$$

$$\Rightarrow \emptyset$$

$$\delta'(\{q_1, q_2\}, b) \Rightarrow \delta(q_1, b) \cup \delta(q_2, b)$$

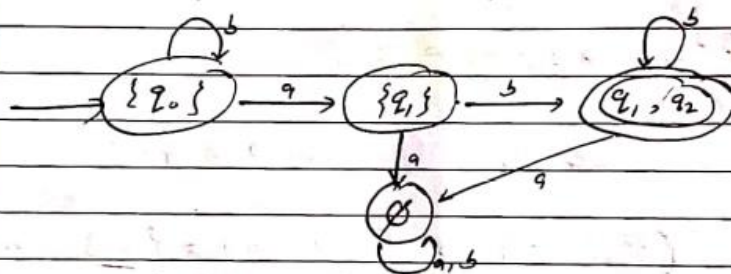
$$\Rightarrow \{q_1, q_2\} \cup \emptyset$$

$$= \{q_1, q_2\}$$

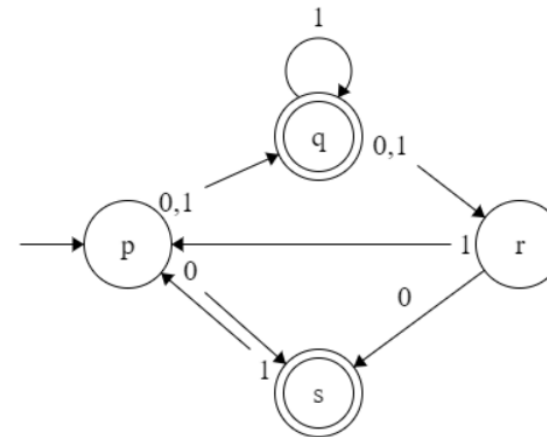
Now, transition table Δ :

Q	a	b
$\{q_0\}$	$\{q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\emptyset$	$\{q_1, q_2\}$

Hence, Equivalent DFA is:-



2. Transform below NFA to DFA:



Here, Initial state of NFA is p

For DFA, initial state is {p}

Now,

Step 1: $\delta'(\{P\}, 0) \Rightarrow \delta(\{P\}, 0) \Rightarrow \{q, s\}$ new state
 $\delta'(\{P\}, 1) \Rightarrow \delta(\{P\}, 1) \Rightarrow \{q\}$ new state

Step 2: $\delta'(\{q, s\}, 0) \Rightarrow \delta(q, 0) \cup \delta(s, 0)$
 $\Rightarrow \{r\} \cup \emptyset$
 For $\{q, s\}$ $\Rightarrow \{r\}$ new state
 $\delta'(\{q, s\}, 1) \Rightarrow \delta(q, 1) \cup \delta(s, 1)$
 $\Rightarrow \{q, r\} \cup \{P\}$
 $\Rightarrow \{q, r, P\}$ new state

Step 3: For $\{q\}$
 $\delta'(\{q\}, 0) \Rightarrow \delta(q, 0) \Rightarrow \{r\}$ new state
 $\delta'(\{q\}, 1) \Rightarrow \delta(q, 1) \Rightarrow \{q, r\}$ new state

Step 4: For $\{r\}$
 $\delta'(\{r\}, 0) \Rightarrow \delta(r, 0) \Rightarrow \{s\}$ new state
 $\delta'(\{r\}, 1) \Rightarrow \delta(r, 1) \Rightarrow \{P\}$

Step 5: For $\{q, r, P\}$
 $\delta'(\{q, r, P\}, 0) \Rightarrow \delta(q, 0) \cup \delta(r, 0) \cup \delta(P, 0)$
 $\Rightarrow \{r\} \cup \{s\} \cup \{q, s\}$
 $\Rightarrow \{r, s, q\}$ new state
 $\delta'(\{q, r, P\}, 1) \Rightarrow \delta(q, 1) \cup \delta(r, 1) \cup \delta(P, 1)$
 $\Rightarrow \{q, r\} \cup \{P\} \cup \{q\}$
 $\Rightarrow \{q, r, P\}$

Step 6: For $\{q, r\}$
 $\delta'(\{q, r\}, 0) \Rightarrow \delta(q, 0) \cup \delta(r, 0)$
 $\Rightarrow \{r\} \cup \{s\}$
 $\Rightarrow \{r, s\}$ new state

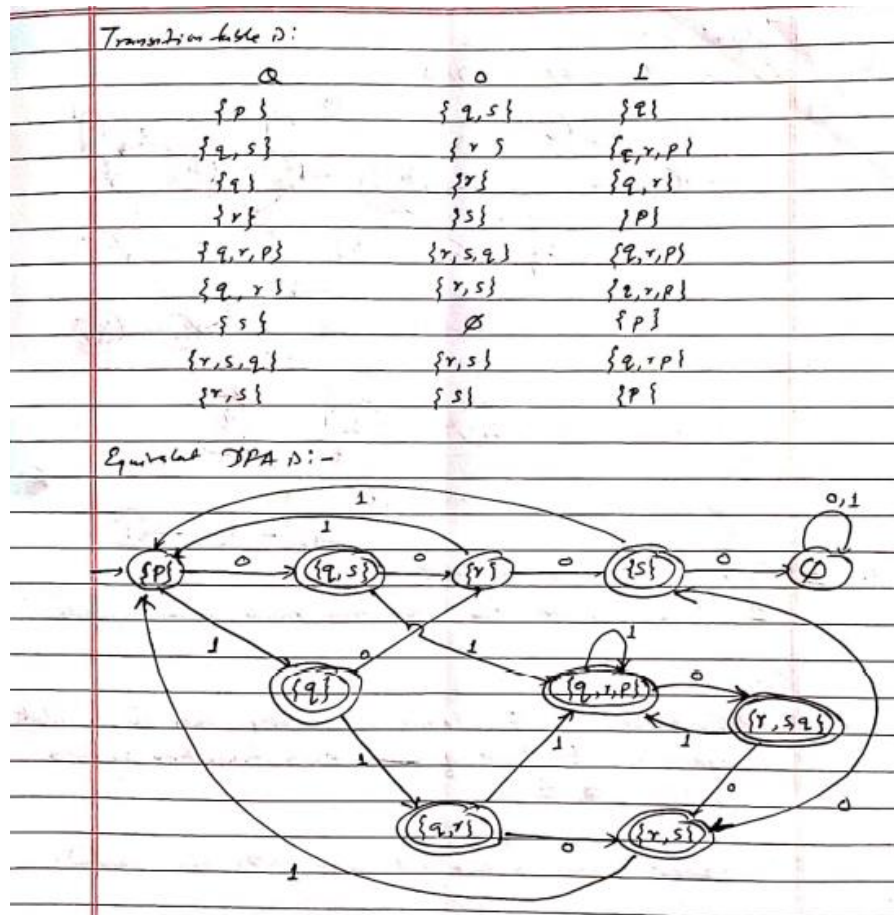
$\delta'(\{q, r\}, 1) \Rightarrow \delta(q, 1) \cup \delta(r, 1)$
 $\Rightarrow \{q, r\} \cup \{P\}$
 $\Rightarrow \{q, r, P\}$

Step 7: For $\{s\}$
 $\delta'(\{s\}, 0) \Rightarrow \delta(s, 0) = \emptyset$
 $\delta'(\{s\}, 1) \Rightarrow \delta(s, 1) = \{P\}$

Step 8: For $\{r, s, q\}$
 $\delta'(\{r, s, q\}, 0) \Rightarrow \delta(r, 0) \cup \delta(s, 0) \cup \delta(q, 0)$
 $\Rightarrow \{s\} \cup \{P\} \cup \{r\}$
 $\Rightarrow \{r, s\}$
 $\delta'(\{r, s, q\}, 1) \Rightarrow \delta(r, 1) \cup \delta(s, 1) \cup \delta(q, 1)$
 $\Rightarrow \{P\} \cup \{P\} \cup \{q, r\}$
 $\Rightarrow \{q, r, P\}$

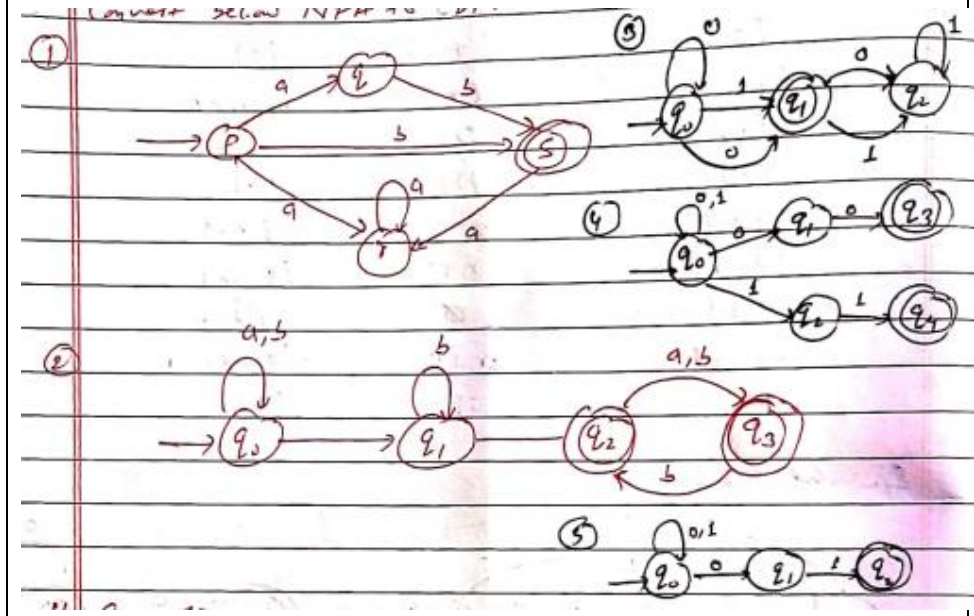
Step 9: For $\{r, s\}$
 $\delta'(\{r, s\}, 0) \Rightarrow \delta(r, 0) \cup \delta(s, 0)$
 $\Rightarrow \{s\} \cup \emptyset$
 $\Rightarrow \{s\}$
 $\delta'(\{r, s\}, 1) \Rightarrow \delta(r, 1) \cup \delta(s, 1)$
 $\Rightarrow \{P\} \cup \{P\}$
 $\Rightarrow \{P\}$

2. FINITE AUTOMATA



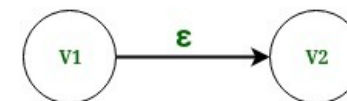
Tutorial:

Convert below NFA to DFA:



2.9 ϵ -NFA

The NFA with epsilon-transition is a finite state machine in which the transition from one state to another state is allowed without any input symbol i.e. empty string ϵ . Here in the figure below, State V1 and V2 have an epsilon move.



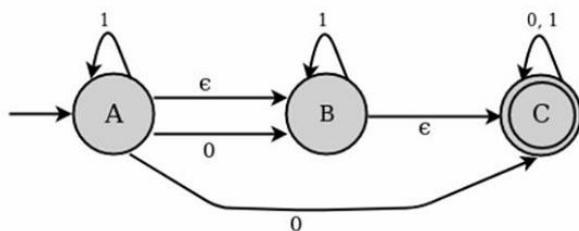
Epsilon (ϵ) - closure

Epsilon closure for a given state X is a set of states which can be reached from the states X with only (null) or ϵ moves including the state X itself.

In other words, ϵ -closure for a state can be obtained by union operation of the ϵ -closure of the states which can be reached from X with a single ϵ move in a recursive manner.

Example

Consider the following figure of NFA with ϵ move –



The transition state table for the above NFA is as follows –

State	0	1	epsilon
A	B, C	A	B
B	-	B	C
C	C	C	-

For the above example, ϵ closure are as follows –

E closure(A) : {A, B,C}

E closure(B) : {B,C}

E closure(C) : {C}

Converting E-NFA to DFA**Steps:**

1. Find out the ϵ -closure of starting state, and calculate union of ϵ -closure of each transition for every symbol of alphabet (Σ).
2. Repeat same process till new states are approved.

Here, ϵ -closure of state q can be obtained by following all transitions out of q the are labelled ϵ .

Numerical:**1. Convert the below ϵ -NFA to its equivalent DFA.**

Here, let us obtain ϵ -closure of each state.

ϵ -closure of q_0 is $E(q_0) = \{q_0, q_1, q_2\}$

ϵ -closure of q_1 is $E(q_1) = \{q_1, q_2\}$

ϵ -closure of q_2 is $E(q_2) = \{q_2\}$

1) Now, for state $\{q_0, q_1, q_2\}$

$$\delta'(\{q_0, q_1, q_2\}, a) = E\{\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)\}$$

$$= E(q_0) \cup \emptyset \cup E(q_2)$$

$$= \{q_0, q_1, q_2\} \cup \emptyset \cup \{q_2\}$$

$$= \{q_0, q_1, q_2\}$$

$$\delta'(\{q_0, q_1, q_2\}, b) = E\{\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b)\}$$

$$= \emptyset \cup E(q_1) \cup \emptyset$$

$$= \{q_1, q_2\} \Rightarrow \text{new state}$$

2. FINITE AUTOMATA

2) For state $\{q_1, q_2\}$

$$\begin{aligned}\delta'(\{q_1, q_2\}, a) &= \epsilon \{ \delta(q_1, a) \cup \delta(q_2, a) \} \\ &= \emptyset \cup \epsilon(q_2) \\ &= \{q_2\} \Rightarrow \text{new state}\end{aligned}$$

$$\begin{aligned}\delta'(\{q_1, q_2\}, b) &= \epsilon \{ \delta(q_1, b) \cup \delta(q_2, b) \} \\ &= \epsilon(q_1) \cup \emptyset \\ &= \{q_1, q_2\}\end{aligned}$$

3) For state $\{q_2\}$:

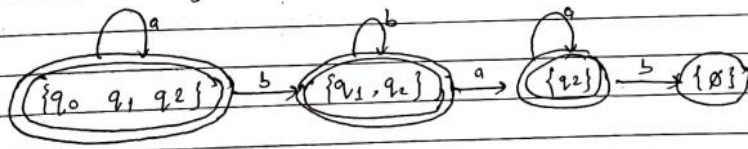
$$\begin{aligned}\delta'(\{q_2\}, a) &= \epsilon \{ \delta(q_2, a) \} \\ &= \epsilon(q_1) \\ &= \{q_1\}\end{aligned}$$

$$\begin{aligned}\delta'(\{q_2\}, b) &= \epsilon \{ \delta(q_2, b) \} \\ &= \emptyset\end{aligned}$$

Hence, transition table is:-

	$\emptyset$	a	b
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
$\{q_2\}$	$\{q_1\}$	$\emptyset$	$\emptyset$

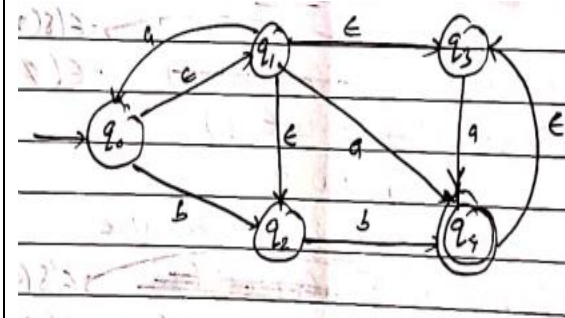
The state diagram of DFA is:-



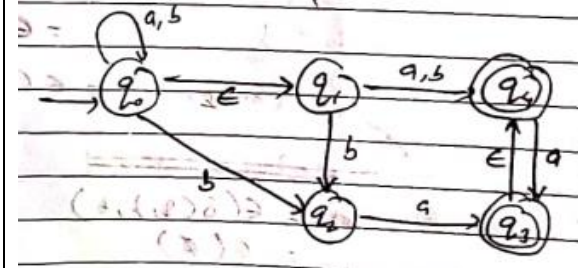
Tutorial:

Convert below ϵ -NFA to its equivalent DFA.

1)



2)



2.10 State Minimization (DFA Minimization)

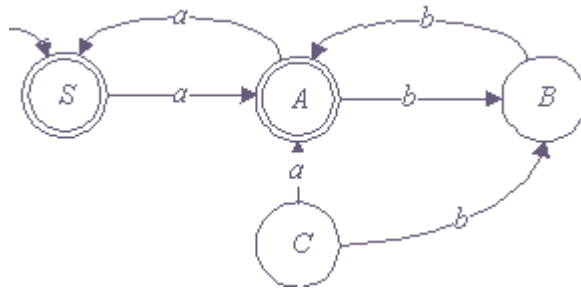
DFA minimization stands for converting a given DFA to its equivalent DFA with minimum number of states. Minimization of DFA thus reduces the number of states from given FA. Thus, we get the FSM (finite state machine) with redundant states after minimizing the FSM.

In this process we minimize the given DFA by removing unreachable states and by merging similar equivalent states.

Inaccessible states:

All the states which can never be reached from initial state are inaccessible/unreachable states. In contrast, Dead states are the states from which no final state is reachable.

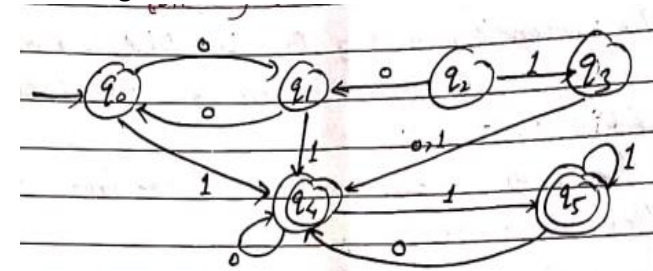
For eg:



Here, state C is never reached from the initial state S. So state C is unreachable state.

State minimization steps:

1. Remove unreachable states from given DFA.
2. Construct a transition table for rest of the states.
3. Divide the transition table of step 2 into two parts:
 - a) Rows that start with final states.
 - b) Rows that start with non-final states.
4. Eliminate one of the equivalent/similar states from table : 3a and 3b respectively.
5. Repeat step 4 until equivalent states are appeared.
6. Combine/merge updated tables.
7. Draw minimized state diagram.

2.11 Numerical**1. Minimize following DFA.**

Sol:

1. Here, q_2 and q_3 are unreachable states, so remove them.

2. Transition table for rest of states:-

Q	0	1
q_0	q_1	q_4
q_1	q_0	q_4
q_4	q_4	q_5
q_5	q_4	q_5

3.

Transition table with non-final states

Q	0	1
q_0	q_1	q_4
q_1	q_0	q_4

4. Transition table with final states

Q	0	1
q_4	q_4	q_5
q_5	q_4	q_5

2. FINITE AUTOMATA

4. Eliminate equivalent/similar rows in state table q_3 and q_5

& we have 3 (s) as:

$$(q_4 = q_5)$$

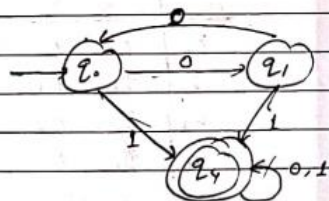
Q	0	1
q_1	q_4	q_4

5. Merge updated table: we get.

Q	0	1
q_0	q_1	q_4
q_1	q_0	q_4
q_4	q_4	q_4

$$\therefore (q_4 = q_5)$$

6. Final minimized DFA is:-



Test: 001001

$\Rightarrow \delta(q_0, 001001)$

$\vdash (q_1, 01001)$

$\vdash (q_0, 1001)$

$\vdash (q_4, 001)$

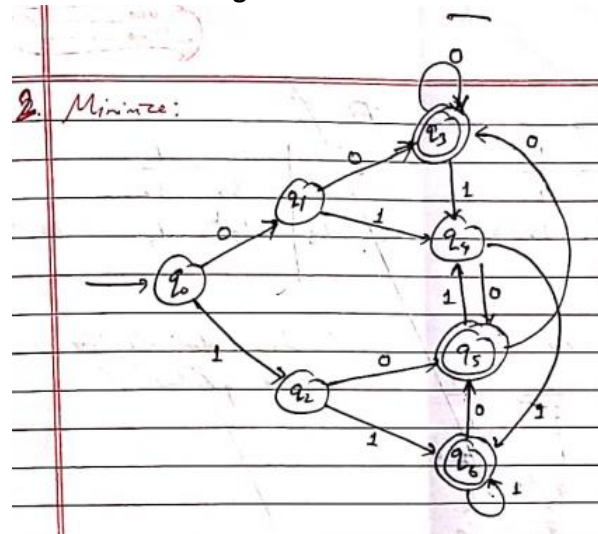
$\vdash (q_4, 01)$

$\vdash (q_4, 1)$

$\vdash (q_4, \epsilon)$

$\therefore q_4$ is in final state, & DFA accepted.

2. Minimize following DFA



Solution:

1. No unreachable states.

2. Transition table:

Q	0	1
q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_5	q_6
q_3	q_3	q_4
q_4	q_5	q_6
q_5	q_5	q_4
q_6	q_5	q_6

3. Splitting into transition tables of Non-final and Final states,

a) Non-final ^{state} Transition table

Q	0	1
q ₀	q ₁	q ₂
q ₁	q ₃	q ₄
q ₂	q ₅	q ₆
q ₄	q ₅	q ₆

b) Final ^{state} Transition table

Q	0	1
q ₃	q ₃	q ₄
q ₅	q ₃	q ₄
q ₆	q ₅	q ₆

4. Eliminating one of the equivalent rows.

a)

Q	0	1
q ₀	q ₁	q ₂
q ₁	q ₃	q ₄
q ₂	q ₅	q ₆
q ₄	q ₅	q ₆

Here, Now, q₂ = q₄

↓

Q	0	1
q ₀	q ₁	q ₂
q ₁	q ₃	q ₂
q ₂	q ₃	q ₆

b)

Q	0	1
q ₃	q ₃	q ₄
q ₅	q ₃	q ₄
q ₆	q ₅	q ₆

Here, Now, q₃ = q₅

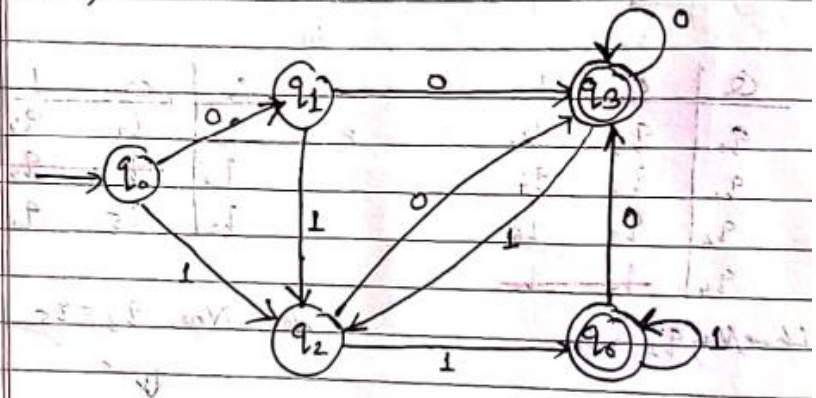
↓

Q	0	1
q ₃	q ₃	q ₂
q ₆	q ₃	q ₆

5. Merging the tables,

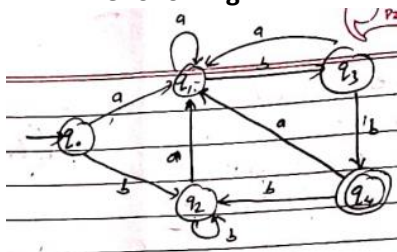
Q	0	1
q ₀	q ₁	q ₂
q ₁	q ₃	q ₂
q ₂	q ₃	q ₆
q ₃	q ₃	q ₂
q ₆	q ₃	q ₆

6. Hence, minimized DFA is:-



2. FINITE AUTOMATA

3. Minimize following DFA.



Solution:

1. No unreachable states

2. Transition table:

Q	a	b
q ₀	q ₁	q ₂
q ₁	q ₁	q ₃
q ₂	q ₄	q ₂
q ₃	q ₁	q ₄
q ₄	q ₁	q ₄

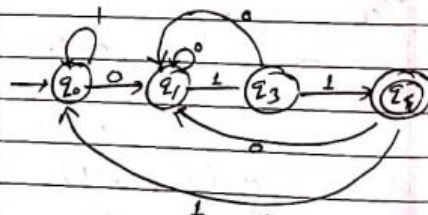
4. Eliminate equivalent rows q₀ and q₂

Q	a	b
q ₀	q ₁	q ₀

5. Merge table:

Q	a	b
q ₀	q ₁	q ₀
q ₁	q ₁	q ₃
q ₃	q ₁	q ₄
q ₄	q ₁	q ₀

6. State diagram is:



b) Final

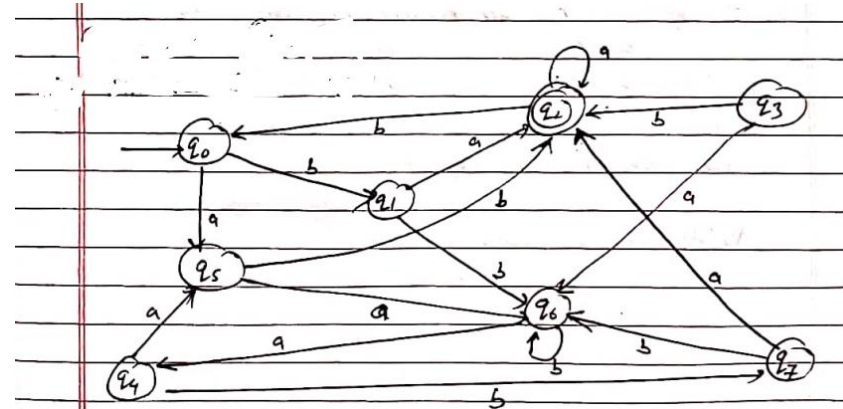
Q	a	b
q ₄	q ₁	q ₀

Tutorial:

1. Minimize following DFA by using State Minimization method, where: → represents initial state and * represents final state.

Q \ Q	0	1
→ q ₀	q ₁	q ₂
* q ₁	q ₁	q ₃
q ₂	q ₂	q ₂
* q ₃	q ₅	q ₂
* q ₄	q ₄	q ₂
* q ₅	q ₄	q ₂
q ₆	q ₅	q ₆
q ₇	q ₅	q ₆

2. Minimize below DFA:



2.12 Regular Expressions and Regular Language

Regular Expression

- The language accepted by finite automata can be easily described by simple expressions called Regular Expressions. It is the most effective way to represent any language.
- The languages accepted by some regular expression are referred to as Regular languages.
- A regular expression can also be described as a sequence of pattern that defines a string.
- Regular expressions are used to match character combinations in strings. String searching algorithm used this pattern to find the operations on a string.
- For instance: In a regular expression, x^* means zero or more occurrence of x . It can generate $\{e, x, xx, xxx, xxxx, \dots\}$
- In a regular expression, x^+ means one or more occurrence of x . It can generate $\{x, xx, xxx, xxxx, \dots\}$

Definition of Regular Expression:

The set of regular expression is defined by the following rules”

- Every letter of Σ can be made into a regular expression, null string, ϵ itself is a regular expression.
- If r_1 and r_2 are regular expression, then:
 - (r_1)
 - $r_1 r_2$
 - $r_1 + r_2$
 - r_1^*
 - r_1^+ are also regular expression.
- Nothing else is regular expression.

Regular Language:

A language is regular if it can be expressed in terms of regular expression.

Regular Expression	Regular language
a	{a}
B	{b}
a+b	{a,b}
a·b	{ab}
a*a*	{ ϵ , a, aa, aaa, aaaa,
b+b+	{b, bb, bbb, bbbb,

2.13 Inductive Steps

There are four parts to the inductive step, one for each of three operators and one for the introduction of parentheses.

- If r_1 and r_2 are regular expressions, then $r_1 + r_2$ is a regular expression denoting the union of $L(r_1)$ and $L(r_2)$.
i.e. $L(r_1 + r_2) = L(r_1) \cup L(r_2)$
- If r_1 and r_2 are regular expressions, then $r_1 r_2$ is a regular expression denoting the concatenation of $L(r_1)$ and $L(r_2)$.
i.e. $L(r_1 r_2) = L(r_1) \cdot L(r_2)$
- If r is a regular expressions, then r^* is a regular expression denoting the closure of $L(r)$.
i.e. $L(r^*) = (L(r))^*$
- If r is a regular expressions, then (r) , a parenthesized r , is also a regular expression denoting the same language as r .
i.e. $L((r)) = L(r)$

2.14 Application of Regular language

1. Validation:	Determines that a string compiles a set of formatting constraints like email validation, password validation, etc.
2. Search and Selection:	Identifying a subset of items from larger set on the basis of a pattern match
3. Tokenization:	Converting a sequence of characters into words, tokens for later interpretation.

2. FINITE AUTOMATA

2.15 Algebraic laws for RE

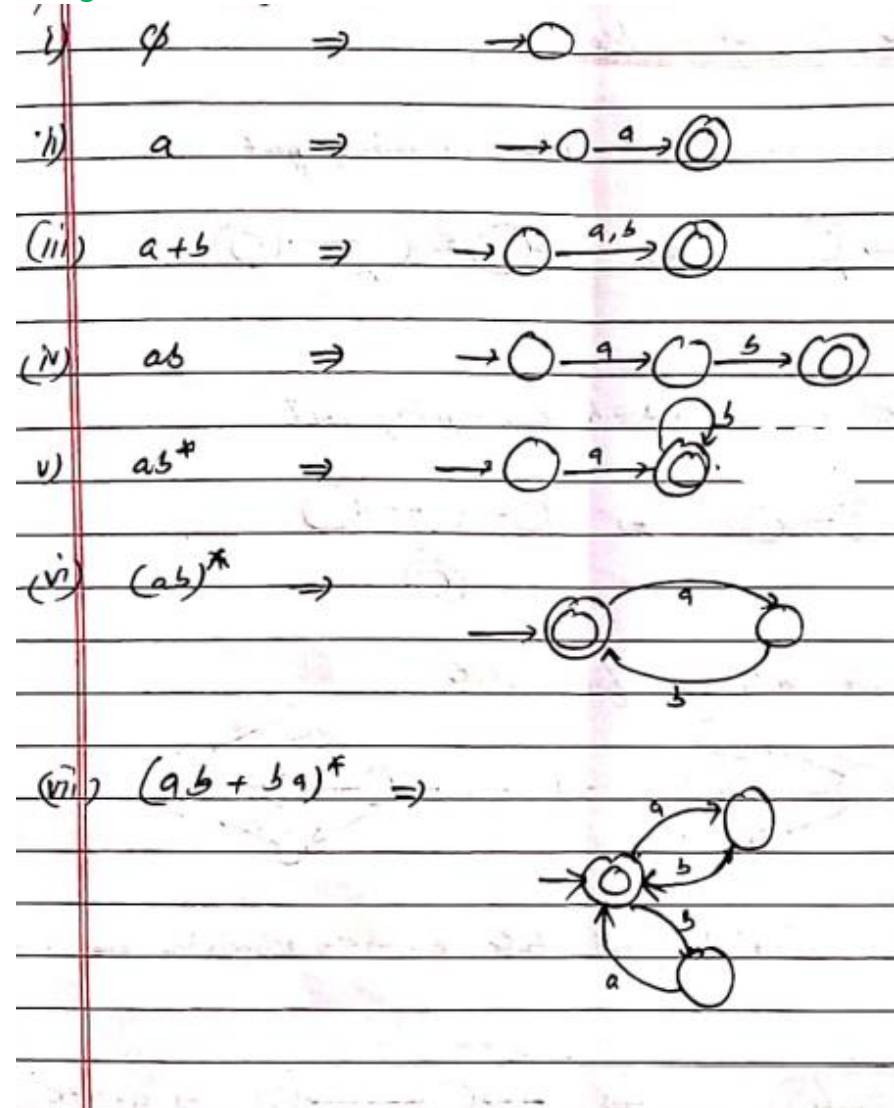
If r , r_1 , r_2 and r_3 are regular expressions, then:

Commutativity Law	$r_1 \cup r_2 = r_2 \cup r_1$ $r_1.r_2 \neq r_2.r_1$
Associativity Law	$r_1 \cup (r_2 \cup r_3) = (r_1 \cup r_2) \cup r_3$ $r_1.(r_2.r_3) = (r_1.r_2).r_3$
Distributive Law	$r_1 (r_2 \cup r_3) = r_1 r_2 \cup r_1 r_3$ $(r_2 \cup r_3) r_1 = r_2 r_1 \cup r_3 r_1$
Identity Law	$r_1 \cup \Phi = \Phi \cup r_1 = r_1$ $r_1.\epsilon = \epsilon.r_1 = r_1$
Annihilator Law	$\Phi.r = r.\Phi = \Phi$
Idempotent Law	$r \cup r = r$

2.16 Finite Automata and RE

We can convert each of regular expression into finite automata. Also, from given automata, we can find out the regular expressions.

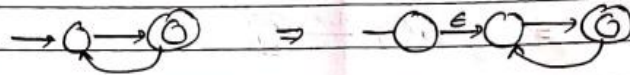
From Regex to FA



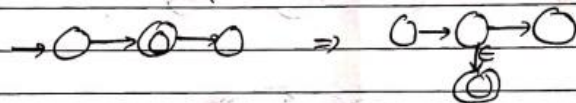
From FA to Regex

1) Using State Elimination Method

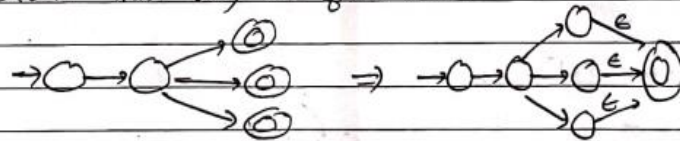
a) Initial state should not have incoming part.



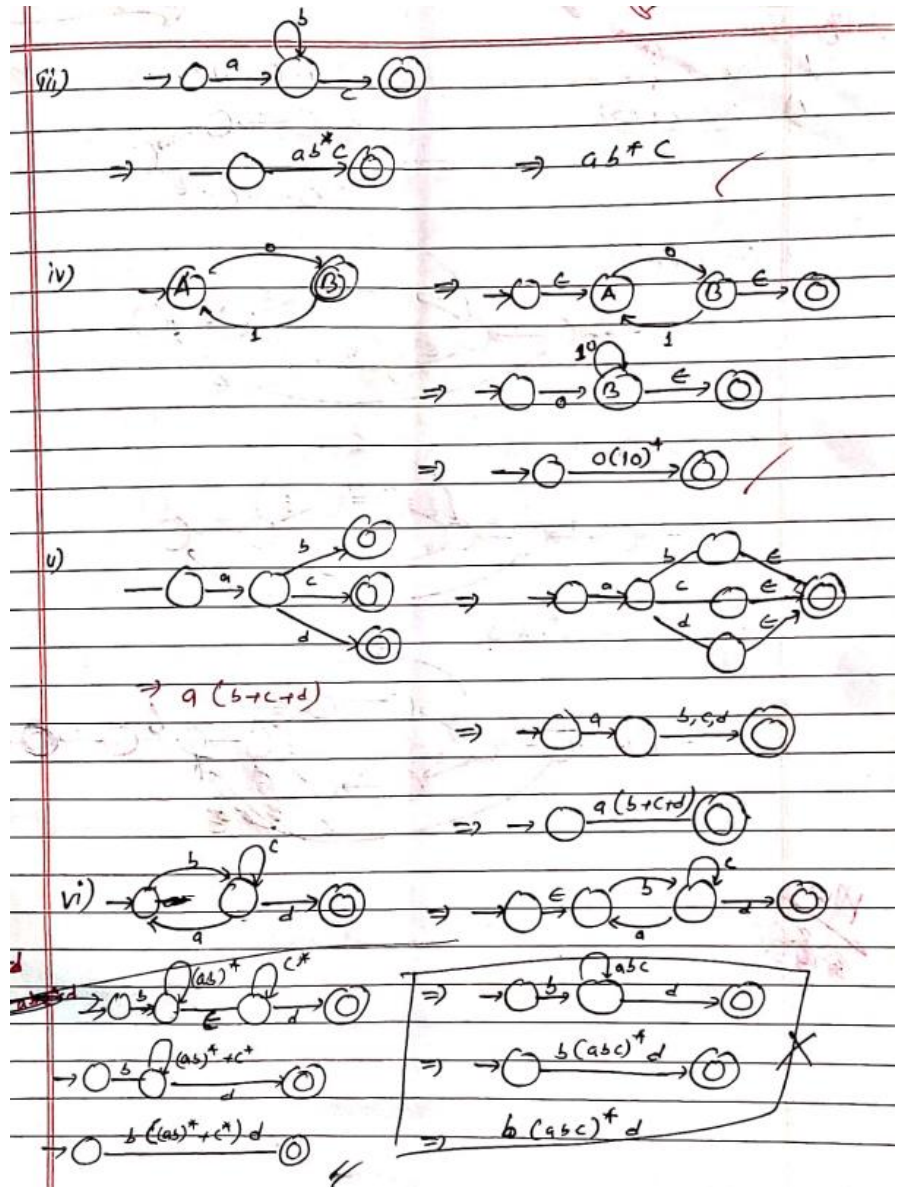
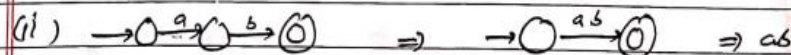
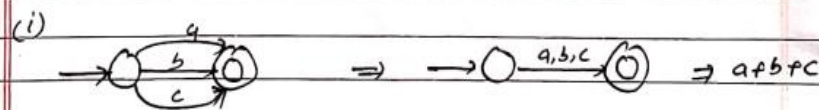
b) Final state should not have outgoing part.



c) Should have only one final state



d) Other than initial & final state, eliminate other state one by one



2.17 Arden's Theorem

- "If P and Q are two RE over alphabet Σ , and if P does not contain null string ϵ , then the following equation in R is given by $R = Q + RP$ has a unique solution, i.e. $R = QP^*$."
- It means, R in the form of $R = Q + RP$ can be replaced by $R = QP^*$.

Proof:

a) Proof of solution.

Let us solve the equation, $R = Q + RP$ -- (1)

Now, replacing R by $Q + RP$ in (1),

$$R = Q + QP + RP^2$$

$$= Q + QP^2 + RP^3$$

$$\therefore R = QP^*$$

b) Proof of for the only solution

(Consider, $R = Q + RP$)

$$= Q + (Q + RP)P \quad (\because R = Q + RP)$$

$$= Q + QP + RP^2$$

Again replace R by $Q + RP$

$$R = Q + QP + (Q + RP)P^2$$

$$= Q + QP + QP^2 + RP^3$$

$$\therefore R = Q + QP + QP^2 + \dots + QP^n + RP^{n+1}$$

$$= Q + QP + QP^2 + \dots + QP^n + QP^{n+1} \quad (\because R = QP^*)$$

$$= Q(\epsilon + P + P^2 + \dots + P^n + P^{n+1})$$

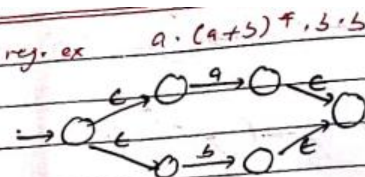
$$= QP^* \quad (\because P^* \text{ is the closure of } P)$$

Hence proved //

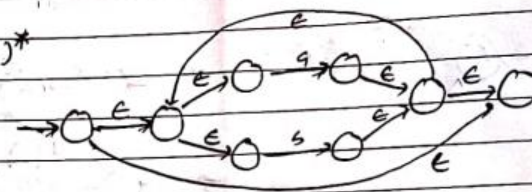
2.18 Numerical

1. Build the automaton for reg. ex $a \cdot (a+b)^+ \cdot b \cdot b$

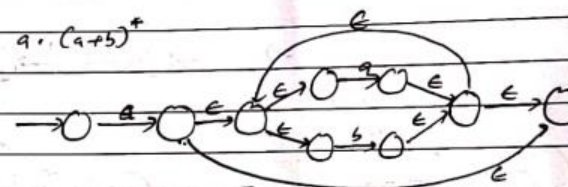
1) For $(a+b)$



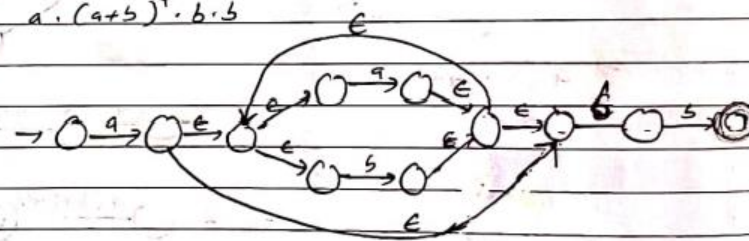
2) For $(a+b)^*$



3) For $a \cdot (a+b)^*$

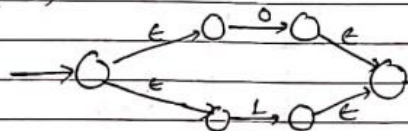


4) For $a \cdot (a+b)^+ \cdot b \cdot b$

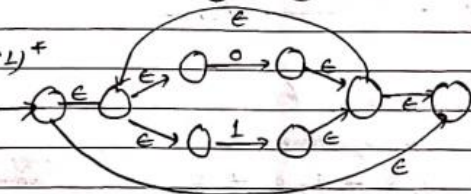


2. Construct ϵ -NFA for the reg. exp $(0+1)^+ 1 (0+1)$

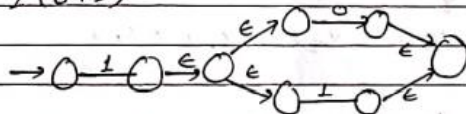
① For $(0+1)$



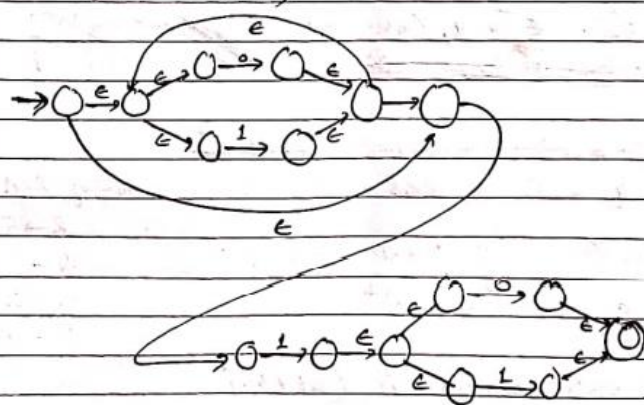
② For $(0+1)^+$



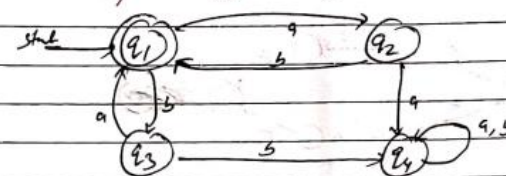
③ For $1(0+1)$



④ For $(0+1)^+ 1 (0+1)$



3) Find the regular expression for transition diagram given in



Sol: Using Arden's theorem, ϵ

Let us form the equations:

$$q_1 = q_2 b + q_3 a + \epsilon \quad (\text{considering incoming edges})$$

$$q_2 = q_1 a$$

$$q_3 = q_1 b$$

$$q_4 = q_2 a + q_3 b + q_4 a + q_4 b$$

Put q_2 and q_3 in q_1 as:

$$q_1 = q_1 ab + q_1 ba + \epsilon$$

$$q_1 = \epsilon + q_1 (ab + ba)$$

Here $q_1 = \epsilon + q_1 (ab + ba)$ is in the form of $P = Q + RP$

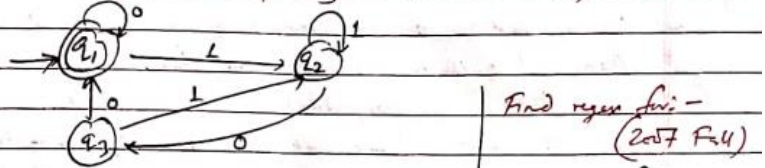
By Arden's Theorem, $P = Q + RP \Rightarrow P = QP^*$

$$\therefore q_1 = \epsilon (ab + ba)^*$$

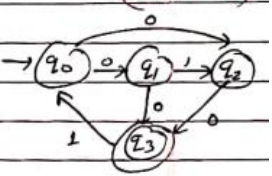
So, reg^d regular expression is $(ab + ba)^*$

2. FINITE AUTOMATA

4) Construct a regex corresponding to the state diagram given as:



Find regex for -
(2017 Feb)



sol: Let us form the equations:

$$q_1 = q_1 0 + q_3 0 + \epsilon$$

$$q_2 = q_1 1 + q_2 1 + q_3 1$$

$$q_3 = q_2 0$$

Now

$$q_2 = q_1 1 + q_2 1 + (q_2 0) 1$$

$$= q_1 1 + q_2 (1 + 01)$$

$$\Rightarrow q_2 = q_1 1 (1 + 01)^*$$

Again

$$q_1 = q_1 0 + q_3 0 + \epsilon$$

$$= q_1 0 + q_2 00 + \epsilon$$

$$= q_1 0 + (q_1 1 (1 + 01)^*) 00 + \epsilon$$

$$= \epsilon + q_1 (0 + 1(1 + 01)^* 00)$$

$$= \epsilon (0 + 1(1 + 01)^* 00)^*$$

$$= (0 + 1(1 + 01)^* 00)^*$$

So regex is: $(0 + 1(1 + 01)^* 00)^*$

Let $R \Rightarrow q_2$

$$Q \Rightarrow q_1 1$$

$$P = (1 + 01)$$

$$\text{Applying } R = QP^*$$

$$\Rightarrow R = QP^*$$

$$\text{where } R \Rightarrow q_1$$

$$Q \Rightarrow \epsilon$$

$$P \Rightarrow (0 + 1(1 + 01)^*)$$

Tutorial: Page 104, Ex: 4.16

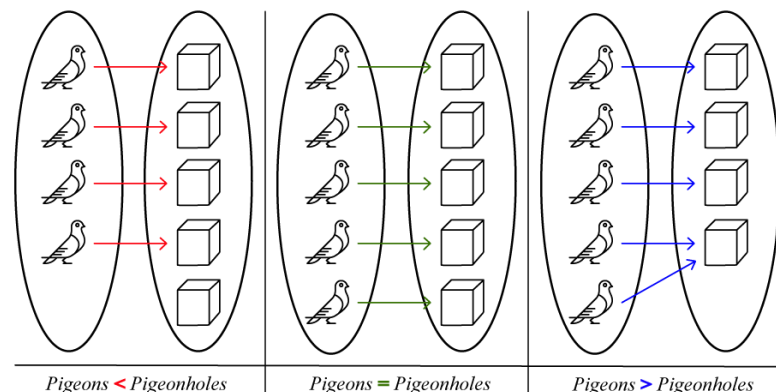
Tutorial:

1. Construct an NFA for $(ab/ba)^*ab$

2.19 The pigeonhole principle

The idea is this.

If there are n number of pigeonholes and $(n+1)$ pigeons, and if all the pigeons come home to roost, then there will be a pigeonhole that contains more than one pigeon.



Pigeonhole Principle Diagram

The pigeonhole principle, also known as the Dirichlet principle, originated with German mathematician Peter Gustave Lejeune Dirichlet in the 1800s, who theorized that given m boxes or drawers and $n > m$ objects, then at least one of the boxes must contain more than one object.

If m is the number of objects (pigeons) and n is the number of boxes (pigeonholes), and let $f: m \rightarrow n$ be a function, then

The abstract formulation of the principle: Let X and Y be finite sets

- If $m < n$, then function is one-to-one but not onto.
- If $m = n$, then function is both one-to-one but onto (bijection).
- If $m > n$, then function is onto but not one-to-one

Example:

For instance, suppose there are 35 different time periods during which classes at the local college can be scheduled. If there are 679 different classes, what is the minimum number of rooms that will be needed?

Let $n = 679$ (pigeons) and $m = 35$ (pigeonholes)

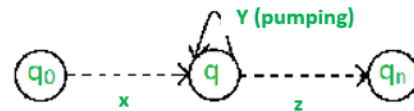
$$\text{Then } \left\lceil \frac{679}{35} \right\rceil = \lceil 19.4 \rceil = 20$$

Therefore, the local college will need 20 different rooms to accommodate the different classes and periods.

2.20 Pumping Lemma for Regular Language**Statement**

If L is a regular language, then L has a pumping length 'p' such that any string 's' where $|s| \geq p$ may be divided into 3 parts $s=xyz$ such that the following conditions must be true:

1. $xy^iz \in L$ for every $i \geq 0$
2. $|y| > 0$
3. $|xy| \leq p$



In simple terms, this means that if a string y is 'pumped', i.e., if y is inserted any number of times, the resultant string still remains in L .

Pumping Lemma is used as a proof for irregularity of a language. Thus, if a language is regular, it always satisfies pumping lemma. If there exists at least one string made from pumping which is not in L , then L is surely not regular.

The opposite of this may not always be true. That is, if Pumping Lemma holds, it does not mean that the language is regular.

Proof

Let $M = (Q, \Sigma, \delta, q_0, q_f)$ be a DFA recognizing a language L .
Let L is regular. Let n be the no. of states.
Consider any string s of length n or more, i.e. $|s| \geq n$.

Let $s = a_1 a_2 \dots a_m$ be the string, and
 $Q = \{q_0, q_1, \dots, q_m\}$ be the states.

The transition function is: $\delta(q_0, a_1 a_2 \dots a_i) = q_i$ for $i = 1, 2, \dots, m$.

Q is the sequence of states in the path with the path value $s = a_1 a_2 \dots a_m$.

As there are only n distinct states, at least two states in Q must coincide. (By pigeonhole principle).

Let us take q_i and q_j ($q_i = q_j$). Then i and j satisfy the condition $0 \leq i < j \leq n$.

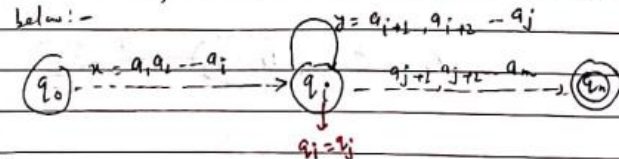
Now let us break $s = xyz$ as follows: three substrings:

$$x = a_1 a_2 \dots a_i$$

$$y = a_{i+1} a_{i+2} \dots a_j$$

$$z = a_{j+1} a_{j+2} \dots a_m$$

The path value with path value ω in the transition diagram of M is shown below:-



The automaton M starts from the initial state q_0 .

On applying the string x , it reaches the state q_1 .

On applying the string y , it comes back to q_i .

~~After application of~~

On applying τ , it reduces to 9μ , a fixed steel.

None, $xy^iz \in L$ for $i \geq 0$ correct!

As every state in G is obtained by applying an input symbol,
 $y \neq \epsilon$. Hence $|y| > 0$ $\rightarrow$ Case 2

As, $i \leq n$, $|xy| \leq p$ — Condition 3

where p is the pump length.

2.21 Steps for proving given language is not Regular using Pumping Lemma theory

~~2.2~~ To prove that a language is not regular using Pumping lemma, follow the below steps.

1. Assume that A is regular
2. It has to have a Pumping-length (say P)
3. All strings longer than P can be pumped ($|S| \geq P$)
4. Now find a string S in A such that $|S| \geq P$
5. Divide S into xy^iz
6. Show that $xy^iz \notin A$ for some i
7. Then consider all ways that S can be divided into xyz
8. Show that none of these can satisfy all the 3 pumping conditions at the same time.
9. S cannot be pumped $\Rightarrow$ Contradiction.

2.22 Numerical

1. Show that $L = \{a^n b^n, n \geq 1\}$ is not regular.

Proof: Assume that L is regular, and the pumping length is P , and S is the string.

Here, $g = a^n b^n$

Let us consider $p = 7$

According to the Pumping Lemma statement, $|s| \geq p$, so we take here,

$$S = a^p b^p = a^7 b^7 \Rightarrow a a a a a a a b b b b b b b$$

This S can be divided in the form of xyz such that $xy^iz \in L$ for every $i \geq 0$, $|x| \geq 0$ and $|y| \leq p$.

Case 1: Let's divide S in xyz form where y is in the part of a .

$\underbrace{a \ a \ a \ a \ a \ a \ a}_x \underbrace{b \ b \ b \ b \ b \ b \ b}_z$

Taking $i=2$, $xy^iz = xy^2z$

$$= 99 \text{ ~~44~~ } 99999999 \text{ } 95555555$$

$$= 9^{11} 5^7$$

Here, $x, y, z \in A$ Not satisfied

$|x| > 0$ satisfied

 $|xy| \leq p$ satisfied

Case 2: The y is in the part of b

$a \ a \ a \ a \ a \ a \ b \ b \ b \ b \ b \ b$
 $\underbrace{\hspace{1.5cm}}_x \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{1.5cm}}_z$

Taking $i=2$,

$$\begin{aligned} xyiz &= xy^2z \\ &= aaaaaabbbbbb \\ &= a^7b^6 \end{aligned}$$

Here, $xyiz \notin A$ Not satisfied
 $|y| > 0$ satisfied
 $|xy| \leq P$ Not satisfied

Case 3: The y is in the part of both a and b

$a \ a \ a \ a \ a \ a \ b \ b \ b \ b \ b \ b$
 $\underbrace{\hspace{1.5cm}}_x \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{1.5cm}}_z$

Taking $i=2$,

$$\begin{aligned} xyiz &= xy^2z \\ &= aaaaaabbbbbb \\ &= a^7b^6 \end{aligned}$$

Here $xyiz \notin A$ Not satisfied
 $|y| > 0$ satisfied
 $|xy| \leq P$ satisfied

From the above three cases, we can observe that none of the cases can satisfy all the three pumping conditions at the same time.

Hence, ~~proved~~ proved by contradiction, Given language is not regular.

2. Show that $A = \{xy \mid y \in \{0,1\}^*\}$ is not regular. → [first half & second half of the string are same]

Proof: Assume that A is regular, so must have pumping length.

Let pumping length = P and S is any string in the language. Let $P=7$

$$\begin{aligned} \text{Let us choose, } S &= 0^P 1 0^P 1 \\ &= 0^7 1 0^7 1 = 0000000100000001 \end{aligned}$$

According to pumping lemma statement, the S can be divided into the form of xyz such that $xyiz \in A$ for every $i \geq 0$, $|y| > 0$ and $|xy| \leq P$.

Case 1: Let's divide S in xyz where y is in the first half.

$$S = \underbrace{0000000}_x \underbrace{10000000}_y \underbrace{1}_z$$

$$\begin{aligned} \text{Taking } i=2, \quad xyiz &= xy^2z \\ &= 000000000100000001 \\ &\Rightarrow \text{1st half} \neq \text{2nd half} \end{aligned}$$

$\therefore xyiz \notin A$ Not satisfied
 $|y| > 0$ satisfied
 $|xy| \leq P$ satisfied

Case 2: Dividing S in xyz where y is in the 2nd half.

$$S = \underbrace{0000000}_x \underbrace{10000000}_y \underbrace{1}_z$$

Here $|xy| \leq P$ Not satisfied.

$$\begin{aligned} \text{Taking } i=2, \quad xyz &= xy^2z \\ &= 00000000000000001 \end{aligned}$$

GURUKUL $\Rightarrow$ 1st half $\neq$ 2nd half
 $\therefore xyz \notin A$, $|y| > 0$ ~~satisfies~~ $|xy| \leq P$ Not satisfied

Language Contradiction, given A is not regular. Hence, proved by contradiction.

2. FINITE AUTOMATA

3. Show that $L = \{a^n b^{2n} : n \geq 1\}$ is not regular.

Proof: Assume that L is regular, and the pumping length is P , and S is the string.

Here, $S = a^P b^{2P}$

Let us consider $P=7$.

According to the pumping lemma statement, $|xy| \leq P$

Let us choose, $S = a^P b^{2P} = a^7 b^{14}$

$= aaaaaaa bbbbbb bbbbbb$

Acc. to pumping lemma statement, S can be divided in xyz form, such that $xy^iz \in A$ and $|y| > 0$ and $|xy| \leq P$.

Case 1: Let us divide S into xyz such that $|xy| \leq P$ and $|y| > 0$ in pump.

$S = \underbrace{aaaaaa}_x \underbrace{aa}_{y} \underbrace{bbbbbb bbbbbb}_z$

Taking $i=2$, $xy^iz = xy^2z$

$= aaaaaa aaaa bbbbbb bbbbbb$

$= a^9 b^{14}$

$\therefore xy^iz \notin A$ Not satisfied.

Case 2: Divide S into xyz , y is part of b such that $|xy| \leq P$ and $|y| > 0$

$S = \underbrace{aaaaaaaaa}_x \underbrace{bbbbb}_y \underbrace{bbbbb}_z$

Taking $i=2$, $xy^iz = aaaaaaa bbbbbb bbbbbb$

$= a^7 b^{20}$

$\therefore xy^iz \notin A$ Not satisfied.

Hence proved by contradiction L is not regular.

GURUKUL

Tutorial:

Prove that below languages are not regular.

1. $L = \{a^n b a^n \text{ for } n=0,1,2,\dots\}$
2. $L = \{a^n b a^{n+1} \text{ for } n=1,2,3,\dots\}$
3. $L = \{0^n 1^m 2^n \text{ for } n, m \geq n\}$
4. $L = \{0^n 1^m \mid n \leq m\}$
5. $L = \{0^n 1^{2n} \mid n \geq 1\}$
6. $L = \{0^n \mid n \text{ is a perfect square}\}$
7. $L = \{0^k \mid k \text{ is prime no}\}$

2.23 Decision properties of Regular Language

Here, we consider some of the fundamental questions about languages.

Is L empty? 1. Is the language described empty?

Is L finite? 2. Is a particular string w in described language?

Are L_1 and L_2 equivalent? 3. Do two descriptions of a language actually describe the same language?

We can describe the above questions as follows:-

1) Testing emptiness of regular language.

If our representation is any kind of finite automata, the emptiness question is whether there is any path whatever from start state to some accepting state.

If yes: language is non-empty

If the accepting states are separated from start state, then language is empty.

2) Testing membership in a regular language

→ Let L be any regular language and w is any string. We have to test whether $w \in L$ or not.

We can represent L by some finite automata. Then simulate the automata processing the string of input symbols w , beginning in the start state.

If automata ends in accepting state, the answer is yes otherwise no.

3) Testing for L_1 and L_2 if they are same.

For testing L_1 and L_2 to be the same language, we follow below algorithm:

1. Convert L_1 and L_2 to DFAs
2. Convert L_1 and L_2 to minimal DFAs.
3. Determine if the minimal DFAs are the same.

2.24 Closure Properties of Regular Language

Let L and M be regular languages. Then the following languages are all regular:

1. Union: $L \cup M$

2. Intersection: $L \cap M$

3. Complement: $\overline{L}$

4. Difference: $L \setminus M$

5. Reversed: $L^R = \{w^R \mid w \in L\}$

6. Closure: L^+

7. Concatenation: $L \cdot M$

8. Homomorphism: $h(L) = \{h(w) \mid w \in L, h \text{ is a homomorphism}\}$

9. Inverse homomorphism:

$$h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L, h: \Sigma^* \rightarrow \Delta^* \text{ is a homomorphism}\}$$

1. Union:

If L_1 and L_2 are two regular languages, their union $L_1 \cup L_2$ will also be regular.

Eg. $L_1 = \{a^n \mid n \geq 0\}$ and $L_2 = \{b^m \mid m \geq 0\}$

Then $L_3 = L_1 \cup L_2 = \{a^n \cup b^m \mid n \geq 0, m \geq 0\}$ is also regular

2. Intersection:

If L_1 and L_2 are two regular languages, their intersection $L_1 \cap L_2$ will also be regular.

Eg. $L_1 = \{a^m b^n \mid n \geq 0 \text{ and } m \geq 0\}$ and

$L_2 = \{a^m b^n \cup b^m a^n \mid n \geq 0 \text{ and } m \geq 0\}$

Then $L_3 = L_1 \cap L_2 = \{a^m b^n \mid n \geq 0 \text{ and } m \geq 0\}$ is also regular

3. Concatenation:

If L_1 and L_2 are two regular languages, their concatenation $L_1 L_2$ will also be regular

Eg. $L_1 = \{a^m \mid m \geq 0\}$ and $L_2 = \{b^n \mid n \geq 0\}$

Then $L_3 = L_1 L_2 = \{a^m b^n \mid m \geq 0 \text{ and } n \geq 0\}$ is also regular

4. Kleene Closure:

If L_1 is a regular language, its Kleene closure L_1^* will also be regular

Eg. $L_1 = (a \cup b)$

$L_1^* = (a \cup b)^+$

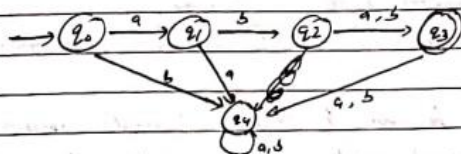
2.25 Numerical

Show that if L is regular then complement of L (L^c) is also regular.

→ If L is a regular language, then L^c is also regular.

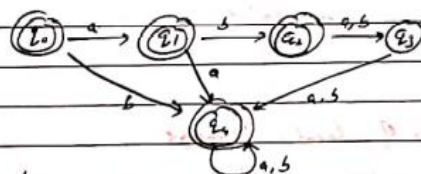
Let us consider a DFA that accepts only the strings aba and abb.

The state diagram is:-



$L = \{w \in \{a,b\}^+ : w \text{ is } aba \text{ or } abb\}$

To find out the regd FA which accepts strings other than aba and abb, we convert every non-final state to final state, and every final state to non-final.



$L^c = \{w \in \{a,b\}^+ : w \text{ is not } aba \text{ or } abb\}$

State diagram

This is the complement of above state diagram, which accepts all other strings except aba and abb.

Thus we can say that if L is regular, then L^c is also regular.

End of Chapter 2

3.1 Introduction to Grammar

- In the literary sense of the term, grammar denotes syntactical rules for conversation in natural language.
- Linguists have attempted to define grammar since the inception of natural languages like English, Sanskrit, Mandarin, etc.
- The theory of languages finds its applicability extensively in the fields of Computer Science.
- Noam Chomsky (Chomsky), in 1958, gave a mathematical model of grammar which is effective for writing computer languages.
- A grammar is nothing but a set of rules to define valid sentences in any language.

A grammar G can be formally written as a 4-tuple (V_n, T, S, P)

where:

$V_n \rightarrow$ Set of variables or non-terminals generally represented by capital letters, $A, B, C, D, \dots$

T or $\Sigma \rightarrow$ Set of terminals, generally represented by small letters $a, b, c, d, \dots$

$S \rightarrow$ starting non-terminal, $S \in V_n$

$P \rightarrow$ Production rules for terminals and non-terminals.

A production rule has the form $\alpha \rightarrow \beta$, where α and β are strings on $V_n \cup \Sigma$ and at least one symbol of α belongs to V_n .

Eg: Grammar G_1 :

$(\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$

Here,

S, A , and B are non-terminal symbols.

a and b are terminal symbols.

S is the start symbol, $S \in V_n$.

Productions, $P: S \rightarrow AB, A \rightarrow a, B \rightarrow b$

Eg: Derivations from a grammar

Let us consider the grammar

$G_2 = (\{S, A\}, \{a, b\}, S, \{S \rightarrow AAb, A \rightarrow aAb, A \rightarrow \epsilon\})$

Some of the strings that can be derived are:

$S \Rightarrow aAb$	using production $S \rightarrow AAb$
$\Rightarrow aaAb$	" " $aA \rightarrow aaAb$
$\Rightarrow aaaaAb$	" " $aA \rightarrow aaAb$
$\Rightarrow aaaa$	" " $A \rightarrow \epsilon$

3.2 Context-free Grammar

- a more powerful tool/method of describing language
- It can describe certain features that have a recursive function structure.
- Collections of languages associated with CFG are called CFL.
- Every regular grammar is context-free, so a regular language is also a context-free one.

3. CONTEXT-FREE GRAMMAR

- It is already proved by pumping lemma that language $\{a^n b^n \mid n \geq 0\}$ is not regular; but it is possible to design a CFG for those languages.
- So, It is very clear that "the family of regular language is a proper subset of the family of context-free language."

Applications of CFG

1. For defining programming languages
2. For parsing the program by constructing syntax tree
3. For translation of programming languages
4. For describing arithmetic expressions
5. For construction of compilers.

Formal Definition of CFG

A CFG is a 4-tuple such that: $G = (V, \Sigma, R, P)$
 where
 V or $V_n =$ A finite non-empty sets of variables / non-terminal symbols
 Σ or T or $V_t =$ Finite set of terminal symbols
 $S =$ Starting non-terminal such that $S \in V$
 R or $P =$ Set of production rules of the form $A \rightarrow \alpha$ where
 $A \in V$ and $\alpha \in (V \cup T)^+$

Consider a grammar $G = (V, \Sigma, R, P)$ where.

$$V = \{S\}$$

$$\Sigma = \{a, b\}$$

$$S = \{S\}$$

$$P = \{S \rightarrow aSbS, S \rightarrow bSaS, S \rightarrow \epsilon\}$$

Strings that can be generated are:

$$S \Rightarrow aSbS$$

$$\rightarrow aSbSaS$$

$$\rightarrow aSbSaSbSaS$$

$$\rightarrow ababba$$

$$\because S \rightarrow aSbS$$

$$\because S \rightarrow bSaS$$

$$\because S \rightarrow bSaS$$

$$\because S \rightarrow \epsilon$$

$$S \Rightarrow aSbS$$

$$\rightarrow abSaSbS$$

$$\rightarrow abSaSbSaS$$

$$\rightarrow ababbaS$$

$$\rightarrow ababbaSaS$$

$$\rightarrow ababbaSbSaS$$

$$\rightarrow ababbaSbSaSbSaS$$

$$\rightarrow ababbaSbSaSbSaSbSaS$$

$$\rightarrow ababbaSbSaSbSaSbSaSbSaS$$

Hence, it generates the strings having equal no of a's and b's.

3.3 Some terminologies

Derivation

Let $G = (V, \Sigma, R, S)$ be a context-free grammar
 If $w_1, w_2, \dots, w_n$ are strings over variable V such that
 $w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n$, then we say w_n is derivable.

The sequence of substitution to obtain a string is called derivation. Simply derivation is a sequence of production rules.

Say w_i derives w_n , i.e. $w_i \xRightarrow{*} w_n$,
 Then, the sequence of steps to obtain w_n from w_i is called derivation.

Language of CFG, $L(G)$

If $G = (V, \Sigma, R, S)$ is a CFG, then the language of G denoted by $L(G)$ is the set of terminal strings that have derivations from start symbol.

$$\text{i.e. } L(G) = \{ w \in \Sigma^* : S \Rightarrow^* w \}$$

Sentential Forms

Derivations from the start symbol produce strings that have a special form. And this is called sentential form.

If $G = (V, \Sigma, R, S)$ is a CFG, then any string α in $(V \cup \Sigma)^*$ such that $S \Rightarrow^* \alpha$ is a sentential form.

If $S \xRightarrow{*}_{lm} \alpha$, then α is a left sentential form.

If $S \xRightarrow{*}_{rm} \alpha$, then α is a right sentential form.

Example:

Let any set of production rules in a CFG G :

$$X \rightarrow X+X \mid X * X \mid X \mid a \quad \text{over an alphabet } \{a\}.$$

Then, the left most derivation is:

Then, the left most derivation is:

$$\begin{aligned} X &\rightarrow X+X & \because X &\rightarrow X+X \\ &\rightarrow a+X & X &\rightarrow X+X \\ &\rightarrow a+X * X & X &\rightarrow X * X \\ &\rightarrow a+a * a & X &\rightarrow a \end{aligned}$$

Hence, $a+a * a$ is a left sentential form.

The right most derivation is:

$$\begin{aligned} X &\rightarrow X * X & \because X &\rightarrow X+X \\ &\rightarrow X * a & X &\rightarrow a \\ &\rightarrow X + X * a & X &\rightarrow X+X \\ &\rightarrow X + a * a & X &\rightarrow a \\ &\rightarrow a + a * a & X &\rightarrow a \end{aligned}$$

Hence $a+a * a$ is a right sentential form.

3.4 Numerical

1. Find the CFL for a CFG where, $G = (V, \Sigma, R, S)$ where

$$V = \{S\}$$

$$\Sigma = \{a, b\}$$

$$R = \{ S \rightarrow aSb, S \rightarrow ab \}$$

3. CONTEXT-FREE GRAMMAR

Solⁿ: Given: $V = \{S\}$
 $\Sigma = \{a, b\}$

$R = \{S \rightarrow aSb, S \rightarrow ab\}$

To find: $L(G) = ?$

Here, S is the only terminal which is the starting symbol for the grammar
 a and b are terminals.

Now,

$S \Rightarrow aSb \quad \because S \rightarrow aSb$
 $\Rightarrow aabbb \quad \because S \rightarrow ab$
 $\Rightarrow a^2b^2$

Applying first production $n-1$ times followed by an application of second production, we get:

$S \Rightarrow aSb \quad \because S \rightarrow aSb$
 $\Rightarrow aaaSbbb \quad \because S \rightarrow aSb$
 $\Rightarrow \dots$
 $\Rightarrow a^{n-1}Sb^{n-1}$
 $\Rightarrow a^n b^n$

Hence language of given grammar is:

$L(G) = \{a^n b^n : n \geq 1\}$

2. Let $G = (V, \Sigma, R, S)$ where, $V = \{S\}$

$\Sigma = \{a, b\}$

$R = \{S \rightarrow aaS, S \rightarrow b\}$

Find $L(G)$

Solⁿ: Using given rules recursively:

(i) $S \rightarrow b$

(ii) $S \rightarrow aaaS$
 $\rightarrow aab$

Hence, language of given grammar is:

(iii) $S \rightarrow aaaS$
 $\rightarrow aaaS$
 $\rightarrow aaaSb$
 $\rightarrow (aa)^2b$

$L(G) = \{(aa)^n b : n \geq 0\}$

(iv) $S \rightarrow aaaS$
 $\rightarrow aaaS$
 $\rightarrow (aa)^n b$

3. Let $G = (V, \Sigma, R, S)$ where, $V = \{S, A\}$

$\Sigma = \{a, b\}$

$R = \{S \rightarrow aA \mid A \rightarrow bS\}$, Find $L(G)$

Solⁿ: Using given rules recursively

(i) $S \rightarrow \epsilon$

(ii) $S \rightarrow aA$

(iv) $S \rightarrow aA$

(ii) $S \rightarrow aA$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow ab$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

$\rightarrow abS$

Hence language of given grammar is:

$L(G) = \{(ab)^n : n \geq 0\}$

7

4. Write a CFG for the language given by:
 $L = \{a^m b^n : m \geq n\}$

Sol: The possible strings are:

$\epsilon, a, b, aab, aaab, aaab, aaaaab, \dots$

Design of R as:

$S \rightarrow aS \mid aSb \mid \epsilon$

Let $G = (V, \Sigma, R, S)$ be required CFG

where,

$V = \{S\}$

$\Sigma = \{a, b\}$

$R = \{S \rightarrow aSb, S \rightarrow aS, S \rightarrow \epsilon\}$

$S = \{S\}$

Now, check for validity:

For <u>a</u>	<u>aab</u>	<u>aaab</u>	<u>aaaab</u>
$S \rightarrow aS$	$S \rightarrow aS$	$S \rightarrow aS$	$S \rightarrow aS$
$\rightarrow a\epsilon$	$\rightarrow aaSb$	$\rightarrow aaaSb$	$\rightarrow aaaSb$
$\rightarrow a$	$\rightarrow aaaSb$	$\rightarrow aaaSb$	$\rightarrow aaaSb$
	$\rightarrow aab$	$\rightarrow aaaSb$	$\rightarrow aaaSb$
		$\rightarrow aaaSb$	$\rightarrow aaaSb$

Rough work:

$S \rightarrow aS \mid \epsilon$ aab $\rightarrow aaaSb$
 $S \rightarrow aSb \mid aS \mid \epsilon$ $S \rightarrow aSb \mid aS \mid \epsilon$

5. Write a CFG for the language given by:
 $L = \{a^m b^n : m \geq 0, n \geq 0\}$

Sol: Possible strings are:

$a, b, ab, aabb, aaabbb, aab, aqaab, aab, aab, \dots$

Design of R as:

$S \rightarrow aSb \mid \epsilon$

$S \rightarrow aS \mid bS$

check for validity:

<u>ab</u>	<u>aaabbb</u>	<u>a bbb</u>	<u>aab</u>
$S \rightarrow aSb$	$S \rightarrow aSb$	$S \rightarrow aSb$	$S \rightarrow aSb$
$\rightarrow ab$	$\rightarrow aaaSb$	$\rightarrow aaaSb$	$\rightarrow aaaSb$
	$\rightarrow aaaSb$	$\rightarrow aaaSb$	$\rightarrow aaaSb$
	$\rightarrow aaaSb$	$\rightarrow aaaSb$	$\rightarrow aaaSb$

Hence, the CFG can be:

$G = (V, \Sigma, R, S)$

where, $V = \{S\}$

$\Sigma = \{a, b\}$

$R = \{S \rightarrow aSb \mid \epsilon$
 $S \rightarrow aS \mid bS\}$

$S = \{S\}$

Rough Work

For ϵ

$S \rightarrow \epsilon$

For a

$S \rightarrow aS \mid \epsilon$

For b

$S \rightarrow bS \mid \epsilon$

For aab

$S \rightarrow aSb \mid \epsilon$

For aaabbb

$S \rightarrow aSb \mid \epsilon$

For abbb

$S \rightarrow aSb \mid \epsilon$

Tutorial:

Book page no: 131 to 147

Write a CFG, -

- ① Which generates string of balanced parentheses. $()(())$
- ② which generates ~~string~~ palindromes for binary numbers 0,1,010,101
- ③ which generates strings having equal no of a and b. ✓ done.
- ④ for the regular expression $r = (a+b)^+$ Ex: ab asab
- ⑤ " " $r = 0^*1(0+1)^*$ Ex: 00101
- ⑥ " " $r = (a+b)^+aa(a+b)^+$ Ex: baaabacab
- ⑦ that generates string having any combination of a's and b's except null string.
- ⑧ for the language:
 - i) $L(G) = \{w w^r : w \in \{0,1\}^+\}$ Ex: 001100
 - ii) $L(G) = \{a^n (b^m a)^n b^m : n \geq 0\}$ Ex: ab(baa)^2 bba
 - iii) $L = \{a^n b^m : n \neq m\}$
 - iv) $L = \{0^n 1^m / n \geq 0\} \cup \{1^n 0^m / n \geq 0\}$
 - v) $L = \{a^n b^m c^n d^m / n \geq 1, m \geq 1\}$ Ex: aaabbbccdd
 $n=3, m=2$
 - vi) $L = \{a^n b^m c^m d^n / n \geq 1, m \geq 1\}$ Ex: abbccdd

3.5 Bacus Naur Form (BNF)

Bacus-Naur Form or Bacus Normal Form is the notation that involves minor changes in the format and some shorthand.

The above grammar can be rewritten in BNF as:-

$$S \rightarrow aSa \mid bSb \mid c$$

3.6 Parse Tree/ Derivation Tree

- It is a tree representation of strings of terminals using the productions defined by the grammar
 - It pictorially shows how the start symbol of a grammar derives a string in language.

- It is an ordered tree that shows the essentials of a derivation.

Formally, given CFG, $G = (V, \Sigma, R, S)$, a parse tree has following properties:-

1. The root is labelled by start symbol.
2. Each interior node of parse tree are variables.
3. Each leaf node is labelled by terminal symbol or ϵ .
4. If an interior node is labelled with non-terminal A and its children are $X_1, X_2, \dots, X_n$ from left to right and then there is a production $A \rightarrow X_1 X_2 \dots X_n$ for each $X_i \in \Sigma$

Example:

Consider a grammar where $S \rightarrow aSa \mid a \mid b \mid \epsilon$

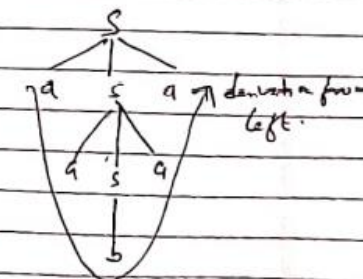
Now, for string aabaa,
we have

Now parse tree is:

$$S \rightarrow aSa$$

$$\rightarrow aaSaa$$

$$\rightarrow aabaa$$



3. CONTEXT-FREE GRAMMAR

3.7 Numerical

1. Consider the grammar G :

$$S \rightarrow A1B$$

$$A \rightarrow 0A \mid \epsilon$$

$$B \rightarrow 0B \mid 1B \mid \epsilon$$

Construct parse tree for: 00101, 1001, 00011

i) For 00101

$$\begin{aligned}
 S &\rightarrow A1B \\
 &\rightarrow 0A1B \\
 &\rightarrow 00A1B \\
 &\rightarrow 00\epsilon 1B \\
 &\rightarrow 0010B \\
 &\rightarrow 00101B \\
 &\rightarrow 00101\epsilon \\
 &\rightarrow 00101
 \end{aligned}$$

2. Let $G = (V, E, R, S)$ where $V = \{s\}$
 $E = \{ () \}$
 $R = \{ s \rightarrow ss, s \rightarrow (s), s \rightarrow \epsilon \}$

Construct parse tree for $() ()$

For $() ()$

$$\begin{aligned}
 S &\rightarrow SS \\
 &\rightarrow (S)S \\
 &\rightarrow (S)(S) \\
 &\rightarrow (\epsilon)(S) \\
 &\rightarrow ()(\epsilon) \\
 &\rightarrow () ()
 \end{aligned}$$

Parse tree

3.8 Left Most and Right-most Derivation

- In LMD, leftmost variable is replaced first
- In RMD, rightmost variable is replaced first

Ex: Consider a grammar

$S \rightarrow S + S$ New derive string
 $S \rightarrow S / S$ $a + (a + a) / a - a$ with LMD and RMD
 $S \rightarrow (S)$ also draw parse tree
 $S \rightarrow S - S$
 $S \rightarrow S * S$
 $S \rightarrow a$

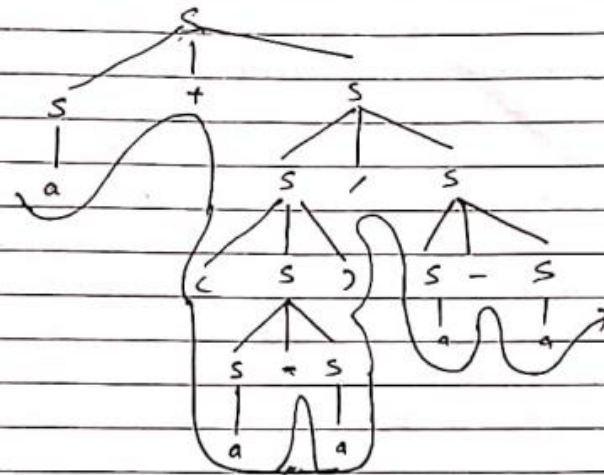
Ex: LMD

$$\begin{aligned}
 S &\rightarrow S + S \\
 &\rightarrow a + S \\
 &\rightarrow a + S / S \\
 &\rightarrow a + (S) / S \\
 &\rightarrow a + (S * S) / S \\
 &\rightarrow a + (a * S) / S \\
 &\rightarrow a + (a * a) / S \\
 &\rightarrow a + (a * a) / (S - S) \\
 &\rightarrow a + (a * a) / a - S \\
 &\rightarrow a + (a * a) / a - a
 \end{aligned}$$

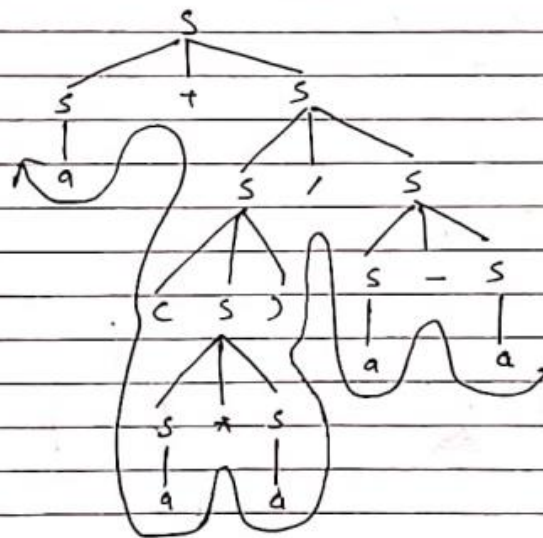
RMD

$$\begin{aligned}
 S &\rightarrow S + S \\
 &\rightarrow S + S / S \\
 &\rightarrow S + S / S - S \\
 &\rightarrow S + S / S - a \\
 &\rightarrow S + S / a - a \\
 &\rightarrow S + (S) / a - a \\
 &\rightarrow S + (S * S) / a - a \\
 &\rightarrow S + (S * a) / a - a \\
 &\rightarrow S + (a * a) / a - a \\
 &\rightarrow a + (a * a) / a - a
 \end{aligned}$$

Parse tree by LMD:



Parse tree by RMD



3.9 Ambiguity in Grammar

- A grammar is said to be ambiguous if for a given string generated by the grammar, there exists:
 - more than one leftmost derivation
 - or more than one rightmost derivation
 - or more than one parse tree (or derivation tree)
- If the grammar is not ambiguous, then it is called unambiguous.
- If the grammar has ambiguity, then it is not good for a compiler construction.
- No method can automatically detect and remove the ambiguity, but we can remove ambiguity by re-writing whole grammar without ambiguity.

Eg:

Consider a grammar which has production rules as:-

$$S \rightarrow AB / aqB$$

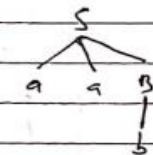
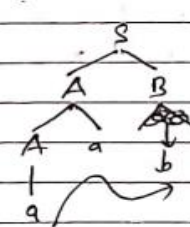
$$A \rightarrow a / Aa$$

$$B \rightarrow b$$

Now for string aaab, we have two leftmost derivations as:-

$$\begin{aligned} \textcircled{1} \quad S &\rightarrow AB \\ &\rightarrow AaB \\ &\rightarrow aqB \\ &\rightarrow aab \end{aligned}$$

$$\begin{aligned} \textcircled{2} \quad S &\rightarrow aqB \\ &\rightarrow aab \end{aligned}$$



Hence the grammar is ambiguous.

3. CONTEXT-FREE GRAMMAR

3.10 Numerical

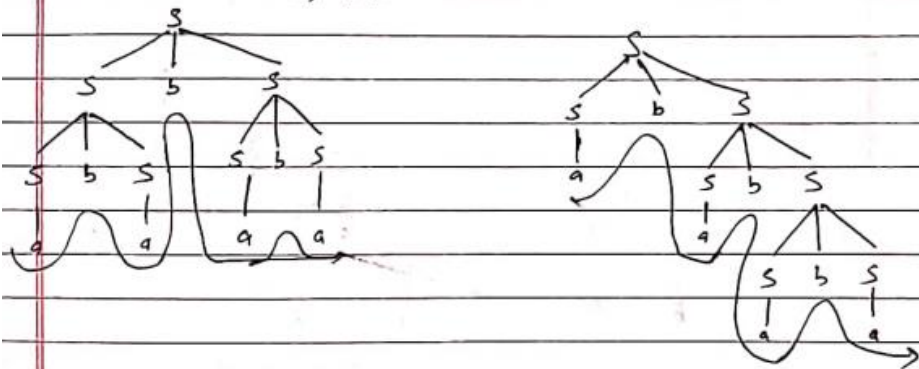
1. If CFG G is $S \rightarrow SbS/a$, show that G is ambiguous.

→ To prove that G is ambiguous, we have to find a string, $w \in L(G)$ which is ambiguous.

→ Let the $w = abababa \in L(G)$.

→ Then we get two derivation tree for w as below.

$S \rightarrow SbS$	$S \rightarrow SbS$
$\rightarrow SbSbS$	$\rightarrow SbSbS$
$\rightarrow aSbS$	$\rightarrow aSbSbS$
$\rightarrow ababS$	$\rightarrow abSbS$
$\rightarrow ababSbS$	$\rightarrow abSbSbS$
$\rightarrow abababS$	$\rightarrow abababa$
$\rightarrow abababa$	



Since we get two different derivation tree by LMD, hence, the given grammar is ambiguous.

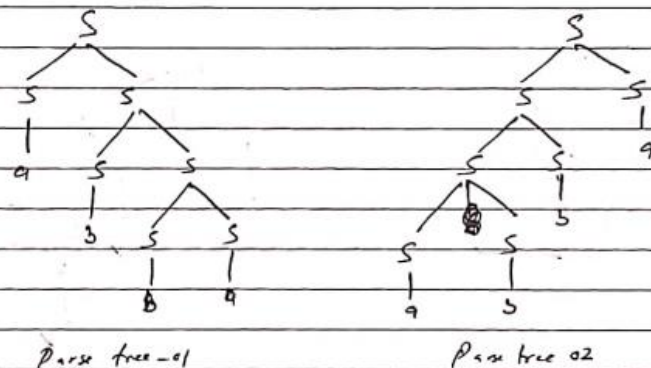
2. Show that the grammar $S \rightarrow a/abSb/aAb$ Consider $w = abab$
 $A \rightarrow bS/aAAb$ is ambiguous.

3. Show that the grammar $S \rightarrow aB/ab$
 $A \rightarrow aAa/a$ Consider $w = ab$
 $B \rightarrow ABb/b$ is ambiguous.

4. Check if given grammar is ambiguous or not: $S \rightarrow SS$ Consider $w = abba$
 $S \rightarrow a/b$

Sol: Let us draw the parse tree for the string w generated by the given grammar $w = abba$.

Now let us draw parse tree for the string w , by LMD



Since two different parse trees exist for string w , therefore the given grammar is ambiguous.

Refer: gatevidyaly.com/grammar-ambiguity-ambiguous-grammar/

3.11 Simplification of CFG

- Various languages can efficiently be represented by a CFG.
- However, all the grammars are not always optimized, that means the grammar may consists of some extra symbols (non-terminals).
- Having extra symbols, unnecessarily increase the length of grammar.
- Simplification of grammar means reduction of grammar by removing useless symbols.

The properties of reduced grammar are:

1. Each variable (i.e. non-terminal) and each terminal of G appears in the derivation of some word in L .
2. There should not be any production $\alpha \rightarrow \gamma$ where α and γ are non-terminals.
3. If ϵ is not in the language L , then there need not to be the production $X \rightarrow \epsilon$.

1. Eliminating useless symbols

- Useless symbols are those variables or terminals that do not appear in any derivation of a terminal string from the start symbol.

For eg:

$$S \rightarrow a \alpha B \mid a \beta A \mid a \gamma S$$

$$A \rightarrow \alpha A$$

$$B \rightarrow \alpha \beta \mid \beta$$

$$C \rightarrow \alpha \delta$$

→ non-generating

→ useless

In the above example,

- The variable C will never occur in any string, so the production $C \rightarrow \alpha \delta$ is useless. So we will eliminate it, and other productions are written in such a way that variable C can never reach from the starting variable S .

Production $A \rightarrow \alpha A$ is also useless because there is no way to terminate it.

For a given grammar $G = (V, \Sigma, R, S)$, a symbol x is useful if, there is some derivation of the form: $S \xRightarrow{*} \alpha x \beta$

Elimination of useless symbols include identifying whether or not the symbol is "generating" and "reachable".

- A symbol x is generating if $x \xRightarrow{*} w$ for some terminal string w .

- A symbol x is reachable if there is derivation $S \xRightarrow{*} \alpha x \beta$ for some α and β .

Thus we eliminate non-generating and then non-reachable symbols.

3. CONTEXT-FREE GRAMMAR

Example:

Consider a grammar:

$$\begin{aligned} S &\rightarrow aB/bX \\ A &\rightarrow Bad/bSX/a \\ B &\rightarrow aSB/bBX \\ X &\rightarrow SBd/aBX/ad \end{aligned}$$

Here,

- A and X can generate terminals, so A and X are generating symbols as $A \rightarrow a$ and $X \rightarrow ad$
- Also S is generating symbol as S can generate terminal string such as $S \rightarrow bX$ and X generate terminal string
- But B cannot produce any terminal symbol, so it is non-generating

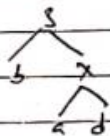
Thus, after eliminating B, we get:

$$\begin{aligned} S &\rightarrow bX \\ A &\rightarrow bSX/a \\ X &\rightarrow ad \end{aligned}$$

Again,

A is unreachable as there is no any derivation of form $S \Rightarrow \alpha A \beta$ in grammar.

After eliminating A

$$\begin{aligned} S &\rightarrow bX \\ X &\rightarrow ad \end{aligned}$$


$\therefore$ This is the simplified grammar

Sample 1:

Consider a CFG, $S \rightarrow AB/a$ Identify & eliminate useless symbols
 $A \rightarrow b$

$\rightarrow$ Here, B is non-generating symbol since B is not deriving any string w in the V^* . So we eliminate $S \rightarrow AB$

So, CFG becomes:

$$\begin{aligned} S &\rightarrow a \\ A &\rightarrow b \end{aligned}$$

Again A is non-reachable symbol from the starting symbol S
 Thus it is to be eliminated.

Hence CFG becomes: $S \rightarrow a$

Sample 2:

Consider below grammar and obtain an equivalent grammar containing no useless grammar symbols.

$$\begin{aligned} A &\rightarrow xyz/Xyz \\ X &\rightarrow Xz/xYz \\ Y &\rightarrow zYg/Xz \\ Z &\rightarrow Zg/z \end{aligned}$$

Here, $A \rightarrow xyz$ and $Z \rightarrow z$, hence A and Z are directly deriving to the string of terminals. So they are useful symbols.

But X and Y do not lead to a string of terminals. So X and Y are useless symbols.

After eliminating these productions, we get:

$$\begin{aligned} A &\rightarrow xyz \\ Z &\rightarrow Zg/z \end{aligned}$$

But Z is not reachable from the starting non-terminal A,
 So, we remove Z.

Thus, $A \rightarrow xyz$ is the required CFG

2. Removal of Empty production

- A grammar is said to have empty production if there is a production of the form $A \rightarrow \epsilon$.
- Here, we have to discover nullable variable.
- A variable 'A' is nullable if $A \Rightarrow^+ \epsilon$
- If $A \rightarrow \epsilon$ is a production to be eliminated, then we look for all productions whose right side contains A and replace each occurrence of A in each of these production to obtain a non-empty production.
- Now these resultant non-empty productions must be added to grammar to keep the language generated to be same.

- Example:

Consider CFG as:
 $S \rightarrow aA$
 $A \rightarrow b/\epsilon$

Sol:

Here $A \Rightarrow \epsilon$ $\therefore$ A is nullable

Now,

Find all productions whose right side contains A.

Here, $S \rightarrow aA$

$\rightarrow a\epsilon$ after replacing A with ϵ
 $\rightarrow a$

Finally we get:

$S \rightarrow aA$
 $A \rightarrow b$
 $S \rightarrow a$

Equivalently,

$S \rightarrow aA/a$

$A \rightarrow b$

is required CFG.

Sample 1:

Consider the following Grammar G:
 $S \rightarrow ABAC$
 $A \rightarrow aA/\epsilon$
 $B \rightarrow bB/\epsilon$

remove the ϵ production from the above grammar $C \rightarrow c$

Sol: 1) $A \rightarrow aA/\epsilon$ and $B \rightarrow bB/\epsilon$ are nullable productions.
 So remove them.

2) Find all productions with A or B on right side.

$S \rightarrow ABAC$

$A \rightarrow aA$

On replacing each occurrence of A by ϵ (one by one), we get fewer no production to be added to the grammar:

$S \rightarrow BAC | AOC | BC$

$A \rightarrow a$

Add these production to the grammar and eliminate $A \rightarrow \epsilon$,
 obtained grammar is:

$S \rightarrow ABAC | BAC | AOC | BC$

$A \rightarrow aA/a$

$B \rightarrow bB/\epsilon$

$C \rightarrow c$

3) Find all the production with B on right side.

$S \rightarrow ABAC | BAC | AOC | OC$

$B \rightarrow bB$

On replacing each occurrence of B by ϵ (one by one) we obtain

$S \rightarrow AAC | AC | C$

$B \rightarrow b$

3. CONTEXT-FREE GRAMMAR

Add these production to the grammar and eliminate $B \rightarrow \epsilon$, obtain grammar D:-

$$S \rightarrow ABAC \mid BAC \mid ABC \mid BC \mid AAC \mid AC \mid C$$
$$A \rightarrow aA \mid a$$
$$B \rightarrow bB \mid b$$
$$C \rightarrow c$$

This is the required grammar without ϵ -production.

3. Removal of unit production

A unit production is a production of the form $A \rightarrow B$ where A and B are both variables.

Unit production increases the cost of derivation in a grammar, hence needs to be removed.

Algorithm:

While (there exists unit production $A \rightarrow B$)

i) Select a unit production $A \rightarrow B$ such that there exists a production $B \rightarrow \alpha$, where α is terminal

ii) for (every non-unit production, $B \rightarrow \alpha$)

- add production $A \rightarrow \alpha$ to grammar

(iii) eliminate $A \rightarrow B$ from grammar

}

Example: $A \rightarrow B$

$$B \rightarrow a$$
$$B \rightarrow c$$

Sol: Here, unit production is $A \rightarrow B$

Handling the unit production, we try to prove directly & replace for each non-terminals involved in the unit production $\Rightarrow$:

$$A \rightarrow B$$

generate $A \rightarrow a$

$$A \rightarrow B$$

generate $A \rightarrow c$

Thus new CFG is:

$$A \rightarrow a \mid c$$

with no unit production

3.12 Numerical:

Identify and remove the unit productions from following grammar:

$$S \rightarrow A | bb$$

$$A \rightarrow B | b$$

$$B \rightarrow S | a$$

Sol: Here unit productions are: $S \rightarrow A$
 $A \rightarrow B$
 $B \rightarrow S$

Handling one unit production at a time, we try to prove generating and reachable for each non-terminals involved in the unit productions as follows:-

$$S \rightarrow A \quad \text{generate } S \rightarrow b$$

$$S \rightarrow A \rightarrow B \quad \text{generate } S \rightarrow a$$

$$A \rightarrow B \quad \text{generate } A \rightarrow a$$

$$A \rightarrow B \rightarrow S \quad \text{generate } A \rightarrow bb$$

$$B \rightarrow S \quad \text{generate } B \rightarrow bb$$

$$B \rightarrow S \rightarrow A \quad \text{generate } B \rightarrow b$$

Then new CFG becomes:

$$S \rightarrow bb | b | a$$

$$A \rightarrow b | a | bb$$

$$B \rightarrow a | bb | a \quad \text{which has no unit productions}$$

Final CFG is:

$$S \rightarrow bb | b | a$$

Note: Capital letter are non-terminals
 i.e. Non-terminals

Consider the context free grammar G : $S \rightarrow AB$

$$A \rightarrow a$$

$$B \rightarrow C | b$$

$$C \rightarrow D$$

$$D \rightarrow E$$

$$E \rightarrow a$$

Sol: Unit productions are: $B \rightarrow C$
 $C \rightarrow D$
 $D \rightarrow E$

To remove unit productions $B \rightarrow C$, we try to prove generating and reachable for each non-terminals.

$$B \rightarrow C \rightarrow D \rightarrow E \quad \text{generate } B \rightarrow a$$

Similarly for $C \rightarrow D$,

$$C \rightarrow D \rightarrow E \quad \text{generate } C \rightarrow a$$

Similarly for $D \rightarrow E$,

$$D \rightarrow E \quad \text{generate } E \rightarrow a$$

Hence, required CFG without unit production is:

$$S \rightarrow AB \quad \text{Final CFG is: } S \rightarrow AB$$

$$A \rightarrow a \quad A \rightarrow a$$

$$B \rightarrow a | b \quad B \rightarrow a | b$$

$$C \rightarrow a \quad \Rightarrow$$

$$D \rightarrow a$$

$$E \rightarrow a$$

3.13 Normal forms

1. Chomsky Normal Form (CNF)
2. Greibach Normal Form (GNF)

3.14 Chomsky Normal Form (CNF)

A CFG $G = (V, \Sigma, R, S)$ is said to be in Chomsky Normal Form (CNF) if every production in G are in one of the two forms:

$$A \rightarrow BC$$

$$A \rightarrow a$$

where, $\{A, B, C\} \in V$ and $a \in \Sigma$

- So a grammar in CNF is one which should not have:-

- i) ϵ production
- ii) Unit production
- iii) Useless symbols.

3.15 Numerical

Q.1. Convert the following CFG into CNF

$G = (V, \Sigma, R, S)$ where $V = \{S, A\}$

$\Sigma = \{a, b\}$

$R = \{S \rightarrow aAB \mid AaB \mid B$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow ab \mid bA$

$\}$

Sol: To convert CFG into CNF, we have to simplify CFG.

Step 1: Removal of useless symbols

Here, there are not any useless symbols.

Step 2: Removal of empty (ϵ) production

Here, $A \rightarrow \epsilon$ and B is nullable production.

Replacing each occurrence of A by ϵ , we get:

$S \rightarrow aAB \mid AaB \mid aB \mid B$

$A \rightarrow aA \mid a$

$B \rightarrow ab \mid bA \mid b$

Step 3: Removal of unit production

Here, unit productions are: $S \rightarrow B$

To remove unit production, $S \rightarrow B$,

$S \rightarrow B$

generate $S \rightarrow ab \mid bA$

So, $S \rightarrow aAB \mid AaB \mid aB \mid ab \mid bA \mid b$

$A \rightarrow aA \mid a$

$B \rightarrow ab \mid bA \mid b$

Step 4: Eliminate terminals from the RHS of the production if they exist with other non-terminals or terminals.

$S \rightarrow CAB | ACB | CB | CD | DA | b$
 $C \rightarrow a$
 $D \rightarrow b$
 $A \rightarrow CA | a$
 $B \rightarrow CD | DA | b$

consider $E \rightarrow a$
 $D \rightarrow b$
 single exist $\rightarrow$ change to non-terminals
 like $S \rightarrow b$

Step 5: ~~Here again~~ Eliminate RHS with more than two terminals
 Here again $S \rightarrow CAB$ and $S \rightarrow ACB$ are not in required form

So, we can perform as below:

$S \rightarrow EB | FB | CB | CD | DA | b$
 $E \rightarrow CA$
 $F \rightarrow AC$
 $C \rightarrow a$
 $D \rightarrow b$
 $A \rightarrow CA | a$
 $B \rightarrow CD | DA | b$

Tutorial:

Convert below CFG into CNF

$S \rightarrow AAC$
 $A \rightarrow AAB | \epsilon$
 $C \rightarrow aC | a$

This is the required CNF.

$S \rightarrow AC | DC | a$
 $A \rightarrow EC | DB$
 $C \rightarrow DC | a$
 $D \rightarrow a$
 $B \rightarrow b$
 $E \rightarrow DA$

35

3.16 Greibach Normal Form

A grammar $G = (V, \Sigma, R, S)$ is said to be in GNF if all the production of grammar are in the form:

$A \rightarrow a\alpha$
 $A \rightarrow a$

where,

$a \in \Sigma$ and α is a string of non-terminals (or variables)

Eg:

$S \rightarrow aAB | bBB | bB$
 $A \rightarrow aA | bB | b$
 $B \rightarrow b$

is in GNF

GNF are used to construct a push-down automata (PDA)

Steps for converting CFG into GNF:

1. Convert the grammar into CNF
2. If the grammar exists left recursion, eliminate it
3. In the grammar, convert the given production rule into GNF form.

Note:

Left Recursion

A production is said to have left recursion if the leftmost variable of its RHS is same as the variable in its LHS.

Eg. $S \rightarrow Sa | \epsilon$

is left recursion grammar

$S \rightarrow EC$
 $C \rightarrow aC | \epsilon$

Eliminate Left Recursion

$A \rightarrow A\alpha | \beta$

$\downarrow$

$A \rightarrow \beta C$
 $C \rightarrow \alpha C | \epsilon$

3. CONTEXT-FREE GRAMMAR

3.17 Numerical

1. Convert the grammar $S \rightarrow AB/BC$ into GNF

$A \rightarrow aB/bA/a$
 $B \rightarrow bB/cC/b$
 $C \rightarrow c$

Here the grammar is not in CNF. $A \rightarrow aB/bA/a$ and $B \rightarrow bB/cC/b$ are not in CNF. So, we simply try to transform $S \rightarrow AB/BC$ into GNF.

The production $S \rightarrow AB/BC$ is not in GNF.
 On applying the substitution rule we immediately get equivalent GNF

$S \rightarrow aBB/bAB/aB/bBC/cCC/bC$
 $A \rightarrow aB/bA/a$
 $B \rightarrow bB/cC/b$
 $C \rightarrow c$

*If we convert it into CNF, we would get:-
 $S \rightarrow AB/BC$
 $A \rightarrow aB/bA/a$
 $B \rightarrow bB/cC/b$
 $C \rightarrow c, D \rightarrow a, E \rightarrow b$*

2. Convert the grammar $S \rightarrow aBASA/aBA$ into GNF

Sol: If we introduce new variables A and B and productions as
 $A \rightarrow a$
 $B \rightarrow b$ and substitute into given grammar, we obtain

$S \rightarrow aBASA/aBA$
 $A \rightarrow a$
 $B \rightarrow b$

This is in GNF

3. Convert the grammar $S \rightarrow XB/AA$
 $A \rightarrow a/SA$
 $B \rightarrow b$
 $X \rightarrow a$ into GNF

Sol: The given grammar G is already in CNF.

1) The production rule $A \rightarrow SA$ is not in GNF. ~~Since~~ substitute $S \rightarrow XB/AA$ in production rule $A \rightarrow SA$

$S \rightarrow XB/AA$
 $A \rightarrow a/XBA/AAA$
 $B \rightarrow b$
 $X \rightarrow a$

*is in GNF
 B is GNF*

2. The production rule $S \rightarrow XB$ and $A \rightarrow XBA$ is not in GNF.
 Substitute $X \rightarrow a$ in production rule $S \rightarrow XB$ and $B \rightarrow XBA$

$S \rightarrow aB/AA$
 $A \rightarrow a/aBA/AAA$
 $B \rightarrow b$
 $X \rightarrow a$

Now in GNF

3. Remove left recursion ($A \rightarrow AAA$), we get.

$S \rightarrow aB/AA$
 $A \rightarrow aC/aBAC$
 $C \rightarrow AAC/E$
 $B \rightarrow b$
 $X \rightarrow a$

$A \rightarrow aC/aBAC$
 $\rightarrow a/aBA$

$A \rightarrow AAA/a$
 $\downarrow$
 $A \rightarrow aC$
 $C \rightarrow AAC/E$

4. Remove null production. $C \rightarrow E$

End of Chapter 3

$S \rightarrow aB \mid AA$

$A \rightarrow aC \mid aBAC \mid a \mid aBA \checkmark$ is in GNF

$C \rightarrow AAC \mid AA$

$B \rightarrow b$

$X \rightarrow a$

5. The production rule $S \rightarrow AA$ is not in GNF, so

substitute $A \rightarrow aC \mid aBAC \mid a \mid aBA$ in production rule $S \rightarrow AA$

$S \rightarrow aB \mid aCA \mid aBACA \mid aA \mid aBAA$

$A \rightarrow aC \mid aBAC \mid a \mid aBA$

$C \rightarrow AAC$

$C \rightarrow aCA \mid aBACA \mid aA \mid aBAA$

$B \rightarrow b$

$X \rightarrow a$

6. The production rule $C \rightarrow AAC$ is not in GNF,

substitute $A \rightarrow aC \mid aBAC \mid a \mid aBA$ in production rule $C \rightarrow AAC$

$S \rightarrow aB \mid aCA \mid aBACA \mid aA \mid aBAA$

$A \rightarrow aC \mid aBAC \mid a \mid aBA$

$C \rightarrow aCAC \mid aBACAC \mid aAC \mid aBAAC$

$C \rightarrow aCA \mid aBACA \mid aA \mid aBAA$

$B \rightarrow b$

$X \rightarrow a$

This is now in GNF

Check Left recursion
 $A \rightarrow aA \mid b$

$\downarrow$

$A \rightarrow AC$

$C \rightarrow aC \mid e$

Left recursion
A production of grammar is said to have left recursion if the leftmost variable of its RHS is same as the variable in its LHS
 $S \rightarrow Sa \mid e$
(Left recursive grammar)

4.1 Pushdown Automata

- Pushdown automata is a way to implement a CFG in the same way we design DFA for a regular grammar.
- A DFA can remember a finite amount of information, but a PDA can remember an infinite amount of information.
- Pushdown automata is simply an NFA augmented with an "external stack memory". The addition of stack is used to provide a last-in-first-out memory management capability to Pushdown automata. Pushdown automata can store an unbounded amount of information on the stack. It can access a limited amount of information on the stack. A PDA can push an element onto the top of the stack and pop off an element from the top of the stack. To read an element into the stack, the top elements must be popped off and are lost.
- A PDA is more powerful than FA. Any language which can be acceptable by FA can also be acceptable by PDA. PDA also accepts a class of language which even cannot be accepted by FA. Thus, PDA is much more superior to FA.

PDA Components:

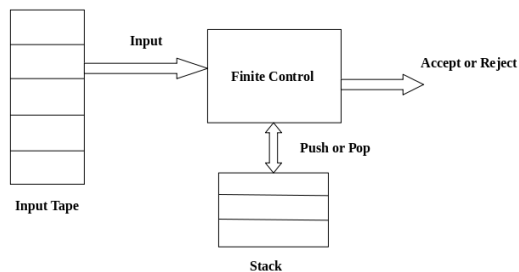


Fig: Pushdown Automata

- **Input tape:** The input tape is divided in many cells or symbols. The input head is read-only and may only move from left to right, one symbol at a time.
- **Finite control:** The finite control has some pointer which points the current symbol which is to be read.

- **Stack:** The stack is a structure in which we can push and remove the items from one end only. It has an infinite size. In PDA, the stack is used to store the items temporarily.

4.2 Formal definition of PDA

The PDA can be defined as a collection of 7 components:

- Q : the finite set of states
- Σ : the input set
- Γ : a stack symbol which can be pushed and popped from the stack
- q0 : the initial state
- Z : a start symbol which is in Γ .
- F : a set of final states
- Δ : mapping function which is used for moving from current state to next state. $(Q \times \Sigma^* \times \Gamma^*) \rightarrow (Q \times \Gamma^*)$

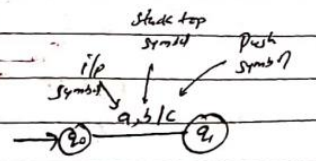
4.3 Graphical notation of PDA

→ We can use transition diagram to represent a PDA where:

- any state is represented by a node in diagram
- any are labelled with start indicates to start state, and
- any doubly circled states are accepting or final states.
- the arc correspond to transition of PDA and:
 - are labelled as $a, x/\alpha$ means the transition $\delta(q, a, x) = (p, \alpha)$
 - for arc from state q to p, this arc label tells what input is used and also gives the old and new ~~leaves~~ ^{leaves} of stack.

4. PUSHDOWN AUTOMATA

The following diagram shows a transition in a PDA from a state q_1 to state q_2 , labeled as $q_1 \xrightarrow{a,b|c} q_2$. This means that at state q_1 , if we encounter an input string 'a' and top symbol of the stack is b, then we pop b and push c on the top of the stack, and move to state q_2 .



4.4 Moves of PDA

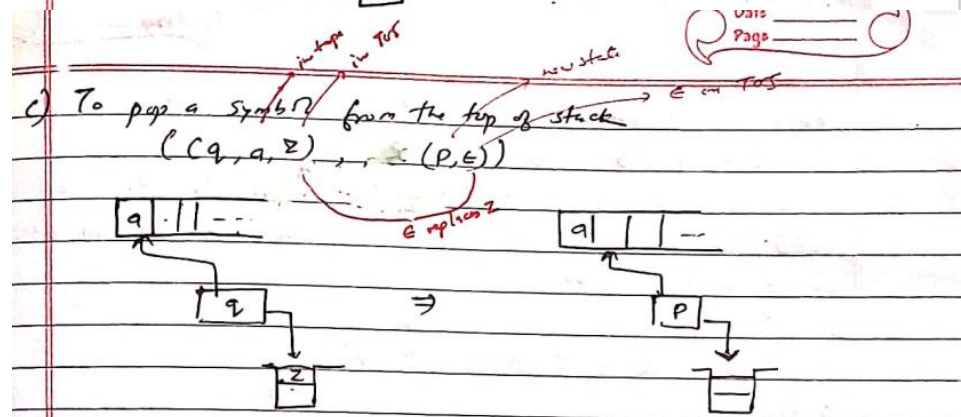
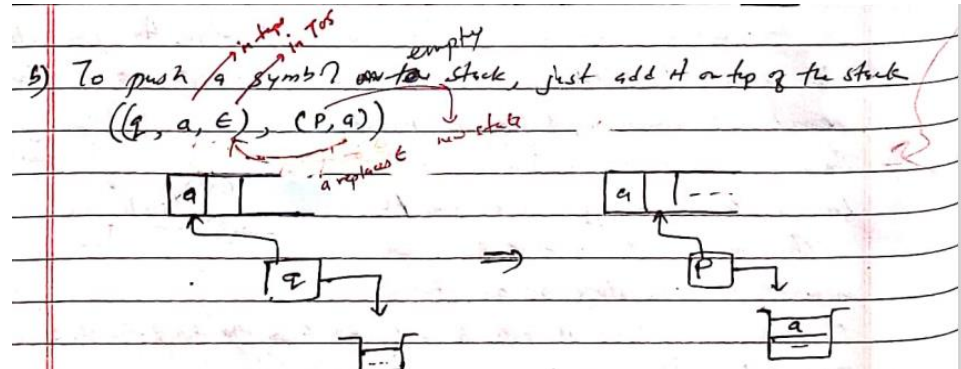
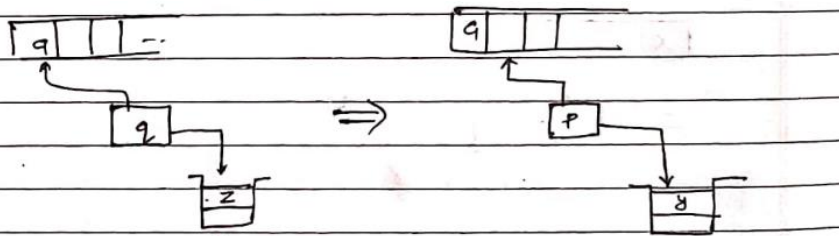
Let us see some moves of PDA in the form of mapping, that

$$(Q \times \Sigma^* \times Z^*) \rightarrow (Q \times Z^*)$$

a) Interpretation of $(q, a, z) \rightarrow (p, y)$

- PDA is in q state

- If z is on top of stack, and input is a , replace z by y on the top of the stack and enter state p

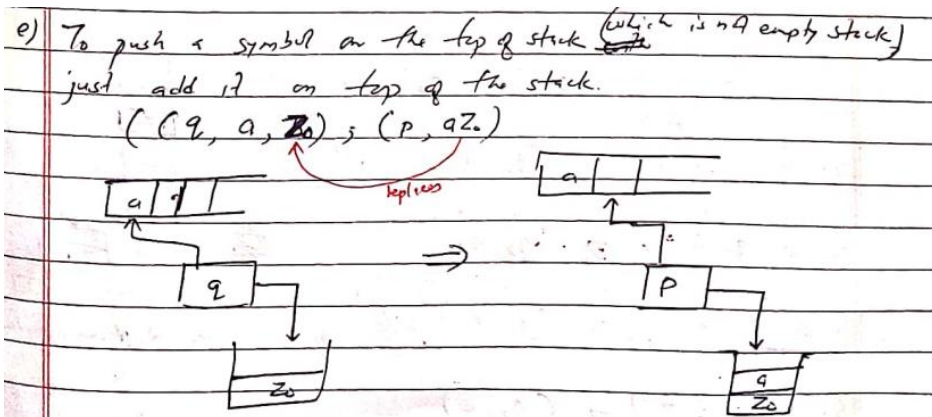


d) PDA also behaves as do nothing machine, just read the input from the input and don't make any change in the state and symbol at the stack

$$((p, a, \epsilon), (p, \epsilon))$$

$$a, \epsilon | \epsilon$$

In this case, PDA just reads input a and without making any change, the stack or at stack top, waits for another input from input tape.



4.5 Numerical

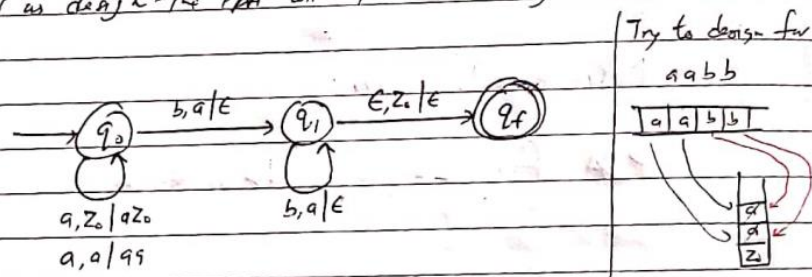
1. Design a PDA that accepts $L = \{a^n b^n \mid n \geq 1\}$

By analysis it is clear that:

PDA accepts $ab, aabb, aaabbb, \dots$

PDA rejects $\epsilon, a, b, aab, abb, aaabb, bbaa, \dots$

Let us design the PDA with the state diagram.



Hence, the PDA is:-

$$P = \{Q, \Sigma, \Gamma, \delta, q_0, z_0, F\}$$

where,

$$Q = \{q_0, q_1, q_f\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{a, b, z_0\}$$

$$q_0 = q_0$$

$$z_0 = z_0$$

$$F = \{q_f\}$$

Transition δ is given by:-

$$1. \delta(q_0, a, z_0) \rightarrow (q_0, az_0)$$

$$2. \delta(q_0, a, a) \rightarrow (q_0, aa)$$

$$3. \delta(q_0, b, a) \rightarrow (q_1, \epsilon)$$

$$4. \delta(q_1, b, a) \rightarrow (q_1, \epsilon)$$

$$5. \delta(q_1, \epsilon, z_0) \rightarrow (q_f, \epsilon)$$

Now take the string $aabbb$

S.NO	state	string	stack	Transition used
1.	q_0	$aabbb$	z_0	—
2.	q_0	$aabbb$	az_0	1
3.	q_0	$abbb$	$aaaz_0$	2
4.	q_0	bbb	$aaaaz_0$	2
5.	q_1	bb	$aaaz_0$	3
6.	q_1	b	$aaaz_0$	4
7.	q_1	ϵ	z_0	5
8.	q_f	ϵ	ϵ	5

$\therefore$ Given string is accepted.

4. PUSHDOWN AUTOMATA

Take another string abb that should be rejected.

S.No.	State	String	Stack	Transition used
1	q_0	abb	Z_0	—
2	q_0	bb	aZ_0	1
3	q_1	b	Z_0	4
4	Cannot move forward.			

Hence the string is rejected.

2. Design a PDA for that accepts:

$L = \{w/w \in \{a,b\}^* \text{ and } a's \text{ and } b's \text{ are equal}\}$

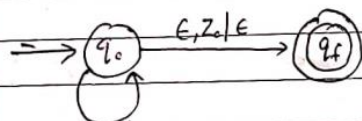
Solⁿ: By analysis it is clear that the PDA accepts any combination of a and b where the no of a 's and b 's are equal.

PDA accepts: $aabb, ab, abab, ababba, \dots$

PDA rejects: $a, b, abb, bba, aba, bab, abbab, \dots$

Now

Let us design the PDA as:-



$a, Z_0 \mid aZ_0$
 $b, Z_0 \mid bZ_0$
 $a, a \mid aa$
 $b, b \mid bb$
 $a, b \mid \epsilon$
 $b, a \mid \epsilon$

Try to design for

$a \mid a \mid b \mid b$
 $a \mid b \mid a \mid b$
 $b \mid a \mid b \mid a$



Hence, the PDA is:

$P = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$

where, transition δ is given by:-

- $Q = \{q_0, q_f\}$
 $\Sigma = \{a, b\}$
 $\Gamma = \{a, b, Z_0\}$
 $q_0 = \{q_0\}$
 $Z_0 = Z_0$
 $F = \{q_f\}$
- 1) $\delta(q_0, a, Z_0) \rightarrow (q_0, aZ_0)$
 2) $\delta(q_0, b, Z_0) \rightarrow (q_0, bZ_0)$
 3) $\delta(q_0, a, a) \rightarrow (q_0, aa)$
 4) $\delta(q_0, b, b) \rightarrow (q_0, bb)$
 5) $\delta(q_0, a, b) \rightarrow (q_0, \epsilon)$
 6) $\delta(q_0, b, a) \rightarrow (q_0, \epsilon)$
 7) $\delta(q_0, \epsilon, Z_0) \rightarrow (q_f, \epsilon)$

Let us check for the string $abbaab$

Step	State	String	Stack	Transition used
1.	q_0	$abbaab$	Z_0	—
2.	q_0	$bbaab$	aZ_0	1
3.	q_0	$baab$	Z_0	6
4.	q_0	aab	bZ_0	2
5.	q_0	ab	Z_0	5
6.	q_0	b	aZ_0	1
7.	q_0	ϵ	Z_0	6
8.	q_f	ϵ	ϵ	7

Hence, the PDA enters q_f state and stack is empty. Hence, the string is accepted by PDA.

Also let us check another string ababb that should be rejected.

step	state	string	stack	Transition used
1.	q_0	ababb	z_0	—
2.	q_0	babb	az_0	1
3.	q_0	abb	z_0	6
4.	q_0	bb	az_0	1
5.	q_0	b	z_0	6
6.	q_0	ϵ	bz_0	2
7.	can't move forward.			

Here, PDA does not enter q_f state and also stack is not empty.

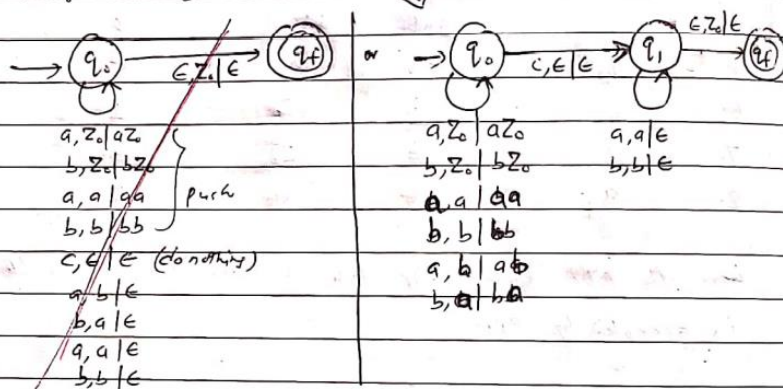
Hence, the string ababb is not accepted by our PDA.

3. Design for $L = \{wcw^r \mid w \in \{a,b\}^*\}$

Sol: Accepted string: abcb, ababcbaba, acac, ---

Rejected string: abcab, abca, cba, ---

Design of state diagram:



Hence, the PDA is: $P = \{Q, \Sigma, \Gamma, \delta, q_0, z_0, F\}$

where, Transition rules (δ) is given by: -

- $Q = \{q_0, q_1, q_f\}$
 $\Sigma = \{a, b\}$
 $\Gamma = \{a, b, z_0\}$
 $q_0 = \{q_0\}$
 $z_0 = \{z_0\}$
 $F = \{q_f\}$
- $(q_0, a, z_0) \rightarrow (q_0, az_0)$
 - $(q_0, b, z_0) \rightarrow (q_0, bz_0)$
 - $(q_0, a, a) \rightarrow (q_0, qa)$
 - $(q_0, b, b) \rightarrow (q_0, qb)$
 - $(q_0, a, b) \rightarrow (q_0, ab)$
 - $(q_0, b, a) \rightarrow (q_0, ba)$
 - $(q_0, \epsilon, \epsilon) \rightarrow (q_1, \epsilon)$
 - $(q_1, a, a) \rightarrow (q_1, qa)$
 - $(q_1, b, b) \rightarrow (q_1, qb)$
 - $(q_1, \epsilon, z_0) \rightarrow (q_f, \epsilon)$

Check for abcbcb

step	state	string	stack	Transition used
1.	q_0	abcbcb	z_0	—
2.	q_0	bcbcb	az_0	1
3.	q_0	cbcb	baZ_0	6
4.	q_0	bcb	$bbaz_0$	4
5.	q_1	bb	$bbaz_0$	7
6.	q_1	b	baZ_0	9
7.	q_1	ϵ	aZ_0	8
8.	q_1	ϵ	z_0	8
9.	q_f	ϵ	ϵ	10

4. PUSHDOWN AUTOMATA

Since, we reached q_f (final state) and the stack is empty, the PDA is accepted.

Check for $abc a$

S.No.	state	String	stack	Transition
1.	q_0	$abc a$	Z_0	—
2.	q_0	$bc a$	aZ_0	1
3.	q_0	$c a$	baZ_0	6
4.	q_0	a	baZ_0	7
5.	No further movement.			

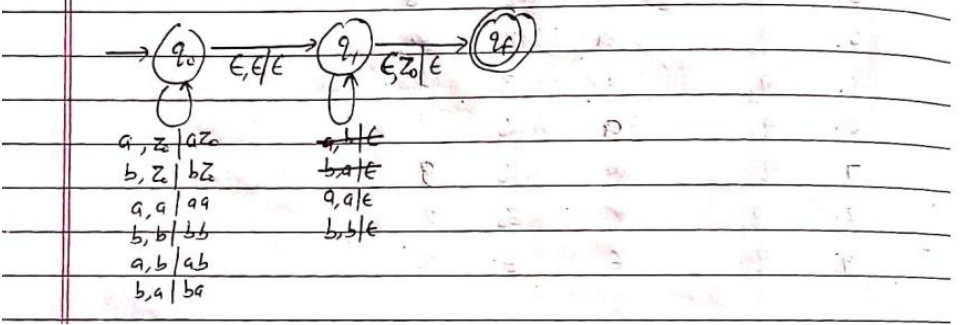
So, PDA rejects this string.

4. Design ^{PDA} for $L = \{ w w^R : w \in \{a, b\}^+ \}$

Accepted string: $a^2, b^2, abb^2, baab^2, aabba^2, \dots$

Rejected string: $abab, baba, baabab, aabbaa, \dots$

PDA design for with state diagram:



Hence rep^d PDA is: $P = \{Q, \Sigma, \Gamma, \delta, Z_0, F, q_0\}$

Where, Transition rule (δ) is given by:-

$Q = \{q_0, q_1, q_f\}$ 1) $(q_0, a, Z_0) \rightarrow (q_0, aZ_0)$

$\Sigma = \{a, b\}$ 2) $(q_0, b, Z_0) \rightarrow (q_0, bZ_0)$

$\Gamma = \{a, b, Z_0\}$ 3) $(q_0, a, a) \rightarrow (q_0, aa)$

$q_0 = \{q_0\}$ 4) $(q_0, b, b) \rightarrow (q_0, bb)$

$F = \{q_f\}$ 5) $(q_0, a, b) \rightarrow (q_0, ab)$

$Z_0 = \{Z_0\}$ 6) $(q_0, b, a) \rightarrow (q_0, ba)$

7) $(q_0, \epsilon, \epsilon) \rightarrow (q_1, \epsilon)$

8) $(q_1, a, a) \rightarrow (q_1, \epsilon)$

9) $(q_1, b, b) \rightarrow (q_1, \epsilon)$

10) $(q_1, \epsilon, Z_0) \rightarrow (q_f, \epsilon)$

Test for string $aabbaa$ (which should be accepted)

Step	state	string	stack	Transition used
1.	q_0	$aabbaa$	Z_0	—
2.	q_0	$abbaa$	aZ_0	1
3.	q_0	$bbaa$	aaZ_0	3
4.	q_0	baa	$baaZ_0$	6
5.	q_1	baa	$baaZ_0$	7
6.	q_1	aa	aaZ_0	9
7.	q_1	a	aZ_0	9
8.	q_1	ϵ	Z_0	9
9.	q_f	ϵ	ϵ	10

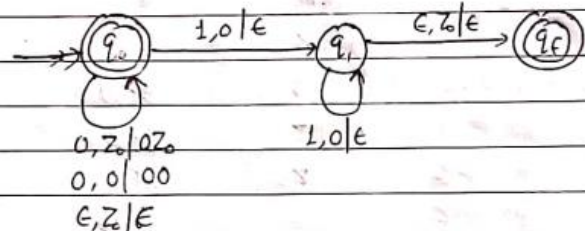
Hence, PDA accepts $aabbaa$

5) Construct a PDA that accepts $L = \{0^n 1^n \mid n \geq 0\}$

PDA accepts: $\epsilon, 01, 0011, 000111, \dots$

PDA rejects: $0, 1, 00, 001, 110, 0101, 1010, \dots$

PDA design with state diagram:



Hence, PDA is:-

$$P = \{Q, \Sigma, \Gamma, Z_0, q_0, F, \delta\}$$

where,

$$Q = \{q_0, q_1, q_f\} \quad \text{Transition rule } (\delta) \text{ given by:-}$$

$$\Sigma = \{0, 1\} \quad 1. (q_0, 0, Z_0) \rightarrow (q_0, 0Z_0)$$

$$\Gamma = \{0, 1, Z_0\} \quad 2. (q_0, 0, 0) \rightarrow (q_0, 00)$$

$$q_0 = \{q_0\} \quad 3. (q_0, \epsilon, Z_0) \rightarrow (q_0, \epsilon)$$

$$Z_0 = \{Z_0\} \quad 4. (q_0, 1, 0) \rightarrow (q_1, \epsilon)$$

$$F = \{q_0, q_f\} \quad 5. (q_1, 1, 0) \rightarrow (q_1, \epsilon)$$

$$6. (q_1, \epsilon, Z_0) \rightarrow (q_f, \epsilon)$$

Test for ϵ

Step	State	String	Stack	Transition
1.	q_0	ϵ	Z_0	—
2.	q_0	ϵ	ϵ	3

$\therefore$ PDA accepts ϵ

Test for 0011

Step	State	String	Stack	Transition
1.	q_0	0011	Z_0	—
2.	q_0	011	$0Z_0$	1
3.	q_0	11	$00Z_0$	2
4.	q_1	1	$0Z_0$	4
5.	q_1	ϵ	Z_0	5
6.	q_f	ϵ	ϵ	6

$\therefore$ PDA accepts 0011

Test for 00111

Step	State	String	Stack	Transition
1.	q_0	00111	Z_0	—
2.	q_0	0111	$0Z_0$	1
3.	q_0	111	$00Z_0$	2
4.	q_1	11	$0Z_0$	4
5.	q_1	1	Z_0	5
6.	No movement			

$\therefore$ PDA rejects 00111

6. Design a PDA that accepts proper parentheses:

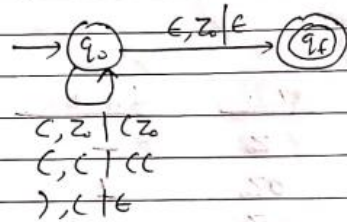
ie. $()$, $()()$, $(())$, $()()()$, $((()))$, —

PDA accepts: $()$, $()()$, $(())$, $((()))$, —

rejects: $)()$, $(,)$, $(()$, $(())$, $()()$, $((())$, —

4. PUSHDOWN AUTOMATA

PDA Design as:



δ is: -

- 1) $(q_0, C, Z_0) \rightarrow (q_0, CZ_0)$
- 2) $(q_0, C, C) \rightarrow (q_0, CC)$
- 3) $(q_0, C, \epsilon) \rightarrow (q_0, \epsilon)$
- 4) $(q_0, \epsilon, Z_0) \rightarrow (q_1, \epsilon)$

Validation

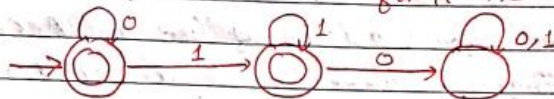
Test for: $()$, $(())$, $(C)(C)$

Should be accepted --

Test for: $($, $(($, $(C)($

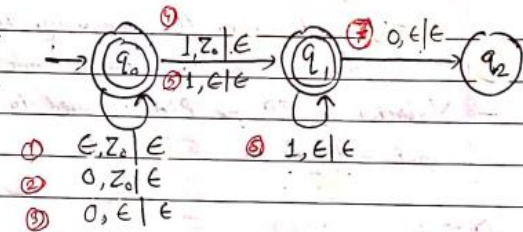
should be rejected!

For below DFA, Write the RE for it and design a PDA



Here, possible strings are: $\epsilon, 0, 00, 000, 1, 11, 111, 001, 0011, \dots$
Hence, the RE is, 0^*1^*

PDA design is: -



Validation

Test for: 0

1. $q_0, 0, Z_0 \rightarrow q_0, 0, Z_0$
2. $q_0, \epsilon, \epsilon \rightarrow q_0, \epsilon, \epsilon$

Test for ϵ

1. $q_0, \epsilon, Z_0 \rightarrow q_0, \epsilon, Z_0$
2. $q_0, \epsilon, \epsilon \rightarrow q_0, \epsilon, \epsilon$

Test for 00

1. $q_0, 00, Z_0 \rightarrow q_0, 00, Z_0$
2. $q_0, 0, \epsilon \rightarrow q_1, 0, \epsilon$
3. $q_1, 0, \epsilon \rightarrow q_1, 0, \epsilon$

Test for 000

$q_0, 000, Z_0 \rightarrow q_0, 000, Z_0$
$q_0, 00, \epsilon \rightarrow q_0, 00, \epsilon$
$q_0, 0, \epsilon \rightarrow q_1, 0, \epsilon$
$q_1, \epsilon, \epsilon \rightarrow q_1, \epsilon, \epsilon$

Test for 01

$q_0, 01, Z_0 \rightarrow q_0, 01, Z_0$
$q_0, 0, \epsilon \rightarrow q_0, 0, \epsilon$
$q_1, \epsilon, \epsilon \rightarrow q_1, \epsilon, \epsilon$

Test for 111

$q_0, 111, Z_0 \rightarrow q_0, 111, Z_0$
$q_1, 11, \epsilon \rightarrow q_1, 11, \epsilon$
$q_1, 1, \epsilon \rightarrow q_1, 1, \epsilon$
$q_1, \epsilon, \epsilon \rightarrow q_1, \epsilon, \epsilon$

Test for 010

$q_0, 010, Z_0 \rightarrow q_0, 010, Z_0$
$q_0, 10, \epsilon \rightarrow q_0, 10, \epsilon$
$q_1, 0, \epsilon \rightarrow q_1, 0, \epsilon$
$q_2, \epsilon, \epsilon \rightarrow q_2, \epsilon, \epsilon$

rejected

Since q_2 is not final state

4.6 Instantaneous Description (ID) of PDA

- ID is an informal notation of how a PDA computes an input string and make a decision that string is accepted or rejected.
- Notation of ID for PDA is used to describe changes in state, input and stack.
- An instantaneous description is a triple (q, w, α) where:

- q describes the current state.
- w describes the remaining input.
- α describes the stack contents, top at the left.

4.7 Turnstile Notation

- $\vdash$ sign describes the turnstile notation and represents one move.
- $\vdash^*$ sign describes a sequence of moves.
- For example,

$$(p, b, T) \vdash (q, w, \alpha)$$

In the above example, while taking a transition from state p to q , the input symbol 'b' is consumed, and the top of the stack 'T' is represented by a new string α .

Another example:

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA.

Define a relation $\vdash$

Suppose, $\delta(q, a, X)$ contains (p, α)

Then for all strings w in Σ^* and β in Γ^*

$$(q, a w, X \beta) \vdash (p, w, \alpha \beta)$$

- This reflects the idea that by consuming a from input and replacing X on top of stack by α , we can go from state q to p .
- What remains on the input w and what is below the TOS as β do not influence the action of PDA.
- We can also use symbol $\vdash^*$ when PDA is understood to represent zero or more moves of PDA.

For eg:

$$(q_0, aabbbbaab, Z_0) \vdash^* (q, \epsilon, \epsilon)$$

4.8 PDA Acceptance

A language can be accepted by Pushdown automata using two approaches:

1. Acceptance by Final State:

The PDA is said to accept its input by the final state if it enters any final state in zero or more moves after reading the entire input.

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$ be a PDA. The language acceptable by the final state can be defined as:

$$L(PDA) = \{w \mid (q_0, w, Z) \vdash^* (p, \epsilon, \epsilon), q \in F\}$$

2. Acceptance by Empty Stack:

On reading the input string from the initial configuration for some PDA, the stack of PDA gets empty.

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$ be a PDA. The language acceptable by empty stack can be defined as:

$$N(PDA) = \{w \mid (q_0, w, Z) \vdash^* (p, \epsilon, \epsilon), q \in Q\}$$

We can define acceptance of any string by PDA in two ways:-

- Acceptance by final state
- Acceptance by empty stack

i) Acceptance by final state:

The PDA is said to accept its input by the final state if it enters any final state in zero or more moves after reading the entire input.

The language acceptable by the final state can be defined as:-

$$L(PDA) = \{w \mid (q_0, w, Z) \vdash^* (p, \epsilon, \epsilon), q \in F\}$$

(ii) Acceptance by empty stack

On reading the input string from the initial configuration for some PDA, the stack of PDA gets empty. The language acceptable by empty stack can be defined as:-

$$L(PDA) = \{w \mid (q, w, z) \vdash^* (p, \epsilon, \epsilon), q \in Q\}$$

4.9 Equivalence of PDA and CFG

- If a grammar G is context free, we can build an equivalent non-deterministic PDA which accepts the language that is produced by the context-free grammar G .

- If P is a PDA, an equivalent CFG can be constructed where, $L(G) = L(P)$

- If $L = N(P_1)$ for some PDA P_1 , then there is a PDA P_2 such that $L = L(P_2)$. That means the language accepted by empty stack PDA will also be accepted by final state PDA.
- If there is a language $L = L(P_1)$ for some PDA P_1 then there is a PDA P_2 such that $L = N(P_2)$. That means language accepted by final state PDA is also acceptable by empty stack PDA.

4.10 CFG to PDA Conversion

Given a CFG, $G = (V, T, P, S)$, we construct a PDA, P that accepts language generated by G i.e. $L(P) = L(G)$.

Non-terminals
terminals
Production
start symbol

The equivalent PDA, P can be defined as:-
 $P = (Q, \Sigma, \Gamma, \delta, q_0, F, Z_0)$

where, $Q = \{q_i\}$

$$\Sigma = T$$

$$\Gamma = V \cup T$$

$$Z_0 = S$$

$$F = \{q_f\}$$

$$q_0 = q$$

and

δ can be defined as:-

i) $\delta(q, \epsilon, A) = \{(q, \alpha)\}$ for each production rule $A \rightarrow \alpha$ in CFG

ii) $\delta(q, a, a) = \{(q, \epsilon)\}$ for each $a \in T$ in CFG.

4.11 Numerical

1. Convert the following grammar to a PDA that accepts the same language. Given $G = (V, T, P, S)$ where P is defined as:-

$$S \rightarrow aSa$$

$$S \rightarrow bSb$$

$$S \rightarrow c$$

Sol: Let PDA be: $P = \{Q, \Sigma, \Gamma, \delta, q_0, F\}$

where $Q = \{q\}$ and δ is defined as:-

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{S, a, b, c\}$$

$$q_0 = \{q\}$$

$$F = \{q\}$$

$$Z_0 = \{S\}$$

Transitions:

$$\delta(q, \epsilon, S) \rightarrow (q, aSa)$$

$$\delta(q, \epsilon, S) \rightarrow (q, bSb)$$

$$\delta(q, \epsilon, S) \rightarrow (q, c)$$

$$\delta(q, a, a) \rightarrow (q, \epsilon)$$

$$\delta(q, b, b) \rightarrow (q, \epsilon)$$

$$\delta(q, c, c) \rightarrow (q, \epsilon)$$

Now, Let us test for $abbcbbba$ which is accepted by CFG, also should be accepted by our PDA.

Stack input state

$$\vdash (q, abbcbbba, S) \vdash (q, abbcbbba, aSa) \quad \text{by rule 1}$$

$$\vdash (q, bbcbbba, Sa) \quad \text{by rule 4}$$

$$\vdash (q, bbcbbba, bSb a) \quad \text{by rule 2}$$

$$\vdash (q, bcbbba, Sb a) \quad \text{by rule 5}$$

$$\vdash (q, bcbbba, bSbba) \quad \text{by rule 2}$$

$$\vdash (q, cbbba, Sbba) \quad \text{by rule 5}$$

$$\vdash (q, cbbba, cSba) \quad \text{by rule 3}$$

$$\vdash (q, bba, bba) \quad \text{by rule 5}$$

$$\vdash (q, ba, ba) \quad \text{by rule 5}$$

$$\vdash (q, a, a) \quad \text{by rule 5}$$

$$\vdash (q, \epsilon, \epsilon) \quad \text{by rule 4}$$

Hence PDA accepted by empty stack

2. Design a PDA for the grammar $G = (V, T, P, S)$ where

$$P \text{ is: } E \rightarrow T \mid E + T$$

$$T \rightarrow F \mid T * F$$

$$F \rightarrow a \mid (E)$$

Sol: Here, $G = (V, T, P, S)$ where

$V = \{T, E, F\}$

$T = \{+, *, (,), a\}$

$S = \{E\}$

$P = \{ E \rightarrow T \mid E + T$

$F \rightarrow T \mid T * F$

$F \rightarrow a \mid (E) \}$

Let the equivalent PDA be:

$P = \{Q, \Sigma, \Gamma, q_0, Z_0, F, \delta\}$

where,

$Q = \{q\}$

$\Sigma = \{+, *, (,), a\}$

$\Gamma = \{T, E, F, +, *, (,), a\}$

$q_0 = \{q\}$

$Z_0 = \{E\}$

$F = \{q\}$

and δ is given by:-

1) $\delta(q, \epsilon, E) \rightarrow \{(q, T), (q, E+T)\}$

2) $\delta(q, \epsilon, T) \rightarrow \{(q, F), (q, T * F)\}$

3) $\delta(q, \epsilon, F) \rightarrow \{(q, a), \{(q, (E))\}$

4) $\delta(q, a, a) \rightarrow (q, \epsilon)$

5) $\delta(q, +, +) \rightarrow (q, \epsilon)$

For Non terminals

For terminals

4. PUSHDOWN AUTOMATA

- c) $\delta(q, *, *) \rightarrow (q, \epsilon)$
- d) $\delta(q, c, c) \rightarrow (q, \epsilon)$
- e) $\delta(q, \epsilon, \epsilon) \rightarrow (q, \epsilon)$

Now trace for $a + (a * a)$

Using CFG	Using PDA
$E \rightarrow E + T$	$(q, a + (a * a), E)$ <i>input stack</i>
$\rightarrow T + T$	$\vdash (q, a + (a * a), E + T)$ by rule 1
$\rightarrow E + T$	$\vdash (q, a + (a * a), T + T)$ by rule 1
$\rightarrow a + T$	$\vdash (q, a + (a * a), F + T)$ by rule 2
$\rightarrow a + F$	$\vdash (q, a + (a * a), a + T)$ by rule 3
$\rightarrow a + (E)$	$\vdash (q, a + (a * a), + T)$ by rule 4
$\rightarrow a + (T)$	$\vdash (q, a + (a * a), T)$ by rule 5
$\rightarrow a + (T * F)$	$\vdash (q, a + (a * a), F)$ by rule 2
$\rightarrow a + (F * F)$	$\vdash (q, a + (a * a), (E))$ by rule 3
$\rightarrow a + (a * F)$	$\vdash (q, a + (a * a), (E))$ by rule 7
$\rightarrow a + (a * a)$	$\vdash (q, a + (a * a), T)$ by rule 1
	$\vdash (q, a * a, T + F)$ by rule 2
	$\vdash (q, a * a, F * F)$ by rule 2
	$\vdash (q, a * a, a + F)$ by rule 3
	$\vdash (q, * a, * F)$ by rule 4
	$\vdash (q, a, F)$ by rule 6
	$\vdash (q, a, a)$ by rule 3
	$\vdash (q, \epsilon, \epsilon)$ by rule 4
	$\vdash (q, \epsilon, \epsilon)$ by rule 8

Thus PDA accepted by empty stack

3. Design a PDA for the grammar G , $S \rightarrow OSIS$

$S \rightarrow ISoS$

$S \rightarrow \epsilon$

sol. Here, CFG is $C(G) = \{V, T, P, S\}$

where, $V = \{S\}$

$T = \{0, 1\}$

$S = \{S\}$

$P = \{S \rightarrow OSIS, ISoS, \epsilon\}$

Let us test string for: 011001 which is accepted by CFG, also should be accepted by our PDA.

$(q, 011001, S)$

Let PDA is $P = \{Q, \Sigma, \Gamma, \delta, Z_0, S, F\}$

where, $Q = \{q\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, S\}$

$S = \{q\}$

$F = \{q\}$

$Z_0 = \{S\}$

δ is defined as:-

1) $\delta(q, \epsilon, S) \rightarrow (q, OSIS)$

2) $\delta(q, \epsilon, S) \rightarrow (q, ISoS)$

3) $\delta(q, \epsilon, S) \rightarrow (q, \epsilon)$

4) $\delta(q, 0, 0) \rightarrow (q, \epsilon)$

5) $\delta(q, 1, 1) \rightarrow (q, \epsilon)$

6) $\delta(q, \epsilon, \epsilon) \rightarrow (q, \epsilon)$

$\vdash (q, 011001, OSIS)$ by rule 1

$\vdash (q, 11001, SIS)$ by rule 4

$\vdash (q, 11001, ISOSIS)$ by rule 2

$\vdash (q, 1001, SOSIS)$ by rule 5

$\vdash (q, 1001, ISOSIS)$ by rule 2

$\vdash (q, 001, SOSIS)$ by rule 5

$\vdash (q, 001, OSIS)$ by rule 3

$\vdash (q, 01, SOSIS)$ by rule 4

$\vdash (q, 01, OSIS)$ by rule 3

$\vdash (q, 1, SIS)$ by rule 4

$\vdash (q, 1, IS)$ by rule 3

$\vdash (q, \epsilon, S)$ by rule 5

$\vdash (q, \epsilon, \epsilon)$ by rule 3

Thus PDA accepted by empty stack

8.) Design a PDA for $L = \{a^n b^{2n} : n \geq 0\}$

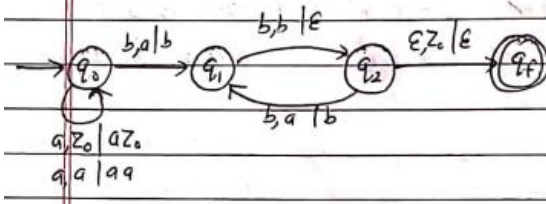
PDA accepts: $abb, aabbb, aaabbbbbb, \dots$

PDA rejects: $a, b, ab, ba, aab, abbb, \dots$

PDA design D:-

Try for.

$abb, aaabbbb$



Tutorial:

1. Design a PDA for $L = \{a^n b^{2n} : n \geq 0\}$
2. Construct a PDA for the RE $r = 0^*1^+$
3. Construct a PDA for $L = \{a^n b^{n+1} : n \geq 0\}$
4. Design a PDA for $L = \{a^n b^m : n \geq m \geq 0\}$
5. Construct a PDA for $L = \{a^3 b^n c^n : n \geq 0\}$

4.12 Properties of CFL

Context free Languages (CFL) are accepted by PDA but not by FA.

1. Union:

If L_1 and L_2 are two CFLs, then their union $L_1 \cup L_2$ is also CFL.

2. Concatenation

If L_1 and L_2 are two CFLs, then $L_1 L_2$ will also be CFL.

3. Kleene Star Closure

If L is CFL, then L^+ will also be CFL.

4. Intersection

If L_1 and L_2 are two CFLs, then their intersection $L_1 \cap L_2$ is not CFL.

5. Complement.

If L_1 is CFL, then $\bar{L}_1$ is not CFL.

4.13 Closure properties of CFL

CFL are closed under:

a) Union

b) Concatenation

c) Kleene Star Operation

4.14 Theorem Proofs

1. Theorem 1:

The family of CFL is closed under Union, Concatenation and Kleene-star closure.

Proof: Let L_1 and L_2 be two CFLs generated by two context-free grammar respectively as:

$$G_1 = (V_1, \Sigma_1, P_1, S_1)$$

$$G_2 = (V_2, \Sigma_2, P_2, S_2)$$

a) Union

Consider the Language $L(G)$, generated by the following grammar:

$$G = \{V, \Sigma, P, S\}$$

Where,

$$V = \{V_1 \cup V_2 \cup \{S\}\}$$

$$\Sigma = \{\Sigma_1 \cup \Sigma_2\}$$

$$S = \{S\}$$

$$P = \{P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}\}$$

Let us choose a string $W \in (V_1 \cup V_2)^*$

If $S_1 \xrightarrow{*} W$ or $S_2 \xrightarrow{*} W$, and in our grammar

$$S \rightarrow S_1 \mid S_2$$

Hence, S will also lead to W

Here, G is clearly a CFG because it can generate the same string as by G_1 and G_2 .

So, we can claim, $L(G) = L(G_1) \cup L(G_2)$

$\therefore$ CFL are closed under union.

Verification:

Suppose, G_1 and G_2 are two CFL where:

For G_1 we have:

$$S_1 \rightarrow a S_1 b$$

$$S_1 \rightarrow \epsilon$$

For G_2 we have:-

$$S_2 \rightarrow c S_2 d$$

$$S_2 \rightarrow \epsilon$$

$$\text{ie. } L(G_1) = \{a^n b^n : n \geq 0\}$$

$$L(G_2) = \{c^n d^n : n \geq 0\}$$

$$\text{Let, } L(G) = L(G_1) \cup L(G_2)$$

$$= \{a^n b^n : n \geq 0\} \cup \{c^n d^n : n \geq 0\}$$

Let us choose a string $aaabbb$

We know: $S \rightarrow S_1 \mid S_2$,

$$\text{So, } S \rightarrow S_1$$

$$\rightarrow a S_1 b$$

$$\rightarrow a a S_1 b b$$

$$\rightarrow a a \epsilon b b$$

$$\rightarrow a a b b$$

$$S \rightarrow S_2$$

$$\rightarrow c S_2 d$$

$$\rightarrow c \epsilon d$$

$$\rightarrow c d$$

For example:

$$L_1 = \{a^n b^n \mid n \geq 0\}$$

$$L_2 = \{c^n d^n \mid n \geq 0\}$$

$$L = L_1 \cup L_2 = \{a^n b^n \cup c^n d^n \mid n \geq 0\} \text{ is also CFL.}$$

Their union says either of two conditions to be true.

b) Concatenation

Let us consider a language $L(G)$ ~~generated by the CFG~~ ^{generated by the CFG}

$$G = (V, \Sigma, P, S)$$

where,

$$V = V_1 \cup V_2 \cup \{S\}$$

$$\Sigma = \Sigma_1 \cup \Sigma_2$$

$$S = \{S\}$$

$$P = \{P_1 \cup P_2 \cup \{S \rightarrow S_1 S_2\}\}$$

Let us choose a string, $w_1 \in L_1$ and $w_2 \in L_2$.

$$\nexists! S_1 \xrightarrow{*}_{G_1} w_1 \quad \text{and} \quad S_2 \xrightarrow{*}_{G_2} w_2$$

We can claim

$$S \xrightarrow{*}_G w_1 w_2 \quad \because S \rightarrow S_1 S_2$$

$$\therefore L(G) = L(G_1) L(G_2)$$

Hence,

CFL are closed under concatenation.

Eg:

$$S_1 \rightarrow a S_1 b \mid \epsilon$$

$$S_2 \rightarrow c S_2 d \mid \epsilon$$

$$S \rightarrow S_1 S_2$$

$$\rightarrow a S_1 b S_2$$

$$\rightarrow a \epsilon b S_2$$

$$\rightarrow a b c S_2 d$$

$$\rightarrow a b c \epsilon d$$

$$\rightarrow a b c d$$

c) Kleene Star

Let $L_1(G_1)$ be a CFL generated by a grammar

$$G_1 = (V_1, \Sigma_1, P_1, S_1)$$

Let Kleene star of language L_1 (i.e. L_1^*) be generated by a grammar $G = (V, \Sigma, P, S)$

where,

$$V = V_1 \cup \{S\}$$

$$\Sigma = \Sigma_1$$

$$P = P_1 \cup \{S \rightarrow S_1 \mid S S_1 \mid \epsilon\}$$

Let us choose a string $w_1 \in L_1$.

$$\nexists! S_1 \xrightarrow{*}_{G_1} w_1, \text{ then we can claim that}$$

$$S \xrightarrow{*}_G (w_1)^* \quad \because S \rightarrow S_1 \mid S S_1 \mid \epsilon$$

4. PUSHDOWN AUTOMATA

$$\therefore L(G) = L(G)^*$$

$\therefore$ CFL are closed under Kleene star

Verification $ab \rightarrow (ab)^*$

$$g: S_1 \rightarrow \cancel{a} \cancel{b} \mid \epsilon \mid abS_1 \mid \epsilon$$

$S \rightarrow S_1$	$S \rightarrow S_1$
$\rightarrow \epsilon$	$\rightarrow \cancel{a} \cancel{b} abS_1$
	$\Rightarrow ab$

$$\begin{aligned} S &\rightarrow SS_1 \\ &\rightarrow \cancel{a} \cancel{b} S_1 \\ &\rightarrow abS_1S_1 \\ &\rightarrow ab \epsilon S_1 \\ &\rightarrow ababS_1 \\ &\rightarrow abab \end{aligned}$$

$$So, ab \rightarrow (ab)^*$$

Theorem 2:

The family of Context Free Language is not closed under intersection and complementation

a) Intersection:

Let us define two CFLs L_1 and L_2 as:-

$$L_1 = \{a^n b^n c^m \mid n \geq 1, m \geq 1\}$$

$$L_2 = \{a^n b^m c^n \mid n \geq 1, m \geq 1\}$$

Here, $L_1 \cap L_2$

$$\begin{aligned} L &= L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 1\} \\ &= \{a^n b^n c^n : n \geq 1\} \end{aligned}$$

But by $L = \{a^n b^n c^n : n \geq 1\}$ is not a CFL. This can be proved by using pumping lemma by contradiction.

Hence, CFLs are not closed under intersection.

b) Complementation:

We know CFL are closed under union.

Let L_1 and L_2 be two CFLs.

Assume, $\bar{L}_1$ and $\bar{L}_2$ are also CFLs. So, we can say that $\bar{L}_1 \cup \bar{L}_2$ are CFL.

By De Morgan's Law,

$$\bar{L}_1 \cup \bar{L}_2 = \overline{L_1 \cap L_2}$$

But $L_1 \cap L_2$ is not a CFL. This contradicts our assumption.

Hence CFL is not closed under complementation.

4.15 Pumping lemma for CFL

- The pumping Lemma for CFL states that for any Context Free Language L , it is possible to find two substrings that can be pumped any number of times, and still be in the same language.
- For any language L , we can break its string into five parts and pump the second and the fourth ~~substring~~ substring.
- Pumping Lemma, here also, is used as a tool to prove that a language is not CFL.
- Because, if any one string does not satisfy its conditions, then the language is not CFL.

4.16 Pumping Lemma statement

If L is a CFL and w be any string in L , there exists an integer n with $|w| \geq n$, then we can write $w = uvxyz$ such that:

- $|vxy| \leq n$
- $|vy| > 0$
- $uv^ixy^iz \in L$ for all $i \geq 0$.

It means two strings v and y can be pumped any number of times, and the resulting string will still be in L .

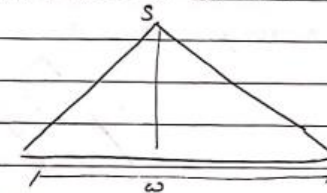
If L is a context free language then L has a pumping length ' n ' such that any string $|S| \geq n$ may be divided into 5 pieces i.e. $S = uvxyz$, such that the following conditions must be true:

- $uv^ixy^iz \in L$ for every $i \geq 0$
- $|vy| > 0$
- $|vxy| \leq n$

4.17 Proof of Pumping Lemma for CFL

- Consider a CNF grammar $G = (V, \Sigma, P, S)$ that has m non-terminals (or variables).
- Let n be the pumping length, and let's choose $n = 2^m$.
- Consider a string w that is produced of length 2^m (i.e. n) derived from the grammar G .
i.e. $|Z| = 2^m$ or more.

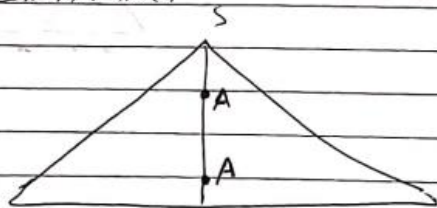
The derivation tree is as shown below:-



- The derivation tree for w must have a path of length $m+1$ or more. Clearly there are $m+1$ non-terminals.

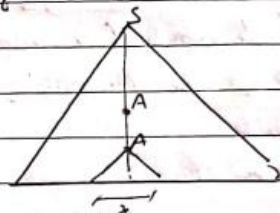
4. PUSHDOWN AUTOMATA

→ But there are only m variables in grammar. So at least two of the last $m+1$ variables on path are same (ie. by pigeonhole principle). Let they be A and A as shown in the derivation tree.



→ The sub-tree subtended at the lower A is as shown below. It generates a string of w .

ie. $A \xRightarrow{*} x$

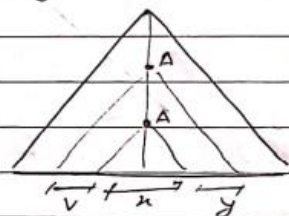


→ The sub-tree subtended at the upper A is as shown below. It derives the string vxy

ie. $A \xRightarrow{*} vxy$

Also,

$A \xRightarrow{*} vAy$

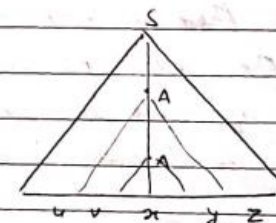


→ The S derives the entire string as $uvxyz$

$S \xRightarrow{*} uvxyz$

Also

$S \xRightarrow{*} uAz$



Now,

$S \xRightarrow{*} uAz$

$\Rightarrow uvAyZ$

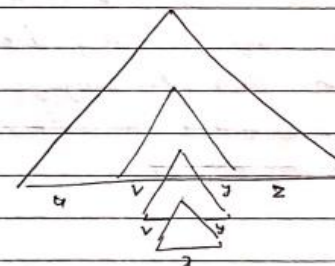
$\Rightarrow uvvAyZ$

$\Rightarrow uvv \dots A \dots yz$

$\Rightarrow uvv \dots x \dots yz$

$\Rightarrow uv^i x y z$ for $i \geq 0$

→ The resulting parse tree is shown as below:-



Hence, we can find any yield of form $uv^i x y z \in L$ for any $i \geq 0$.

3. Prove that the language $L = \{ww \mid w \in \{a,b\}^*\}$ is not context free.

Solⁿ: Assume L is context free, and w be any string in L .

Let n be the pumping length with $|w| \geq n$.

Then, we can decompose $w = uvxyz$ such that $|vy| \leq n$ and $|vy| > 0$.

So it must satisfy the condition $uv^i xy^j z \in L$ for all $i \geq 0$.

Here, let's take ω for the given language L as:-

$$\omega = a^n b^n a^n b^n$$

Taking $n=4$, then ~~w~~ $L = a^4 b^4 a^4 b^4$

Sw

$$Z = a a a a b b b b a a a a b b b b$$

News

decompose z into $z = uvxyz$ such that $|vxy| \leq n$

and 10/17/20

$\overbrace{aa}^4 \overbrace{bb}^6 \overbrace{aa}^2 \overbrace{bb}^2$

Let us take $i=2$,

After pumping V_y , we get:

a a a b b b b a b a a a a b b b b

$$= a^4 b^4 a b a^5 b^4$$

which is not in the grammar.

Hence, given language L is not CFL.

Thus, proved by contradiction.

3. Check if $L = \{a^{2n}b^n \mid n \geq 0\}$ is context free

→ Assume L is context free, and w be any string in L .

Let n be the pumping length with $|w| \geq n$.

Then we can decompose $\omega = uvvz$ such that $|v| \leq n$ and $|v| > 0$.

So it must satisfy the condition $uv^i x y^j z \in L$ for all $i, j \geq 0$.

Here for gives language L,

$$C_0 = a^{27} b^{12}$$

~~= 9999 99 6666~~

Let us take $n = 4$, then, $L = a^3 b^4$

42

$$Z = a a a a a a a a b b b b$$

Now, decompose z into $z = uvwz$ such that $|u| \leq n$ & $|v|$

a a a a a a a a b b b b
 $\underbrace{\hspace{1.5cm}}_u \quad \underbrace{\hspace{1.5cm}}_v \quad \underbrace{\hspace{1.5cm}}_x \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{1.5cm}}_z$

Let us take $i=2$.

After pumping V_g , we get

a. a a a a a a a a a a b b b b

$$= 9^{11} 65$$

which is not in the given grammar

Hence given language L is not CFL

proved by contradiction.

4.19 Decision Algorithms for CFL

Basically three algorithms

- ① Algorithm for deciding Emptiness of CFL
- ② Algorithm for deciding finiteness of CFL
- ③ Membership - The CYK algorithm

1) Emptiness Algorithm

- If we cannot derive any string of terminals from the given grammar, then its language is called as an Empty Language. i.e. $L(G) = \emptyset$

If the

Algorithm:

1. Remove all the useless symbols from the grammar.

A useless symbol is one that does not derive any string of terminals.

2. If the start symbol is found to be useless, then language is empty otherwise not.

Eg. Consider the grammar: $S \rightarrow XY$

$X \rightarrow AX$

$X \rightarrow AA$

$A \rightarrow a$

$Y \rightarrow BY$

$Y \rightarrow BB$

$B \rightarrow b$

Here, $S \rightarrow XY$

$\rightarrow AAY$

$\rightarrow aAY$

$\rightarrow aAY$

$\rightarrow aaBB$

$\rightarrow aabB$

$\rightarrow aabB$

So, $L(G) \neq \emptyset$

2) Algorithm for Finiteness

Algorithm

1. Reduce the grammar by eliminating ϵ , unit and useless production.
2. Draw a directed graph whose nodes are variables of given grammar.

There exists an edge from A to B if there exists a production of the form $A \rightarrow \alpha B \beta$.

3. If cycle in directed graph $\rightarrow$ infinite CFL
else (no cycle) $\rightarrow$ finite CFL

Eg: Check whether Language of the following grammar is finite or not.

$S \rightarrow AB/a$

$A \rightarrow BC/b$

$B \rightarrow CC/c$

Eg: Check whether Language of the following grammar is finite or not.

$S \rightarrow AB/a$

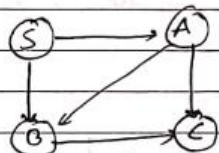
$A \rightarrow BC/b$

$B \rightarrow CC/c$

4. PUSHDOWN AUTOMATA

Solⁿ:

- 1) Grammar is already completely reduced.
- 2) Draw directed graph for S, A, B, C



$$S \rightarrow AB \Rightarrow S \rightarrow A, S \rightarrow B$$

$$A \rightarrow BC \Rightarrow A \rightarrow B, A \rightarrow C$$

$$B \rightarrow CC \Rightarrow B \rightarrow C$$

- 3) Since no cycle, the CFG is finite.

3) The membership - CYK algorithm

- It is used to decide whether a given string belongs to the language of grammar or not.
- named after Cocke-Younger-Kasami
- operates on CFG given in CNF

Example:

For the given grammar, check the acceptance of string $w = baaba$ using CYK algorithm.

$$S \rightarrow AB \mid BC$$

$$A \rightarrow BA \mid a$$

$$B \rightarrow CC \mid b$$

$$C \rightarrow AB \mid a$$

Here, $w = baaba$

Length of word $|w| = 5$

$$\text{No. of sub-strings possible} = n(n+1)/2 = 5 \times 6/2 = 15$$

Let us construct a triangular table and fill the cells.

V_{15}	$\{S, A, C\}$				
V_{14}	$\emptyset$	V_{24}	$\{S, C, A\}$		
V_{13}	$\emptyset$	V_{23}	$\{B\}$	V_{33}	$\{B\}$
V_{12}	$\{A, S\}$	V_{22}	$\{B\}$	V_{32}	$\{S, C\}$
V_{11}	$\{B\}$	V_{21}	$\{A, C\}$	V_{31}	$\{A, C\}$

Working notes:

For 1st row:

b	a	a	b	a
$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$

For 2nd row:

$$V_{12} = V_{11} \cdot V_{21} = \{B\} \cdot \{A, C\} = \{BA, BC\} = \{A, S\}$$

$$V_{22} = V_{21} \cdot V_{31} = \{A, C\} \cdot \{A, C\} = \{AA, AC, CA, CC\} = \{C\} = \{B\}$$

$$V_{32} = V_{31} \cdot V_{41} = \{A, C\} \cdot \{B\} = \{AB, CB\} = \{S, C, \emptyset\} = \{S, C\}$$

$$V_{42} = V_{41} \cdot V_{51} = \{B\} \cdot \{A, C\} = \{BA, BC\} = \{A, S\}$$

For 3rd row:

$$\begin{aligned}
 V_{13} &= V_{11} \cdot V_{22} \cup V_{12} \cdot V_{31} \\
 &= \{B\} \cdot \{B\} \cup \{A, S\} \cdot \{A, C\} \\
 &= \{BB\} \cup \{AA, AC, SA, SC\} \\
 &= \emptyset \cup \emptyset = \emptyset
 \end{aligned}$$

$$\begin{aligned}
 V_{23} &= V_{21} \cdot V_{32} \cup V_{22} \cdot V_{41} \\
 &= \{A, C\} \cdot \{S, C\} \cup \{B\} \cdot \{B\} \\
 &= \{AS, AC, CS, CC\} \cup \{BB\} \\
 &= \{B\} \cup \emptyset = \{B\}
 \end{aligned}$$

$$\begin{aligned}
 V_{33} &= V_{31} \cdot V_{32} \cup V_{32} \cdot V_{31} \\
 &= \{A, c\} \{A, S\} \cup \{S, c\} \{A, c\} \\
 &= \{AA, AS, CA, CS\} \cup \{SA, SC, CA, CC\} \\
 &= \emptyset \cup \{B\} \\
 &= \{B\}
 \end{aligned}$$

For 5th row:

$$\begin{aligned}
 V_{14} &= V_{11} \cdot V_{23} \cup V_{12} \cdot V_{32} \cup V_{13} \cdot V_{41} \\
 &= \{B\} \{B\} \cup \{A, S\} \{S, c\} \cup \{\emptyset\} \{B\} \\
 &= \{BB\} \cup \{AS, AS, SS, SC\} \cup \{B\} \\
 &= \emptyset \cup \emptyset \cup \emptyset \\
 &= \emptyset
 \end{aligned}$$

$$\begin{aligned}
 V_{24} &= V_{21} \cdot V_{33} \cup V_{22} \cdot V_{42} \cup V_{23} \cdot V_{51} \\
 &= \{A, c\} \{B\} \cup \{B\} \{A, S\} \cup \{B\} \{A, c\} \\
 &= \{AB, CB\} \cup \{BA, BS\} \cup \{BA, BC\} \\
 &= \{S, c\} \cup \{A\} \cup \{A, S\} \\
 &= \{S, c, A\}
 \end{aligned}$$

For 5th row:

$$\begin{aligned}
 V_{15} &= V_{11} \cdot V_{24} \cup V_{12} \cdot V_{33} \cup V_{13} \cdot V_{42} \cup V_{14} \cdot V_{51} \\
 &= \{B\} \{S, c, A\} \cup \{A, S\} \{B\} \cup \{\emptyset\} \{A, S\} \\
 &\quad \cup \{\emptyset\} \{A, c\} \\
 &= \{BS, BC, BA\} \cup \{AB, SB\} \cup \{A, S\} \cup \{A, c\} \\
 &= \{S, A\} \cup \{S, c\} \cup \{A, S\} \cup \{A, c\} \\
 &= \{S, A, c\}
 \end{aligned}$$

V14	V24			
V13	V23	V33		
V12	V22	V32	V42	
V11	V21	V31	V41	V51

V15				
V14	V24			
V13	V23	V33		
V12	V22	V32	V42	
V11	V21	V31	V41	V51

Here,

We can see the S in the set V_{15} where $n=5$, $re(V_{15} = \{S, A, c\})$

So, If $S \in V_5$, then $baaba \in L(G)$

Reference:

<https://gatevidyalay.com/cyk-cyk-algorithm/>

End of Chapter 4

5.1 Turing Machine Introduction

Turing machine was invented in 1936 by Alan Turing. It is an accepting device which accepts Recursive Enumerable Language generated by type 0 grammar. There are various features of the Turing machine:

- It has an external memory which remembers arbitrary long sequence of input.
- It has unlimited memory capability.
- The model has a facility by which the input at left or right on the tape can be read easily.
- The machine can produce a certain output based on its input. Sometimes it may be required that the same input has to be used to generate the output. So in this machine, the distinction between input and output has been removed. Thus a common set of alphabets can be used for the Turing machine.

A Turing machine is a hypothetical machine thought of by the mathematician Alan Turing in 1936.

Despite its simplicity, the machine can simulate ANY computer program, no matter how complicated it is!

It is the theoretical foundation of modern computers.

It can do everything that a real computer can do.

At any time, the machine has a head which is positioned over one of the squares on the tape. With this head, the machine can perform three very basic operations:-

1. Read the symbol on the square under the head.
2. Edit the symbol by writing a new symbol or erasing it.
3. Move the tape left or right by one square so that the machine can read and edit the symbol on a neighbouring (adjacent) square.

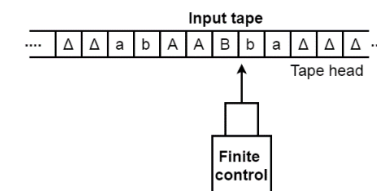
Basic Model of Turing machine

The Turing machine can be modelled with the help of following representation.

1. The input tape is having an infinite number of cells, each cell containing one input symbol and thus the input string can be placed on tape. The empty tape is filled by blank characters.



2. The finite control and the tape head which is responsible for reading the current input symbol. The tape head can move to left to right.
3. A finite set of states through which machine has to undergo.
4. Finite set of symbols called external symbols which are used in building the logic of Turing machine.



5.2 TM as physical computing device

We can visualize a TM as a physical computing device that can be represented as a diagram as shown below:-

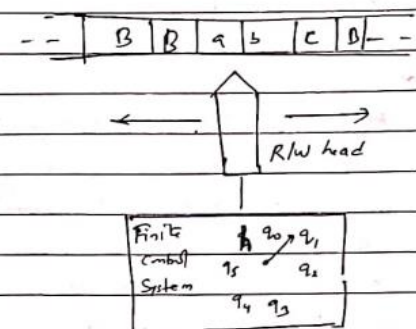


Fig: Turing machine

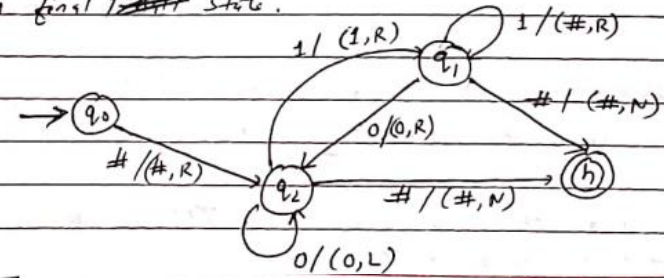
5. TURING MACHINE

Ex: Let $TM = \{Q, \Sigma, \Gamma, \delta, q_0, F, B\}$ where,
 $Q = \{q_0, q_1, q_2, h\}$
 $\Sigma = \{0, 1\}$
 $\Gamma = \{0, 1, \#\}$ and

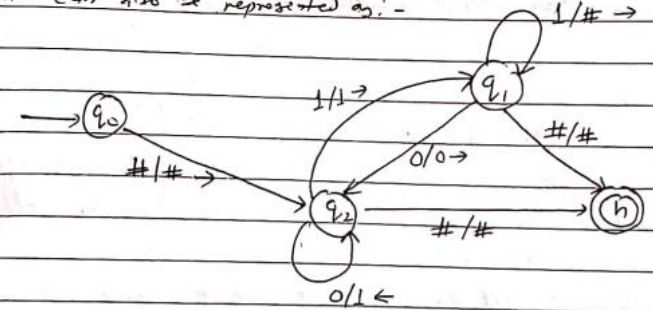
δ is given by the following transition table:-

	0	1	#
q_0	-	-	$(q_1, \#, R)$
q_1	$(q_1, 0, R)$	$(q_1, \#, R)$	$(h, \#, N)$
q_2	$(q_2, 0, L)$	$(q_1, \#, R)$	$(h, \#, N)$
h	-	-	-

The transition diagram for above TM will be as shown below, where we assume that q_0 is the initial state and h is a final state.



This can also be represented as:-



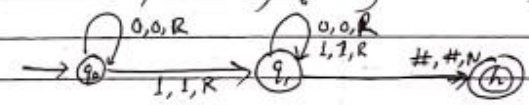
5.7 TM as language acceptor

- A TM works as language acceptor for a language L if it is able to tell whether a string ' w ' belongs to the language L or not. i.e. $w \in L(M)$.
- If the TM halts in a final state, then it accepts string w .
- If the TM halts in a non-final state or never halts, then the TM doesn't accept the string ' w ', i.e. $w \notin L(M)$.

Ex: A TM to accept the set of all strings of 0's and 1's containing at least one 1.

Let $TM = \{Q, \Sigma, \Gamma, \delta, q_0, h, B\}$
 $Q = \{q_0, q_1, h\}$
 $\Sigma = \{0, 1, \#\}$
 $\Gamma = \{0, 1\}$
 $q_0 = \{q_0\}$

State δ is represented by following transition diagram:-



Here, halting state is reached only when at least one 1 is encountered and h works as accepted state.

Test for $\#00110\#$

- $q_0 \#00110\#$
- $q_0 \#00110\#$
- $q_0 \#00110\#$
- $q_1 \#00110\#$
- $q_1 \#00110\#$
- $q_1 \#00110\#$
- $h \#00110\#$

5.8 Numerical: TM design for Language Acceptor

1. Design a TM that accepts $L = \{a^n b^n : n \geq 1\}$

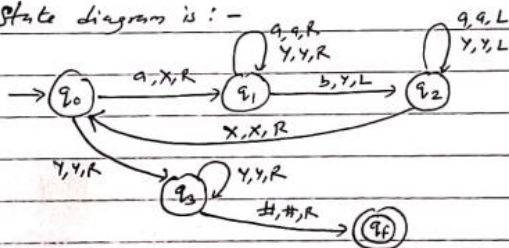
Solⁿ: Let us design assume that the input string is terminated by a blank symbol # at each end of the string.

| # | a | a | a | b | b | b | # |

The TM can be constructed by the following moves:

- TM will change a to X and b to Y until all a's and b's are matched.
- Starting at left end of input, it repeatedly changes a to X and moves over whatever a's and Y's

State diagram is:-



Hence, TM is:-

$$M = \{Q, \Sigma, \Gamma, \delta, q_0, f, B\}$$

where

$$Q = \{q_0, q_1, q_2, q_3, q_f\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{X, Y, \#\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_f\}$$

$$B = \{\#\}$$

δ is given by transition table as:-

	a	b	X	Y	#
q_0	(q_1, X, R)	-	-	(q_3, Y, R)	-
q_1	(q_1, X, R)	(q_2, Y, L)	-	(q_1, Y, R)	-
q_2	(q_2, X, L)	-	(q_0, X, R)	(q_2, Y, L)	-
q_3	-	-	-	(q_3, Y, R)	$(q_f, \#, R)$
q_f	-	-	-	-	-

Verification: Test for $aabbb$

q_0 a a b b b #

$\vdash q_1$ X a b b b #

$\vdash q_1$ X a b b b #

$\vdash q_2$ X a Y b b #

$\vdash q_2$ X a Y b b #

$\vdash q_1$ X X Y b b #

$\vdash q_1$ X X Y b b #

$\vdash q_1$ X X Y b b #

$\vdash q_1$ X X Y b b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

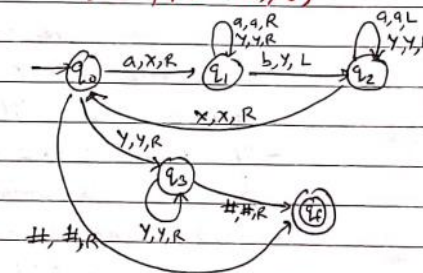
$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #

$\vdash q_2$ X X Y Y b #



Hence accepted

2. Design a TM that accepts $L = \{a^n b^n c^n : n \geq 1\}$

Solⁿ: Let us assume that the input string is terminated by a blank symbol #, at each end of the string.

| a | a | b | b | c | c |

The TM can be constructed by the following moves:-

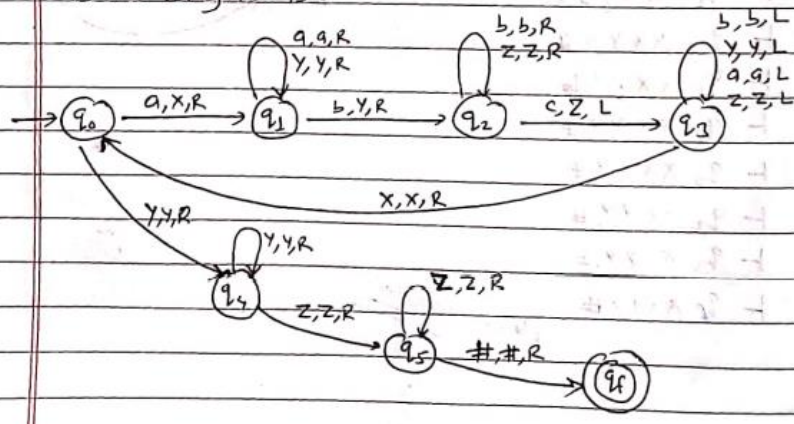
- TM will change a to x, b to y and c to z until all a's, b's and c's are matched.

- Starting at left end of input, it repeatedly changes a to x. and moves over whatever a's and y's it sees until it reaches to state q_1 .

- It changes b to y and moves over whatever b's and z's it sees until it reaches to state q_2 .

- It changes c to z and moves left over z's, y's, b's and a's until it finds an x.

- State diagram is:-



Hence, TM is: $M = \{Q, \Sigma, \Gamma, \delta, q_0, F, B\}$

where, $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_f\}$ $q_0 = \{q_0\}$

$\Sigma = \{a, b, c\}$

$F = \{q_f\}$

$\Gamma = \{x, y, z, \# \}$

$B = \{\# \}$

δ is given by:-

	a	b	c	x	y	z	#
q_0	(q_1, x, R)	-	-	-	(q_4, y, R)	-	-
q_1	(q_1, a, R)	(q_2, y, R)	-	-	(q_1, y, R)	-	-
q_2	-	(q_2, b, R)	(q_3, z, L)	-	-	(q_1, z, R)	-
q_3	(q_3, a, L)	(q_3, b, L)	-	(q_0, x, R)	(q_3, y, L)	(q_3, z, L)	-
q_4	-	-	-	-	(q_4, y, R)	(q_5, z, R)	-
q_5	-	-	-	-	-	(q_5, z, R)	$(q_f, \#, R)$
q_f	-	-	-	-	-	-	-

Verification: for aabbcc

q_0 aabbcc #	$\vdash q_2$ xxYYZc #
$\vdash q_1$ xaabbcc #	$\vdash q_2$ xxYYZc #
$\vdash q_1$ xaabbcc #	$\vdash q_3$ xxYYZc #
$\vdash q_2$ xaybcc #	$\vdash q_3$ xxYYZZ #
$\vdash q_2$ xaybcc #	$\vdash q_3$ xxYYZZ #
$\vdash q_3$ xaybzc #	$\vdash q_1$ xxYYZZ #
$\vdash q_3$ xaybzc #	$\vdash q_0$ xxYYZZ #
$\vdash q_3$ xaybzc #	$\vdash q_4$ xxYYZZ #
$\vdash q_3$ xaybzc #	$\vdash q_4$ xxYYZZ #
$\vdash q_0$ xaybzc #	$\vdash q_5$ xxYYZZ #
$\vdash q_1$ xxYbZc #	$\vdash q_5$ xxYYZZ #
$\vdash q_1$ xxYbZc #	$\vdash q_f$ xxYYZZ #

accepts

GURUKUL

3. Design a TM that accepts $L = \{w \in \{a, b\}^* \mid \text{no. of } a\text{'s and no. of } b\text{'s are equal}\}$.

Solⁿ: Clearly, the language accepts strings like $ab, ba, aabb, abab, abba, aabbaa, abbaab, \dots$

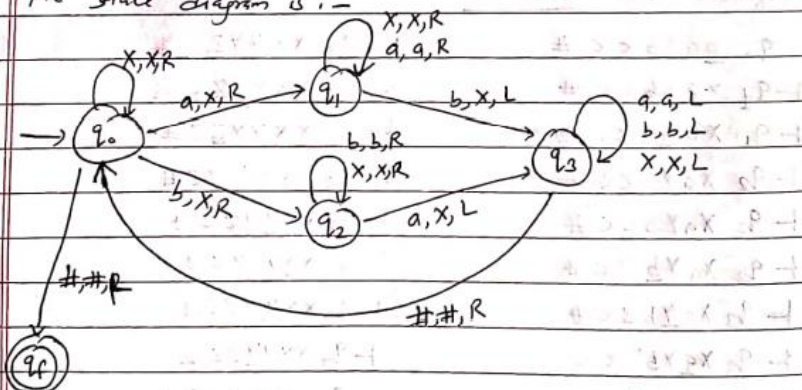
Let us assume that the input string is terminated by a blank symbol $\#$, at each end of the string.

4 | 9 | 9 | 5 | 5 | 6 | 9 | #

TM is constructed by the following moves:-

- TM will change a to X , and find b and change it to X .
- Repeatedly move left and right, each pair of a and b are replaced with X .

The state diagram is:-



Homework 7M) u: $M = \{0, \varepsilon, \tau, \delta, \eta, \text{FB}\}$

where $Q = \{q_0, q_1, q_2, q_3, q_4\}$ $q_0 = \{q_0\}$

$$E = \{g, b\} \quad F = \{g, f\}$$
$$\tau = \{x, \#\} \quad B = \{\#\}$$

δ is given by:-

	<u>a</u>	<u>b</u>	<u>x</u>	<u>#</u>
q_0	(q_1, x, R)	(q_2, x, R)	(q_0, x, R)	$(q_f, \#, L)$
q_1	(q_1, q, R)	(q_3, x, L)	(q_1, x, R)	—
q_2	(q_3, x, L)	(q_2, b, R)	(q_2, x, R)	—
q_3	(q_3, q, L)	(q_3, b, L)	(q_3, x, L)	$(q_0, \#, R)$
q_f	—	—	—	—

Verification: For $aabbba$:

[illegible]

4. Design a TM for $L = \{ w \in \{0,1\}^* \mid w \in \{0,1\}^* \}$

Solⁿ: Clearly the string generated by the language L is like:

$ab^2cab, abb^2cabb, abab^2cabb, \dots$

Let us assume that the input string has block symbol # at both the extreme ends.

#	a	b	a	a	b	c	a	b	a	a	b	#
---	---	---	---	---	---	---	---	---	---	---	---	---

We design the TM as in below fashion: -

- TM scans a in the first half and changes it into x, and finds the corresponding a in the second half and changes it to x.
- TM scans b in first half and changes to y, and correspondingly changes b in second half to y.
- Thus all a are changed into x and b into y.

The ~~state diagram~~ TM is :

$$M = \{Q, \epsilon, \tau, \delta, q, F, B\}$$

where, $\Theta = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}\}$

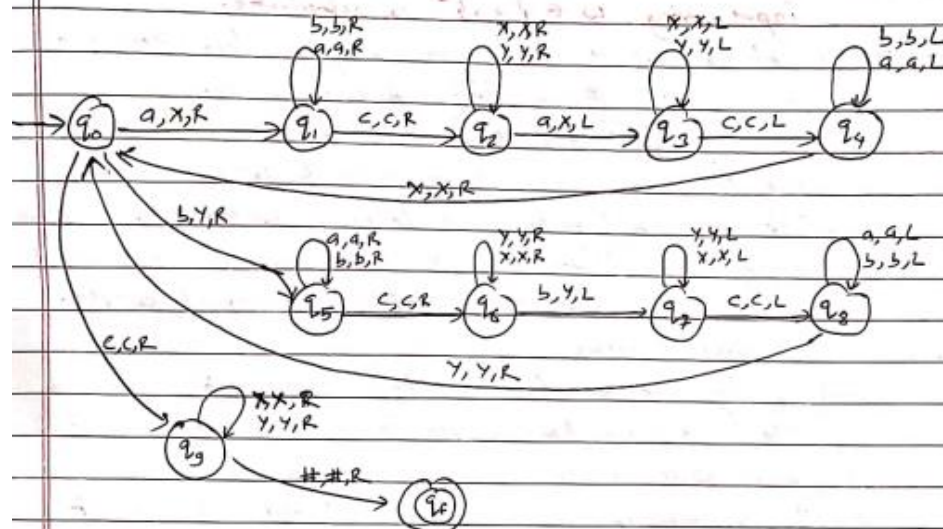
$$E = \{9, 3\}$$
$$Z = \{x \notin A\}$$
$$q_0 = \{q_0\}$$
$$F = \{v_f\}$$
$$B = \{ \# \}$$

δ is given by:

	a	b	c	#
q_0	-	-	-	-
q_1	-	-	-	-
...	-	-	-	-
...	-	-	-	-
q_f	-	-	-	-

GURUKUL

The state diagram is:-



Verification: for string abb cabb

9. ~~a~~ # a b b c a b b #

1-9, # X b b c a b b #

1. $\vdash q, \# \times b b c a b b \#$

$$\vdash q_1 \# \times b b \subseteq a b b \#$$

1- 22 # x b b c a b b #

$$\vdash q_2 \# xbb \subseteq xbb \#$$

- - -

1

$$\vdash^* q.c \nVdash XY Y \subset XY Y \nVdash$$

~~Hence accepted~~

5. Design a TM to ~~decide~~ decide whether or not any input string $w \in \{a,b\}^*$ is palindrome.

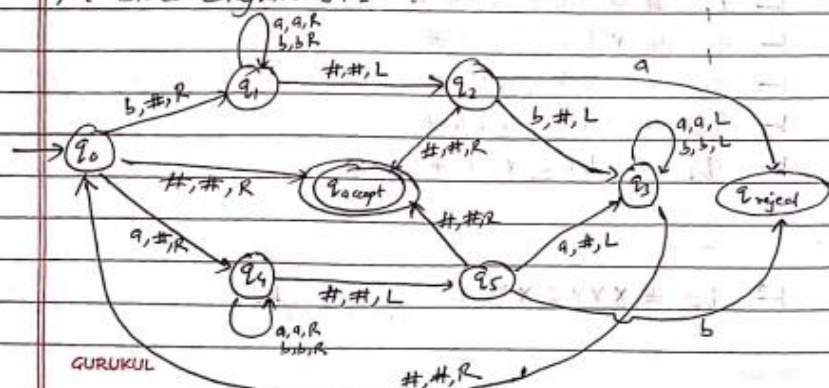
Sol: Let us assume that input string is terminated by a blank symbol #, at ~~end~~ ^{right} end of the string.

|a|b|b|a|#|

The TM can be constructed by the following moves:-

- The tape head moves reads leftmost symbol of w , replaces it by #. Then the tape head moves to the right most symbol and test whether it is equal to the leftmost symbol.
- If they are equal, then Right most symbol is deleted, then the tape head moves to the new left-most symbol and above process is repeated.
- If corresponding left most and right-most cells are not equal, TM enters the reject state and stops.
- TM enters accept state as soon as string currently stored in tape is empty.

The state diagram is:-



Here TM is :-

$M = \{Q, \Sigma, \Gamma, \delta, q_0, F, B\}$

Where,

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_{accept}, q_{reject}\}$

$\Sigma = \{a, b\}$

$\Gamma = \{a, b, \#\}$

δ is given by:

$q_0 = \{q_0\}$

		a	b	#
$F = \{q_{accept}\}$	q_0	$(q_1, \#, R)$	$(q_1, \#, R)$	q_{accept}
$B = \{\#\}$	q_1	(q_2, a, R)	(q_2, b, R)	$(q_2, \#, L)$
	q_2	-	$(q_3, \#, L)$	q_{accept}
	q_3	(q_4, a, L)	(q_4, b, L)	$(q_0, \#, R)$
	q_4	(q_5, a, R)	(q_5, b, R)	$(q_5, \#, L)$
	q_5	$(q_0, \#, L)$	-	q_{accept}
	q_{accept}	-	-	-

Verification: test for abba

q_0 : <u>a</u> bba#	$\vdash q_1 \# \# b \# \#$	Also check for ab#
$\vdash q_1 \# b b a \#$	$\vdash q_2 \# \# b \# \#$	ab#
$\vdash q_2 \# b b a \#$	$\vdash q_3 \# \# \# \# \#$	q_0 a b #
$\vdash q_3 \# b b a \#$	$\vdash q_0 \# \# \# \# \#$	$q_1 \# b \#$
$\vdash q_4 \# b b a \#$	$\vdash q_{accept} \# \# \# \# \#$	$q_4 \# b \#$
$\vdash q_5 \# b b a \#$		$q_5 \# b \#$
$\vdash q_3 \# b b \# \#$		$q_{rej} \# b \#$
$\vdash q_3 \# b b \# \#$		
$\vdash q_0 \# b b \# \#$		
$\vdash q_1 \# \# b \# \#$		

Since, the TM replaces all symbol by # and enters q_{accept} state. The input string is palindrome.

Since TM enters reject state, the input string is not palindrome.

5. Design a TM that works as a simple eraser, which changes every non-blank symbol to blank with the alphabets $\Sigma = \{0, 1, \#\}$. Hence test your design for $\#0101\#$ to $\#\#\#\#\#$

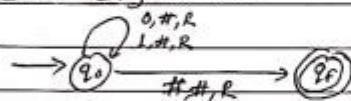
Sol: Let us assume that input string is terminated by a blank symbol $\#$ at each end of the string.

$\#0101\#$

The TM can be constructed as:

- The TM reads 0 and replaces 0 or 1 whatsoever with a blank $\#$ and moves right.
- If the TM reaches the extreme right it stops at final state.

The state diagram is:-



Hence, the TM is as:

$$M = \{Q, \Sigma, \delta, q_0, F, \tau, B\}$$

where,

$$Q = \{q_0, q_f\} \quad \delta \text{ is given as:-}$$

$$\Sigma = \{0, 1\}$$

$$\tau = \{\#\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_f\}$$

$$B = \{\#\}$$

	0	1	#
q_0	$(q_0, \#, R)$	$(q_0, \#, R)$	$(q_f, \#, R)$
q_f	-	-	-

Verification: for $\#0101\#$

$q_0 \#0101\#$

$\vdash q_0 \#\#101\#$

$\vdash q_0 \#\#\#01\#$

$\vdash q_0 \#\#\#\#1\#$

$\vdash q_0 \#\#\#\#\#$

$\vdash q_f \#\#\#\#\#$

Verified

Tutorial:

Tutorial:

1. Design a TM that recognizes the language of all strings of even length over alphabet $\{a, b\}$.
2. Design a TM that accepts the language of all strings which contain aba as a substring.
3. Design a TM that recognizes the set of all strings of 0's and 1's containing at least one 1.
4. Design a TM that replace every 0 and 1 with every 1 and 0 in a binary string.
5. Design a TM which works as eraser.
6. Design a TM for the RE $x = aa^*$
7. Design a TM for the language $L = \{(ab)^n \mid n \geq 0\}$
8. Design a TM which accepts the language $L = \{w \in \{0,1\}^+ \mid w \text{ has equal number of 1's and 0's}\}$
9. Design a TM that accepts the language $L = \{1^n 2^m 1^n \mid n \geq 0\}$ [2015 Fall]
10. Design a TM that accepts the language $L = \{w \in \{a,b\}^+ \mid w \text{ has equal number of a's and b's}\}$
11. Design a TM that accepts $L = \{p^n q^m r^n \mid m, n \geq 0\}$ [2014 Fall]

5.9 TM for computing functions

- A TM can be used to compute function.
- The input string W is presented in the form of $\#W\#$.
- The head of TM is placed positioned at the blank symbol which immediately follows the string W .
- We use an underscore to show the current position of machine head in the tape.
- A TM is said to halt on input W if we can reach to a halting state 'h' after performing some operations.
- A TM, say $T_M = (Q, \Sigma, \Gamma, \delta, q_0, F, B)$, is said to halt to an input W if and only if $(q_0, \#W\#)$ yields to $(h, \#W\#)$.

5.10 Definition of TM for Computing function

A function $f(x) = y$ is said to be computable by a TM M if

$$(q_0, \#W\#) \xrightarrow{M} (h, \#y\#).$$

5.11 Numerical: TM for Computing Function

- Design a TM which computes the function $f(n) = n+1$ for each $n \in \mathbb{N}$. [2013 Fall, 2018 Spring]

Solⁿ: Given function is $f(n) = n+1$

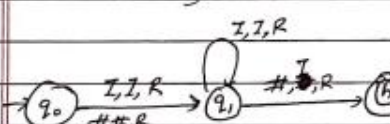
Let us represent input n on the tape by a number of I on the tape.

If $n=1$, input will be $\#I\#$, and output will be $\#II\#$

If $n=2$, input will be $\#II\#$, and " " " $\#III\#$

If $n=3$, input will be $\#III\#$, " " " $\#IIII\#$

The state diagram is:-



Verification:

$q_0 \#III\#$

$\vdash q_1 \#III\#$

$\vdash q_1 \#IIII\#$

$\vdash q_1 \#IIII\#$

$\vdash q_1 \#IIII\#$

$\vdash h \#IIII\#$

hence verified

Hence TM is:-

$M = \{Q, \Sigma, \Gamma, \delta, q_0, F, B\}$

where,

$Q = \{q_0, q_1, h\}$

$\Sigma = \{I, \#\}$

$\Gamma = \{I, \#\}$

$q_0 = \{q_0\}$

$F = \{h\}$

δ is given by:-

δ	I	#
q_0	(q_1, I, R)	$(q_1, \#, R)$
q_1	(q_1, I, R)	(h, I, R)
h		

5. TURING MACHINE

2. Prove that following function is computable: $f(n) = n+2$

Solⁿ: If any function is computable, then there exist a TM for it. So, it will be sufficient to construct a TM to prove any function computable.

Given function is $f(n) = n+2$

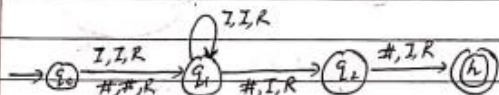
Let us represent n on the tape by number of I on the tape.

If $n=1$, input will be $\#I\#$, then output will be $\#III\#$

If $n=2$ " " " $\#II\#$, " " " $\#IIII\#$

If $n=3$ " " " $\#III\#$, " " " $\#IIII\#$

The state diagram would be:-



Here, TM is: $M = \{Q, \Sigma, \Gamma, \delta, q_0, F, B\}$

Where;

$Q = \{q_0, q_1, q_2, h\}$

$\Sigma = \{I, \#\}$

$\Gamma = \{I, \#\}$

$q_0 = \{q_0\}$

$F = \{h\}$

$B = \{\#\}$

Verification:

$q_0 \#III\#\#\#$

$\vdash q_1 \#IIII\#\#\#$

$\vdash q_2 \#IIII\#\#\#$

$\vdash q_1 \#IIII\#\#\#$

$\vdash q_1 \#IIII\#\#\#$

$\vdash q_2 \#IIII\#\#\#$

$\vdash h \#IIII\#\#\#$

δ is given by:-

δ	I	#
q_0	(q_1, I, R)	$(q_1, \#, R)$
q_1	(q_2, I, R)	$(q_2, \#, R)$
q_2	-	(h, I, R)

Hence, given function is

computable.

GURJUKUL

3. Design a TM for computing a function $f(w) = w\#w$

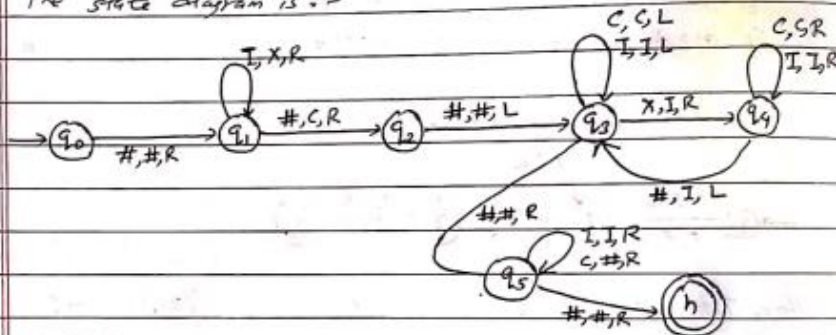
Solⁿ: Here,

We have to design a TM that takes w and gives output $w\#w$.

Let us assume, w as $\#III\#$,

then, the output should be $\#III\#III\#$

The state diagram is:-



Hence, the TM is:-

$M = \{Q, \Sigma, \Gamma, \delta, q_0, F, B\}$

Where;

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, h\}$

$\Sigma = \{I, \#\}$

$\Gamma = \{I, \#, \#\}$

$B = \{\#\}$

$q_0 = \{q_0\}$

$F = \{h\}$

Tutorial

δ is given by:-

	I	#	X	C
q_0	-	$(q_1, \#, R)$	-	-
q_1	(q_2, X, R)	(q_2, C, R)	-	-
q_2	-	$(q_3, \#, L)$	-	-
q_3	(q_3, I, L)	$(q_5, \#, R)$	(q_4, I, R)	(q_2, C, L)
q_4	(q_4, I, R)	(q_3, I, L)	-	(q_4, C, R)
q_5	(q_5, I, R)	$(h, \#, R)$	-	$(q_5, \#, R)$
h	-	-	-	-

Verification: for #II#

① $\vdash q_0 \# II \# \# \# \#$	⑬ $\vdash q_3 \# I I C I I \#$
② $\vdash q_1 \# I I \# \# \# \#$	⑭ $\vdash q_3 \# I I C I I \#$
③ $\vdash q_1 \# X I \# \# \# \#$	⑮ $\vdash q_3 \# I I C I I \#$
④ $\vdash q_1 \# X X \# \# \# \#$	⑯ $\vdash q_3 \# I I C I I \#$
⑤ $\vdash q_3 \# X X C \# \# \# \#$	⑰ $\vdash q_5 \# I I C I I \#$
⑥ $\vdash q_4 \# X I C \# \# \# \#$	⑱ $\vdash q_5 \# I I C I I \#$
⑦ $\vdash q_4 \# X I C \# \# \# \#$	⑲ $\vdash q_5 \# I I C I I \#$
⑧ $\vdash q_3 \# X I C I \# \# \# \#$	⑳ $\vdash q_5 \# I I \# I I \#$
⑨ $\vdash q_3 \# X I C I \# \# \# \#$	㉑ $\vdash q_5 \# I I \# I I \#$
⑩ $\vdash q_3 \# X I C I \# \# \# \#$	㉒ $\vdash q_5 \# I I \# I I \#$
⑪ $\vdash q_4 \# I I C I \# \# \# \#$	㉓ $\vdash h \# I I \# I I \#$
⑫ $\vdash q_4 \# I I C I \# \# \# \#$	
⑬ $\vdash q_4 \# I I C I \# \# \# \#$	
⑭ $\vdash q_4 \# I I C I \# \# \# \#$	
⑮ $\vdash q_3 \# I I C I \# \# \# \#$	

Note:

Take #I#
for verification
to
minimize
time consumption

hence
accepted

1. Design a TM that accepts every 0 and 1 with every 1 and 0 in a binary string.

2. Prove that the function $f(w) = \bar{w}$ is computable where $w \in \{0,1\}^*$ and $\bar{w}$ the one's complement of w .

3. Construct a TM that computes the following function
 $f(n, m) = n + m$

4. Design a TM which works as eraser.

5. Prove that following function is Turing computable:

$$f(m) = \begin{cases} m-2 & \text{if } m > 2 \\ 1 & \text{if } m \leq 2 \end{cases}$$

$$f(n) = \begin{cases} n-1 & , n > 0 \\ 0 & , n = 0 \end{cases}$$

6. Design a TM for the following function: $f(x, y) = y$

7. Design a TM for the following function $f(x, y) = x$

8. Design a TM for the regular expression $x = a^*a^*$

9. Design a TM for the language $L = \{ (ab)^n \mid n \geq 0 \}$

10. Design a TM which accepts the language $L = \{ w \in (a,b)^* \mid w \text{ has equal no of } a's \text{ and } b's \}$.

11. Design a TM which accepts the language $L = \{ w \in (a,b,c)^+ \mid w \text{ has equal no of } a's, b's \text{ and } c's \}$.

12. Design a TM for the following language $L = \{ a^n \mid n > 0 \}$

13. Design a TM which computes following function.

$$f(w) = ww^R, \text{ where } w^R \text{ is the reverse of string } w, (w \in (a,b)^*)$$

14. Design a TM which works as copying machine(C), for $w \in (a,b)^+$.

5.12 TM as Transducer

It consists of :-

- set of states representing states of TM.
- an arc from any state q to p is labelled by the items of the form X/YD where X and Y are tape symbols where X and Y are tape symbols
- D is a direction either Left or Right
- ie. $\delta(q, x) = (p, y, d)$

Ex: Let $TM = \{Q, \Sigma, \Gamma, \delta, q_0, F, B\}$ where,

$$Q = \{q_0, q_1, q_2, h\}$$

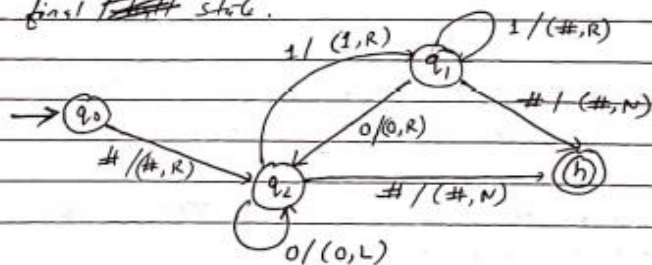
$$\Sigma = \{0, 1\}$$

$$\Gamma = \{0, 1, \#\}$$
 and

δ is given by the following transition table:-

	0	1	#
q_0	-	-	$(q_1, \#, R)$
q_1	$(q_2, 0, R)$	$(q_1, \#, R)$	$(h, \#, N)$
q_2	$(q_2, 0, L)$	$(q_1, \#, R)$	$(h, \#, N)$
h	-	-	-

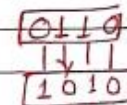
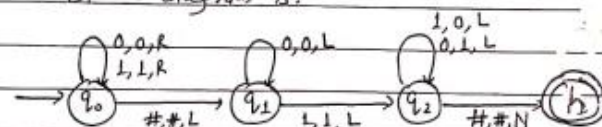
The transition diagram for above TM will be as shown below, where we assume that q_0 is the initial state and h is a final state.



5.13 Numerical: TM as Transducer

Design a TM that converts a binary string of 0 and 1 to its equivalent 2's complement.

The state diagram is:-



How TM is as:-

$$M = \{Q, \Sigma, \Gamma, \delta, q_0, F, B\}$$

Verification for: 0110

where,

$$Q = \{q_0, q_1, q_2, h\}$$

$$q_0 \# 0110 \#$$

$$\Sigma = \{0, 1\}$$

$$\vdash q_0 \# 0110 \#$$

$$\Gamma = \{0, 1, \#\}$$

$$\vdash q_0 \# 0110 \#$$

$$q_0 = \{q_0\}$$

$$\vdash q_0 \# 0110 \#$$

$$F = \{h\}$$

$$\vdash q_0 \# 0110 \#$$

$$B = \{\#\}$$

$$\vdash q_1 \# 0110 \#$$

δ is given as:-

$$\vdash q_1 \# 0110 \#$$

	0	1	#
q_0	$(q_0, 0, R)$	$(q_0, 1, R)$	$(q_1, \#, L)$
q_1	$(q_1, 0, L)$	$(q_2, 1, L)$	-
q_2	$(q_2, 1, L)$	$(q_2, 0, L)$	$(h, \#, N)$
h	-	-	-

$$\vdash q_2 \# 0110 \#$$

$$\vdash q_2 \# 0010 \#$$

$$\vdash q_2 \# 1010 \#$$

$$\vdash h \# 1010 \#$$

Hence finished and accepted

5.14 Extensions of TM

- TMs can perform fairly powerful computations.
- There are a lot of extensions we can make to our basic TM model. They may make it easier to write TM programs, but none of them increase the power of TM because we can show that every extended TM has an equivalent basic machine.

- The extensions of TM are:-

i) Multiple tapes TM:

There may be several tapes instead of only one, each tape has its own independent head.

ii) Two Way infinite tape.

The tape may be allowed to be infinite in both the directions.

iii) Multiple head TM:

There may be more than one head scanning various cells of the tape.

Ex. $\# \# b c \# d e f \# \# \dots$
 $\uparrow \quad \uparrow$
 $H_1 \quad H_2$

δ (state, symbol under H_1 , symbol under H_2)

= (new state, (S_1, M_1) , (S_2, M_2))

iv) K-dimensional Turing Machine.

The tape may be K-dimensional, $K \geq 2$, instead of only one dimensional.

v) Non-deterministic TM

For a given pair of current state and symbol under the head, instead of at most one possible move, there may be any finite number of next moves.

vi) Random Access TM:

- A Random Access TM has fixed no. of registers and a one-way infinite tape.

- Tape acts as Random Access memory chip.

- Random Access TM has a finite length program, composed of instructions with operations such as Read, Write, Load, Store, Add, Sub, Jump.

- The machine acts on its tape squares and its registers as data dictated by a fixed program.

5.14.1 Multi-tape Turing Machine

- It consists of several tapes.

- Each tape is connected to the finite control by means of a R/W head (one on each tape).

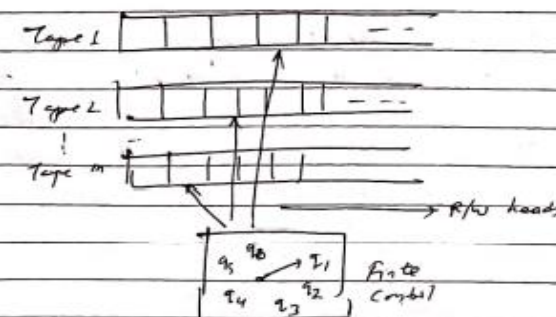


Fig: Multi-tape TM

- The machine can, in one step, read the symbols scanned by all its heads.
- Depending on those symbols in current state, it can rewrite some of those scanned squares and move some of the heads to the left or right, in addition to change, and also change the state.

Formal definition of m -tape TM:

Let $m \geq 1$ be an integer.

An m -tape TM is a six tuple machine as follows:-

$$T_m = (Q, \Sigma, \Gamma, \delta, q_0, h)$$

where,

Q : finite set of states of the finite control

Σ : finite set of input symbols

Γ : tape symbol where $\Gamma = \Sigma \cup \#$

h : halt state $h \in Q$

q_0 : initial state

δ : transition function that maps:

$$(Q \times \Sigma^m) \rightarrow Q \times (\Sigma \cup \{L, R, N\}^m).$$

5.15 Non-Deterministic TM

- It is powerful than multitape and Random Access TM due to its non-deterministic feature.
- It may contain certain combination of states and scanned symbol, and more than one possible choice of behavior.
- Non-deterministic machine can produce two different outputs or final states from the same input.
- The figure below shows a NDTM.
- It is one which at any point in a computation may proceed according to several possibilities.
- NDTM can choose. The transition function δ are subsets rather than single element of set $Q \times \Gamma \times \{R, L, S\}$.
- Here, transition function δ is such that for each state q and tape symbol x , $\delta(q, x)$ is set of triplets $\{(q_1, x_1, D_1), (q_2, x_2, D_2), \dots, (q_k, x_k, D_k)\}$ where k is finite integer.
- The NDTM can choose, at each step, any of the triplets to be the next move.
- But it can't however pick a state from one, a tape symbol from another, and direction from yet another.
- The computation of a NDTM is a tree whose branches correspond to different possibilities for the machine.
- If some branch of the computation leads to the accept state, the machine accepts its input.
- If M_N is a NDTM, then there is a deterministic TM M_D such that $L(M_N) = L(M_D)$.

5.14.2 Two-way Infinite Tape

Refer book: Page 248

5.14.3 Multi-Head TM

Refer book: Page 249

5.14.4 K-dimensional TM

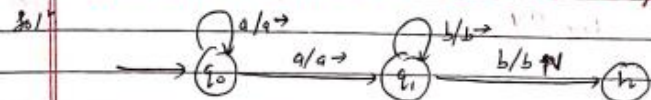
Refer book: Page 250

5.14.5 Random Access TM.

Refer: cs.utexas.edu/courses/csl/clino/lect/automata/CS341-Fall-2004-Packet/1-Lochman/23-24-Turing Machines Handout.pdf

Automata Theory and
Formal Language
- Adesh K. Pandey

Construct a NDTM which accepts the language $\{a^n b^m : n \geq 1, m \geq 1\}$ that is language of all strings over $\{a, b\}$ in which there is at least one 'a' and one 'b' and all 'a's precede all 'b's.



Here $TM = \{Q, \Sigma, \Gamma, \delta, q_0, h\}$

where $Q = \{q_0, q_1, h\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a, b, \# \}$

$q_0 = \{q_0\}$, $h = \{h\}$

δ is defined as:-

$\delta(q_0, a) = \{(q_0, a, R), (q_1, a, R)\}$

$\delta(q_1, b) = \{(q_1, b, R), (h, b, N)\}$

or:

	a	b
q_0	$\{(q_0, a, R), (q_1, a, R)\}$	—
q_1	—	$\{(q_1, b, R), (h, b, N)\}$
h	—	—

Hence, $ab, aabb$ are accepted by the NDTM.

End of Chapter 5

6.1 Introduction to undecidability

In the theory of computation, we often come across such problems that are answered either 'yes' or 'no'. The class of problems which can be answered as 'yes' are called solvable or decidable. Otherwise, the class of problems is said to be unsolvable or undecidable.

6.2 Decidable and Undecidable problems

A problem is decidable if we can construct a TM which will halt in finite amount of time for every input and give answer as 'Yes' or 'No'. A decidable problem has an algorithm to determine the answer for a given input.

Some of the decidable problems are mentioned below. There is an algorithm and TM to decide for the below problems:

1. Equivalence of two regular language
2. Finiteness of regular language
3. Emptiness of CFL

A problem is undecidable if there is no TM which will always halt in finite amount of time to give answer 'yes' or 'No'. An undecidable problem has no algorithm to determine the answer for a given input.

Some of the undecidable problems are mentioned below. There is no TM which halts in finite amount of time and decide for below problems:-

1. Ambiguity of CFL
2. Equivalence of two CFL

Two popular undecidable problems are: The halting problem and the PCP (Post Correspondence Problems).

6.3 The Halting problem

- Halting means that the program on certain input will accept it and halt, or reject it and halt, and would never go into an infinite loop.
- Basically, halting means terminating.

- Halting problem is undecidability.
- It asks the question - "Is it possible to tell whether a given machine will halt for some given input?"
- The answer is No. We cannot design a generalized algorithm which can appropriately say that given a program, the machine will ever halt or not.

- Example:

Input: A Turing Machine (TM) and input string w
Problem: Does the TM finish computing of the string w in a finite no. of steps?

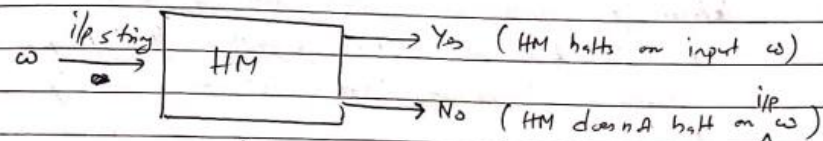
Proof:

Assume TM exists to solve this problem and then we will show it is contradicting itself.

Let us call this TM as a Halting machine that produce a 'Yes' or 'No' in a finite amount of time.

If the Halting machine (Hm) finishes in a finite amount of time, the opp comes as 'Yes' otherwise as 'No'.

The block diagram of HM is:-



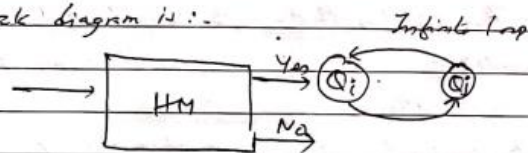
This allows us to design another machine.

Now let us design an inverted halting machine as:-

If it returns 'Yes', then loop forever

If it returns 'No', then halt.

The block diagram is:-



Here, If HM halts on given input, then it loops forever.

But if HM does not halt, It returns 'No' and hence halts.

This can be depicted by an algorithm.

$C(x)$ if $(H(x, w) == \text{Halt})$ Loop forever; else Return;	If we run C on itself: $C(C)$ $H(C, C) == \text{Halt}$ $\downarrow$ Not Halt $H(C, C) == \text{Not Halt}$ $\downarrow$ Halt
---	--

Hence, By contradiction, the Halting problem is undecidable.

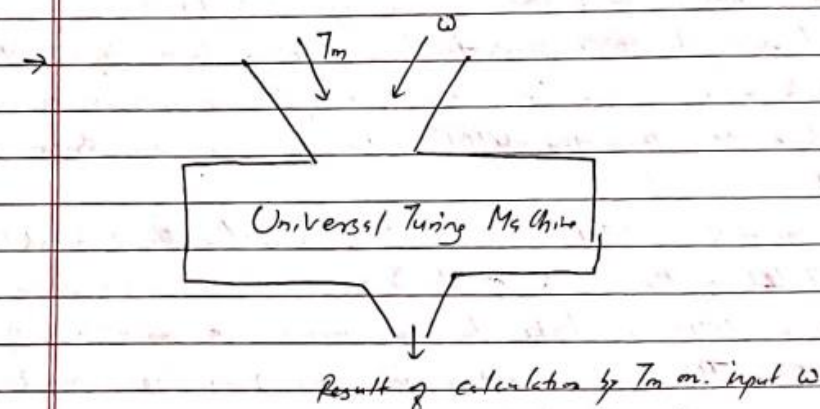
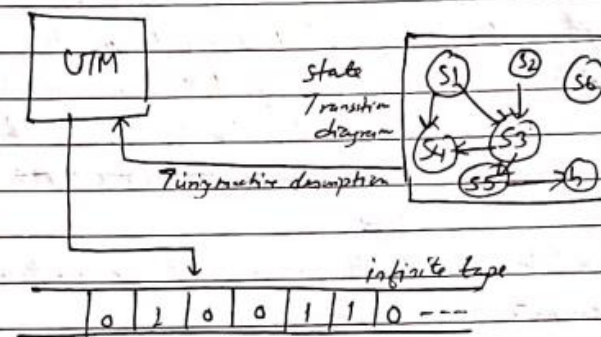
6.4 Church's Thesis

- It is believed that there are no functions that can be defined by humans, whose calculation can be described by a well-defined mathematical algorithm that people can be taught to perform, that cannot be computed by TM.
- Church's Thesis consider the TM to be the ultimate calculating mechanism.
- This Church Thesis was submitted by ~~Alonzo~~ Alonzo Church (1936).
- The Church Thesis states that - "No computational procedure will be considered an algorithm unless it can be represented by a TM".
- Church actually said that any machine that can do certain list of operations, will be able to perform all conceivable algorithms.
- He tied together the idea of recursive functions and computable functions.
- However, Church Thesis cannot be a theorem because ideas such as "can ever be defined by humans" and "algorithm that people can be taught to perform" are not part of any known mathematics.

6.5 Universal TM

- A Universal TM (UTM) is a TM that can simulate the behavior of an arbitrary TM over any set of input symbols.
- The UTM achieves this by reading both the description of machine to be simulated as well as the ip from its own tape.
- Thus, it can be possible to create a single machine that can be used to compute any computable sequence.
- Alan Turing introduced this machine in 1936-1937.
- If this machine is supposed to be supplied with the tape on the begining of which is written the input string of quintuple of some computing machine M , then the UTM will compute the same strings as those by M .
- The model of a UTM is considered to be a theoretical breakthrough that led to the concept of stored programmed computing device.
- Thus a UTM 'U' takes two arguments, a description of a machine T_m , " T_m " and a description of an input string ω , " ω ". Then the UTM has following property: -
 U halts on input " T_m " " ω " iff T_m halts on input ω .
 ie $U("m" \omega) = "m(\omega)"$
- It is the functional notation of UTM.

- The UTM simulation on other machine is shown below:-



- We can modify our basic TM and design a UTM by:-
 - increasing the no. of R/W heads
 - Increasing the no. of dimensions of input tape
 - Adding a special purpose memory.

6.6 Undecidable problems about TM

- The problems for which no algorithms exist are called Undecidable or Unsolvable. One famous undecidable problem is Halting Problem.
- The following problems about TM are undecidable:
 - a) Given a TM, M and an input string w , does M halt on input w ?
 - b) Given a TM, M , does M halt on the empty tape?
 - c) Given a TM, M , is there any string at all on which M halts?
 - d) Given a TM M , does M halts on every input string?
 - e) Given two TMs, M_1 and M_2 , do they halt on the same input language & strings?
 - f) Given a TM M , is the language that M semidecides regular? Is it context free? Is it recursive?

6.7 Encoding of TM

- It is a process of formulating a notation system where we can encode both an arbitrary TM, T_L and an input string x over an arbitrary alphabet as strings $e(T_L)$ and $e(x)$ over some fixed alphabet.
- This encoding must not destroy any ^{original} information. i.e. we must be able to reconstruct TM T_L and string x .
- For encoding of TM, we use alphabets $\{0,1\}$, ^{although} however, TM may have much larger alphabet.

Steps for Encoding:

1. Start by assigning positive integer to each state, each tape symbol and each of three directions in TM that we want to encode.
2. Represent a state or a symbol by a string of 0's of appropriate length. Here 1's are used as separator.
3. For transition rule, use encoding function S .
 e.g.: $\delta(q_i, q_j) = (q_k, a_x, D_m)$
 encoded as:
 $S(q_i) \mid S(q_j) \mid S(q_k) \mid S(a_x) \mid S(D_m)$ - say m_1
4. Separate the entire transition rules by pair of 1's.
 i.e. $m_1 \mid\mid m_2 \mid\mid m_3 \mid\mid \dots \mid m_n$
5. Now code for TM and i/p string x will be formed by separating them by three consecutive 1's.
 i.e. $e(TM) \mid\mid\mid e(x)$

Example:

Consider a TM, $T = (Q, \Sigma, \Gamma, \delta, q_1, F, B)$ where $Q = \{q_1, q_2, q_3\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a, b, B\}$, $q_1 = \{q_1\}$, $F = \{q_3\}$, $B = \{B\}$, and δ is:-

$$\begin{aligned}\delta(q_1, b) &= (q_2, a, R) \\ \delta(q_2, a) &= (q_1, b, R) \\ \delta(q_2, b) &= (q_2, a, R) \\ \delta(q_3, B) &= (q_3, B, L)\end{aligned}$$

and input string $x = ab$

Solⁿ: Here,

Step 1:

Let Here, $\Sigma = \{a, b\}$

Let $q_1 = a$

$q_2 = b$

Step 2:

Let us represent states and symbols by a string of 0's of appropriate length.

$S(B) = 0$

$S(q_i) = 0^{i+1}$ for each $q_i \in Q$

$S(q_i) = 0^{i+2}$ for each $q_i \in Q$

$S(N) = 0$

$S(L) = 00$

$S(R) = 000$

Using encoding function s as defined above:

$S(q_1) = 000$ $S(q_1) = S(a) = 00$ $S(N) = 0$

$S(q_2) = 0000$ $S(q_2) = S(b) = 000$ $S(L) = 00$

$S(q_3) = 00000$ $S(B) = 0$ $S(R) = 000$

Step 3: Using encoding function for transition rules

$e(m_1) = S(q_1) \mid S(b) \mid S(q_2) \mid S(a) \mid S(R)$

$= 000 \mid 1 \mid 000 \mid 1 \mid 00000 \mid 00 \mid 1 \mid 000$

$e(m_2) = S(q_2) \mid S(a) \mid S(q_1) \mid S(b) \mid S(R)$

$= 00000 \mid 1 \mid 00 \mid 1 \mid 000 \mid 1 \mid 000 \mid 1 \mid 000$

$e(m_3) = S(q_2) \mid S(b) \mid S(q_2) \mid S(a) \mid S(R)$

$= 00000 \mid 1 \mid 000 \mid 1 \mid 0000 \mid 1 \mid 00 \mid 1 \mid 000$

$e(m_4) = S(q_3) \mid S(B) \mid S(q_3) \mid S(b) \mid S(L)$

$= 00000 \mid 1 \mid 0 \mid 1 \mid 00000 \mid 1 \mid 000 \mid 1 \mid 00$

Step 4: Code for TM, T is:

$e(T) = e(m_1) \mid \mid e(m_2) \mid \mid e(m_3) \mid \mid e(m_4)$

$= 000 \mid 1 \mid 000 \mid 1 \mid 00000 \mid 1 \mid 00 \mid 1 \mid 000 \mid \mid$

$00000 \mid 1 \mid 00 \mid 1 \mid 000 \mid 1 \mid 000 \mid 1 \mid 000 \mid \mid$

$00000 \mid 1 \mid 000 \mid 1 \mid 000 \mid 1 \mid 00 \mid 1 \mid 000 \mid \mid$

$00000 \mid 1 \mid 0 \mid 1 \mid 00000 \mid 1 \mid 000 \mid 1 \mid 00 \mid \mid$

Step 5: Now for T and any input string x where $x = ab$,

code will be: $e(T) \mid \mid e(x)$

Here, $e(x) = S(a) \mid S(b)$

$= 00 \mid 1 \mid 000$

Hence,

$e(T) \mid \mid e(x) = 000 \mid 1 \mid 000 \mid 1 \mid 00000 \mid 1 \mid 00 \mid 1 \mid 000 \mid \mid 00 \mid 1 \mid 000$

6.8 Recursive and Recursively Enumerable Language

When a TM executes an input, there are four possible outcomes of execution. Then TM

- Halts and accept the input
- Halts and rejects the input
- Never halts (fall into loop), or
- Crash.

Recursive Enumerable Language (RE):

- also called type-0 language, or TM recognizable language
- RE languages are those languages that can be accepted by TM
- Strings: Strings that are not in the language may be rejected or may cause the TM to go into an infinite loop.
- A language is Recursively Enumerable if there exists a TM that accepts every string of the language, and does not accept strings that are not in the language.
- RE languages are the superset of Recursive Language.
- Every Recursive Language is RE language but not vice-versa.

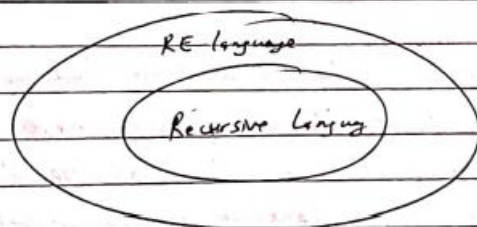


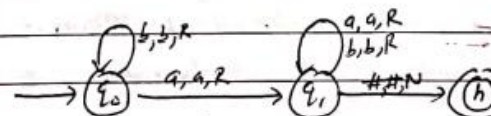
Fig: Relationship between RE and REC language

Recursive Language (REC Language):

- A language is recursive if there exists a TM that accepts every strings of the language and rejects every strings that are not in the language.
- A REC language is the subset of RE language.
- A REC language is decidable by TM, which means it will enter into final state for acceptable strings and into rejecting state for non-acceptable strings.
- So, the TM will always halt in this case.
- For eg: $L = \{a^n b^n c^n \mid n \geq 1\}$ is recursive language because we can construct a TM which will move to final state if the string is of the form $a^n b^n c^n$ else move to non-final state.

Example:

Let $L = \{w \in \{a,b\}^* : w \text{ contains at least one 'a'}\}$.
Then we can design a TM for L as:-



This machine scans to the right to find one 'a'.

If no 'a' is found, it goes forever, never halting.

It halts only if there is at least one 'a'.

Therefore, given language is Recursively Enumerable.

6.9 Turing Recognizable language

- can run forever without deciding

A language L is Turing recognizable if there exists a TM M such that for all strings w :

- If $w \in L$, eventually M enters q_{accept}
- If $w \notin L$, either M enters q_{reject} or M never terminates

6.10 Turing decidable language

- Always terminates.

A language L is Turing decidable if there exists a TM M such that for all strings w :

- If $w \in L$, M enters q_{accept}
- If $w \notin L$, M enters q_{reject}

6.11 Properties of Recursive and Recursively Enumerable Language

1. The complement of a Recursive language is recursive
2. The union of two recursive language is recursive
3. The union of two RE language is recursively enumerable
4. If a language L and its complement $\bar{L}$ are both recursively enumerable, then L and $\bar{L}$ here $\bar{L}$ is recursive.
5. If L is recursive language then $\Sigma^* - L$ is recursive.

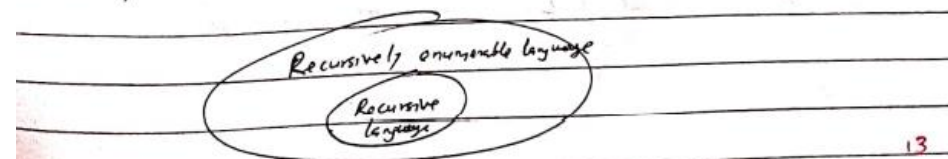
6.12 Theorem Proof

1. If a language is recursive then it is recursively enumerable.

A language L is recursive if a TM accepts every strings of the language and rejects then enters into final state, and rejects every strings that are not in the language then enters into rejecting state.

A language L is recursively enumerable if a TM accepts every strings of the language then enters into final state, and rejects every string that are not in the language then may enter into rejecting state or may loop forever.

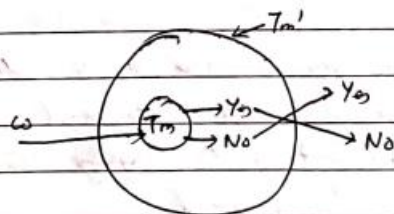
Hence, we can say that every recursive language is also a recursively enumerable. This is shown below with a relationship diagram.



2. The complement of a Recursive Language is recursive.

Proof: - Let $L \rightarrow$ Recursive language
 $T_m \rightarrow$ Turing machine that halts on all inputs and accepts L
 Let us construct a TM $T_{m'}$ from T_m so that if T_m enters a final state on input w , then $T_{m'}$ halts without accepting.
 If T_m halts without accepting, $T_{m'}$ enters a final state

6. UNDECIDABILITY



Since, one of these two events occurs, $T_{m'}$ is an algorithm. So clearly, $T(T_{m'})$ is the complement of L and thus the complement of L is recursive language.

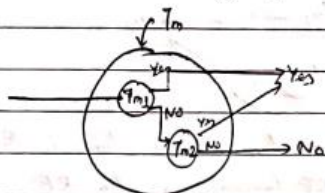
3. The union of two recursive language is recursive.

Proof: Let, L_1 and L_2 be two recursive language accepted by TM T_{m1} and T_{m2} .

Let us construct a TM T_m that first simulates T_{m1} .

If T_{m1} accepts, then T_m accepts

If T_{m1} rejects, then T_m simulates T_{m2} and accepts if and only if T_{m2} accepts.



Since both T_{m1} and T_{m2} are algorithm, So T_m is guaranteed to halt.

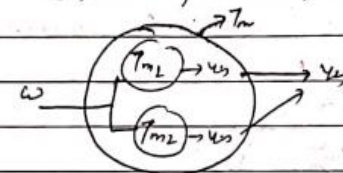
Clearly T_m accepts $L_1 \cup L_2$.

Hence The union of two recursive language is also recursive.

4. The union of two recursively enumerable language is recursively enumerable.

Proof: Let L_1 and L_2 are recursively enumerable language and their enumerative TM are T_{m1} and T_{m2} respectively.

Let us construct a TM T_m which can simulate T_{m1} and T_{m2} simultaneously on separate tape.



Here, If either T_{m1} or T_{m2} accepts, T_m also accepts.

Thus union of two RE language is also RE.

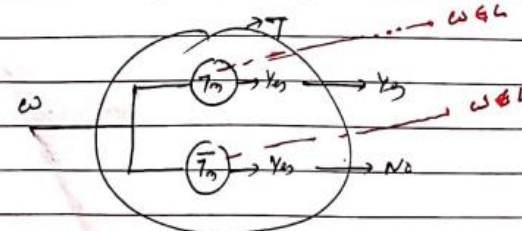
5. If a Language L and its complement L' are both recursively enumerable, then L (and hence L') is recursive.

Proof: Let, T_{m1} and $\bar{T}_{m1}$ accept L and $\bar{L}$ respectively.

Let us construct a TM T which simulate T_{m1} and $\bar{T}_{m1}$ simultaneously.

T accepts w if T_{m1} accepts w , and

T rejects w if $\bar{T}_{m1}$ accepts w

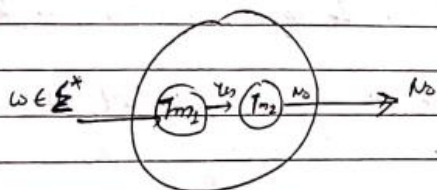


Thus T will always say either 'Yes' or 'No' but never says

both. Since T_1 is algorithm that accepts L , it follows that L is recursive.

6. If L is recursive then $\Sigma^* - L$ is recursive.

Proof: The required T_m -complement can be represented by as:-



The machine T_m -complement functions as follows:-

- When a string $w \in \Sigma^*$ is given as input to T_m -complement, its control passes the string to T_{m1} as input.
- As T_{m1} decides the language L , therefore, for $w \in L$ after a finite no. of moves, T_{m1} outputs "Yes" which is given as input to T_{m2} , which in turn returns "No".
- Similarly for $w \notin L$, T_{m1} returns "Yes".

Hence there exists a T_m -complement for $\Sigma^* - L$. So it is turing decidable, that is recursive.

Tutorial:

Prove:

1. Intersection of two recursive language is also recursive.
2. Intersection of two recursive enumerable language is also recursive enumerable.

6.13 Recursive Function Theory

1. Recursion

- A function which calls itself directly or indirectly and terminates after infinite no. of steps is known as Recursive function.
- In recursive function, terminating point is also known as base point.
- Each and every time, the function calls itself, it should be nearer to the base point.

2. Initial function for Natural numbers

Let $N = \{0, 1, 2, \dots\}$ be a set of Natural numbers, We have three initial functions over N defined below:-

1) Zero function

- denoted by Z and defined as:

$$Z(n) = 0 \text{ for } \forall n \in N \quad \text{e.g.: } Z(2) = 0$$

2) Successor function

- denoted by S and defined as:

$$S(n) = n + 1 \text{ for } \forall n \in N$$

$$\text{e.g.: } S(2) = 2 + 1 = 3$$

3) Projection function (P_i^n)

- defined as:-

$$P_i^n(a_1, a_2, \dots, a_n) = a_i \text{ where}$$

where, $a_i \in N$ for $i = 1, 2, \dots, n$ and $1 \leq i$

Projection function takes n arguments and returns their i^{th} argument.

3. Computation of function

We can define a new function by the combination of two or more functions. Such defined functions are called composition functions.

For eg: $S(Z(n)) = S(0) = 1$

$$S(S(Z(n))) = S(S(0)) = S(1) = 2$$

4. Primitive Recursive function

A function is primitive if:

i) It is an initial function

ii) It is obtained from recursion or composition of initial functions.

5. Partial Recursive function

A partial function is recursive if:

i) It is an initial function over N , or

ii) It is obtained by applying recursion or composition or minimization on initial function over N .

6. Total Recursive function

A ~~total~~ recursive function is said to be total recursive function if it is defined for all of its arguments.

Let $f(a_1, a_2, a_3, \dots, a_n)$ be a function and defined on function $g(b_1, b_2, \dots, b_m)$, then f is total function if every element of f is assigned to some unique element of function g .

End of Chapter 6

7.1 Computational complexity theory

- Computational Complexity theory is a branch of TCC in computer science & mathematics that focuses on classifying computational problems according to their inherent difficulty (or complexity).
- It includes the:
 - + the efficiency of algorithms
 - + the inherent difficulty of problems of practical and/or theoretical importance.
- It deals with the resources required during computation to solve a given problem.
- The most common resources are:
 - + time (how many steps it takes to solve a problem)
 - + space (how much memory it takes).
- Other resources may be; how many parallel processes are needed to solve a problem in parallel.
- Much of complexity theory deals with decision problems. A decision problem is a problem where the answer is always Yes/No.
- The goal of Complexity Theory is to classify decidable problems according to the amount of resources required to solve them. Space and Time are the two most common measures of complexity.

7.2 Computability theory

- Computability Theory deals only with whether a problem can be solved at all, regardless of the resources required.
- It deals primarily with the question of whether a problem is solvable at all on the computer.
- The statement that the halting problem cannot be solved by a TM is one of the most important results in computability theory.
- Computability theory addresses four main questions:
 1. What problems can TM solve?
 2. What other systems are equivalent to TMs?
 3. What problems require more powerful machines?
 4. What problems can be solved by less powerful machines?
- The computability theory does not ~~focus~~ deal with the time and space factor of a machine.

7.3 Space and Time Complexity

Space Complexity

- Space complexity is defined as the process of determining a formula for prediction of how much memory space will be required for the successful execution of the algorithm.
- The memory space we generally consider is the space of primary memory.
- Space complexity specifies the amount of temporary storage required for running the algorithm.

Time Complexity

- The time complexity is defined as the process of determining a formula for total time required towards execution of that algorithm.
- This calculation will be independent of implementation & details and programming language.

7.4 Polynomial-Time and Exponential Algorithm

The basic idea for problem classification lies on whether there exists a polynomial-time (computer) algorithm that can provide the solution to a problem. The algorithms can be classified into two categories:-

- a) Polynomial-time algorithm
- b) Exponential algorithm.

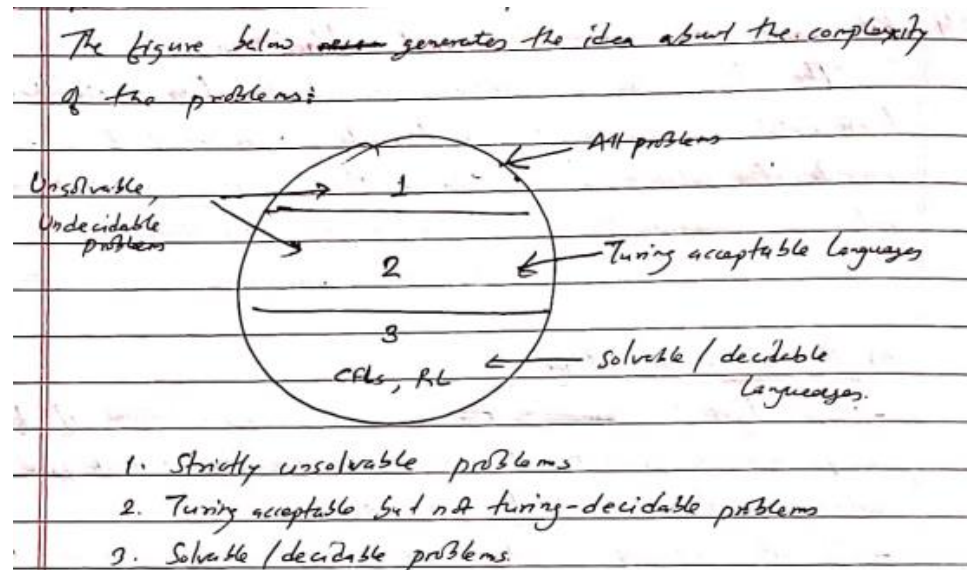
Polynomial-Time Algorithm

- ~~an~~ Algorithms ~~are~~ ~~in~~ ~~that~~ run in polynomial-time if for a problem of size n , the number of steps needed to find the solution is a polynomial function of n .
- Their order-of-magnitude time performance is bounded by a polynomial function of n , where n is the size of its input.
- Polynomial functions are like $O(1)$, $O(\log n)$, $O(n)$, $O(n \times \log n)$, $O(n^2)$, $O(n^3)$, and so on.
- Polynomial algorithms are considered to be efficient because the execution time grows less rapidly as the problem ^{size} increases.

Exponential Algorithm

- Exponential algorithms are not bounded by polynomial-time.
- Exponential functions are like $O(k^n)$, $O(n!)$, $O(2^n)$.
- Exponential functions are considered to be less efficient because the execution times of the latter grow much more rapidly as the problem size increases.

7.5 Intractable and Tractable problems



a) Tractable problems

- A decision problem is tractable if there is an algorithm to solve the given problem and time required is expressed as a polynomial $P(n)$, n being the length of the input string.
- Simply tractable problems are those if there exists an algorithm of polynomial complexity for all instances and solves the problem.
- The P and NP classes of problems fall in tractable problems.

b) Intractable problems

- A problem will be said to be computationally intractable (also computationally complex or computationally hard) if the optimal algorithm for solving the problem cannot solve all of its instances in polynomial time.
- Usually problems are intractable if the time required for any of the algorithm (which can solve the problem) is at least $f(n)$, where f is an exponential function of n .
- NP-hard, and NP-complete problems fall in this category.

Examples of Tractable problems:

1. Searching an unordered list
2. Searching an ordered list
3. Sorting a list
4. Multiplication of integers
5. Finding a minimum spanning tree in a graph

Examples of Intractable problems:

1. Towers of Hanoi
2. TSP

- Problems that are solvable in theory, but cannot be solved in practice are called intractable.

7.6 Tractable Problems: class P and class NP

a) Class P

- A problem which can be solved on polynomial time is known as P-class problem.
- Eg: All sorting and searching algorithms
- P is a class of problems that can be solved deterministically in polynomial time.
- These problems can be solved in time $O(n^k)$ in worst-case, where k is constant.
- The class P is important because:
 - i) it is invariant over all models of computations
 - ii) Practical problems in P-class have efficient (low-degree polynomial) algorithms.
- Class P is also a sub-set of NP class, but not $P=NP$.
- The P-class problems are easy to solve and also easy to verify in polynomial time.

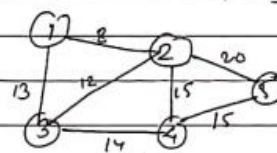
Examples of P-class:

i) Kruskal's Algorithm: Minimum Weight Spanning Tree

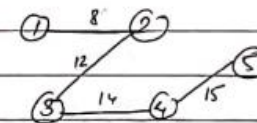
A minimum weight spanning tree is a sub-graph that connects all the nodes (or vertices) together with least possible weight of the edges, yet there are no cycles.

Eg.

XYZ company is providing cable services to 5 new housing developments. The goal is to determine the most economical cable network. Find the minimal spanning tree for below:



The solution is (using Kruskal's algorithm):



This problem is class P problem because it is possible to implement Kruskal's algorithm (using computer) on a graph with m nodes and x edges in time $O(m + x \log x)$.

ii) Problem of finding the maximum number

Let us consider there are 5 numbers: $(n_1, n_2, n_3, n_4, n_5)$. The problem is to find the largest among them. This can be solved as:-

```
int max;
```

```
max = n1;
```

```
if (n2 > max)
```

```
max = n2
```

```
if (n3 > max)
```

```
max = n3
```

```
if (n4 > max)
```

```
max = n4
```

```
if (n5 > max)
```

```
max = n5
```

If there are N numbers in the list, this takes roughly N steps.

N is a polynomial function of N , so this problem is the example of P-class.

b) Class NP

- NP is the set of problems that can be solved in non-deterministic polynomial time. (but in exponential time)
- A problem which cannot be solved in polynomial time, but is verified in polynomial time is called known as Non-deterministic polynomial or NP-class problem.
- For eg: Su-Do-Ku, Prime factor, Scheduling, Travelling Salesman problem.
- For class NP problems, solving the problem is difficult, but verifying it is easy.
- The class NP is also invariant over all reasonable models of computation.

Example: The Su-Do-Ku Problem

Goal: Fill the 9×9 grid with numbers so that each row, column and 3×3 section contains all of the digits between 1 and 9 without repetition.

		8			7		3		5
2	5	7			8		9	6	
					5		7		
4		9	3	6					
	6						4		
3	7			1			2		
	3				1	9		4	
7		5				1	8	2	
		6	4	8	2				

It is very difficult to solve the problem. It takes an exponential time to solve this problem.

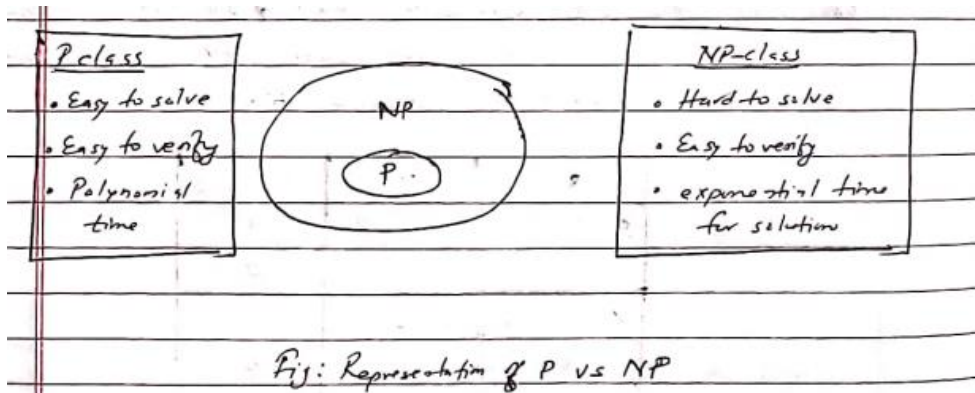
However, it takes very less time to verify if the problem is solved or not.

Hence, it is a class NP problem.

7.7 P-class Vs NP-class

The P class problems, not only it is solved in polynomial time but it is verified also in polynomial time.

The NP class problems cannot be solved in exponential time, but it is verified in polynomial time.



7.8 IS $P=NP$?

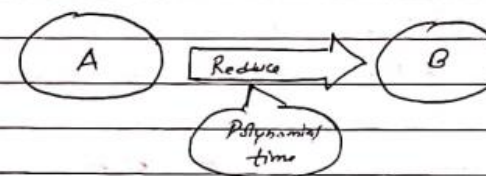
- This is one of the seven unsolved mathematical problems laid out by the Clay Mathematical Institute, each with a \$1 million prize for those who solve them.
- Is $P=NP$? This simply asks whether computationally hard problems actually contain hidden, and computationally easy solutions.
- Proving that $P \neq NP$ would be a major milestone. If $P \neq NP$ then we could prove that there are some problems that can never be solved.
- The majority of modern cryptography relies on codes that are hard to crack but easy to check. The encryption behind securing our credit card number when ordering something on Amazon too is an example of NP cryptography.

- If $P=NP$, then cracking nearly every kind of encryption would suddenly become much, much easier.
- The above is the disadvantage if $P=NP$.
- There would have been advantages ^{also} if $P=NP$.
- Everything would be more efficient such as transportation, scheduling, understanding DNA, etc.
- So if $P=NP$, then the world would be a profoundly different place than we usually assume it to be.

7.9 Intractable Problems: NP-hard and NP-complete

Before discussing on NP-hard and NP-complete classes, let us ~~discuss~~ get into the topic "Polynomial-Time Reduction".

Polynomial Time Reduction



Let A and B are two problems.

- Then A reduces to B ~~iff~~ There is a way to solve A by deterministic algorithm that solve B in polynomial time.
- If A is ^{reducible} reducible to B, we denote it by $A \leq B$

NP-hard and NP-Complete problems:

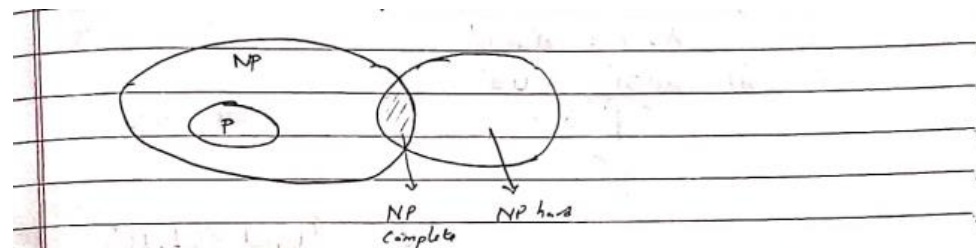


Fig: NP-hard and NP-complete problems.

a) NP-hard

- A problem is NP-hard problem if every problem in NP can be polynomially reduced to it.
- The NP-hard problem may ^{or may not} fall within NP.
- NP hard problems are basically optimization problems.
- NP hard means "at least as hard as any NP-problem".
- Some common examples of NP-hard problems are:-
 1. Hamiltonian Cycle Problem (HCP)
 2. Travelling Salesman Problem (TSP)
 3. Vertex Cover Problem (VCP)
 4. Partition Problem

b) NP-Complete

- A problem is NP-complete if it is NP and it is NP-hard.
- It is a subset of NP-hard (as shown in figure).
- So every NP-complete problems are NP-hard but not vice-versa.
- NP-complete problems are decision problems.
- Some common examples of NP-complete problems are:-

HCP, TSP, VCP, partition problem

End of Chapter 7

